Optimización de la Red Logística de Recepción y Procesamiento de Residuos Agropecuarios utilizando Modelos de Redes y Algoritmos Genéticos

Angie Y. Gutiérrez Guerrero, Cristian F. Muñoz Martínez, Freddy L. Vega Cisneros y Juan N. Cabrera Miranda

Resumen—Este informe presenta el diseño y desarrollo de un sistema de optimización para la red de recepción de residuos agropecuarios de EcoResiduos del Llano S.A.S., formada por 10 empresas aliadas y 4 plantas en el departamento del Meta. La red se modela como un grafo bipartito dirigido y como un problema de transporte cuyo costo unitario cij = α dij + pj + βi integra el transporte, la operación de la planta y la compra del residuo. El modelo se resuelve con un método Simplex de dos fases implementado desde cero, que obtiene un costo óptimo de 120.472.500 COP por semana en 0,61 ms y coincide con el solucionador HiGHS en 201 instancias de prueba. Se diseñó un algoritmo genético con cromosoma matricial, aptitud inversa del costo penalizado por exceso de capacidad, selección por torneo o ruleta, cruce por filas de uno o dos puntos y mutación por transferencia o intercambio en ciclo; el algoritmo alcanza el óptimo en las 30 corridas del caso base. El análisis de sensibilidad identifica como cuellos de botella las plantas de Restrepo, Villavicencio y Puerto Gaitán, con precios sombra de 66.750, 63.000 y 52.000 COP por tonelada, y revela una solución degenerada con óptimos alternativos. La comparación muestra que el método exacto es más rápido y garantiza el óptimo, mientras que el gap del algoritmo genético crece hasta 8,15 % en instancias de 100 empresas y 20 plantas. Se documentan además los requerimientos, la matriz RACI, la arquitectura por capas, los diagramas de clases, de datos y de secuencia, y la interfaz web.

Índice de Términos—Algoritmos genéticos, análisis de sensibilidad, arquitectura por capas, logística de residuos agropecuarios, método Simplex, precios sombra, problema de transporte.

I. Introducción

EcoResiduos del Llano S.A.S. es una empresa del departamento del Meta dedicada a la valorización de residuos agropecuarios (biomasa, subproductos de los cultivos de palma y arroz, estiércol de la ganadería y residuos forestales), que transforma en abonos orgánicos, biocombustibles y suplementos para nutrición animal [33]. La compañía recibe el residuo de una red de 10 empresas agropecuarias aliadas, que generan un volumen constante cada semana, y lo procesa en 4 plantas ubicadas en Villavicencio, Restrepo, Granada y Puerto Gaitán. Hasta ahora la asignación de las rutas entre empresas y plantas se hace de forma empírica, lo que produce sobrecostos de transporte y de operación.

El problema de decidir cuántas toneladas envía cada empresa a cada planta para minimizar el costo total, respetando la oferta de las fuentes y la demanda de los destinos, es el problema clásico de transporte, caso particular del problema de flujo de costo mínimo en redes [1], [2], [7]. Por ser un programa lineal, admite una solución óptima global mediante el método Simplex [5]; sin embargo, la gerencia pidió además evaluar un enfoque metaheurístico, el algoritmo genético (AG) [8], [9], para comparar la calidad de su solución, su tiempo de cómputo y su escalabilidad frente al método exacto, y exigió que el sistema permita modificar los parámetros del negocio y realizar un análisis de sensibilidad detallado.

Este informe documenta el proyecto completo, desde la definición del equipo hasta el análisis comparativo. El objetivo general fue diseñar y desarrollar una solución computacional basada en modelos de optimización de redes, métodos de transporte y algoritmos genéticos para minimizar el costo logístico de la red de recepción de residuos. Los objetivos específicos fueron los siguientes:

1) Estructurar y parametrizar el problema de transporte considerando la oferta de las 10 empresas y la demanda de las 4 plantas.
2) Implementar el modelo matemático exacto para hallar la solución óptima global.
3) Diseñar e implementar un algoritmo genético adaptado a la asignación de flujos de la red.
4) Realizar el análisis de sensibilidad ante variaciones de los costos unitarios, la oferta y la demanda.
5) Comparar el método exacto y el algoritmo genético en calidad de la función objetivo, tiempo de cómputo, escalabilidad y respuesta ante los escenarios de sensibilidad.

Los aportes del trabajo son: un modelo de transporte parametrizable que incluye el costo de compra del residuo, el costo de transporte y el costo de operación de cada planta; una implementación propia del método Simplex de dos fases, sin librerías de optimización, con cálculo de precios sombra y rangos de variación; un AG con cromosoma matricial que conserva la oferta por construcción; y una aplicación web con inicio de sesión, base de datos relacional y reporte en PDF. Con los datos regionales de prueba el método exacto obtiene un

costo óptimo de **120.472.500 COP por semana**, y el AG alcanza ese mismo valor en las 30 corridas del caso base.

El resto del documento se organiza así. La Sección II presenta el equipo, la metodología por fases y la matriz RACI. La Sección III recoge los requerimientos funcionales y no funcionales, las historias de usuario y la trazabilidad. La Sección IV formula el modelo de la red logística (Fase 1). La Sección V describe la solución exacta y su validación (Fase 2). La Sección VI justifica el diseño del algoritmo genético (Fase 3). La Sección VII presenta la arquitectura y el diseño del software (Fase 4). Las Secciones VIII y IX muestran los resultados del análisis de sensibilidad y del análisis comparativo. La Sección X resume las pruebas y la Sección XI presenta las conclusiones.

II. Organización del equipo y metodología

A. Integrantes y roles

El proyecto se desarrolló en un grupo de cuatro integrantes, cada uno con un rol fijo durante todo el ciclo de vida, tal como lo exige el enunciado. La Tabla I resume la asignación formal de roles y sus responsabilidades principales.

TABLA I
INTEGRANTES DEL GRUPO Y ASIGNACIÓN DE ROLES

Integrante	Rol	Responsabilidades principales
Angie Yessenia Gutiérrez Guerrero	Líder de proyecto y analista de negocio	Coordinación general, seguimiento del cronograma, documentación del informe técnico (IEEE) y validación de las restricciones del modelo de negocio.
Cristian Fernando Muñoz Martínez	Ingeniero de modelado y optimización	Formulación matemática de la red (grafo, variables, función objetivo, restricciones) e implementación del módulo con el solucionador exacto.
Freddy Leonardo Vega Cisneros	Desarrollador de metaheurísticas	Diseño, codificación y ajuste de parámetros del algoritmo genético (representación cromosómica, aptitud, selección, cruce y mutación).
Juan Nicolás Cabrera Miranda	Desarrollador de software e interfaz	Arquitectura de software, interfaz de usuario para la parametrización dinámica, módulo de análisis de sensibilidad y reportes comparativos.

B. Metodología por fases

El trabajo siguió las cuatro fases del enunciado, cada una con productos verificables que alimentan a la siguiente (Tabla II). Los requerimientos se levantaron al inicio con plantillas de requerimientos e historias de usuario [28]; el código se organizó en un repositorio Git con ramas por módulo, y las responsabilidades se controlaron con una matriz RACI [29].

TABLA II
FASES DEL PROYECTO Y PRODUCTOS ENTREGADOS

Fase	Alcance	Productos
1. Modelado de la red	Grafo dirigido, parámetros y modelo de programación lineal.	Grafo G = (V, A), matriz de costos unitarios, modelo de transporte balanceado y no balanceado (Sección IV).
2. Solución exacta	Módulo que resuelve el modelo y valida la solución.	Simplex de dos fases propio, verificación de factibilidad y verificación independiente (Sección V).
3. Metaheurística	Algoritmo genético adaptado al problema.	Cromosoma matricial, aptitud penalizada, operadores, múltiples corridas (Sección VI).
4. Interfaz, sensibilidad y comparación	Aplicación web, análisis de sensibilidad y reporte comparativo.	Arquitectura por capas, base de datos, 12 vistas, precios sombra, rangos, simulación y reporte PDF (Secciones VII a IX).

C. Matriz de responsabilidades RACI

La matriz RACI asigna a cada actividad un responsable que la ejecuta (R), un aprobador que rinde cuentas (A), los integrantes consultados (C) y los informados (I); cada actividad tiene un único R y un único A distinto del R (Tabla III). La matriz cubre 63 actividades agrupadas en planificación, las cuatro fases, los entregables y la gestión transversal, y está alineada con los 11 HU, 19 RF y 13 RNF del documento de requerimientos.

TABLA III
CONVENCIONES DE LA MATRIZ RACI

Letra	Rol	Significado
R	Responsable	Ejecuta directamente la tarea; solo existe un R por actividad.
A	Aprobador	Rinde cuentas y aprueba el resultado; solo existe un A por actividad y es distinto del R.
C	Consultado	Aporta criterio o retroalimentación; comunicación bidireccional.
I	Informado	Es notificado del avance o del resultado; comunicación unidireccional.

Con la elaboración de este informe la matriz se actualizó: todas las actividades quedaron completadas, excepto la 60, la sustentación práctica con la demostración funcional ante el docente, que se realiza después de la entrega de este informe. La Tabla IV presenta la matriz completa con el estado actualizado.

TABLA IV
MATRIZ RACI DEL PROYECTO (ESTADO ACTUALIZADO)

#	Actividad y requisitos asociados	AG	CM	FV	JC	Estado
PLANIFICACIÓN Y GESTIÓN INICIAL | Requerimientos de proyecto, restricciones y datos base						
1	Conformación del grupo y asignación formal de roles *Req. de proyecto*	R	A	C	C	Completado
2	Análisis del enunciado y del documento de requerimientos (RF01–RF19, RNF01–RNF13, HU-01–HU-11) y matriz de trazabilidad *Req. de proyecto*	R	C	C	A	Completado
3	Repositorio Git: estructura modular (exacto, genético, interfaz, sensibilidad) y ramas por módulo *RNF04 · RNF08*	R	I	I	A	Completado
4	Entorno Python y archivo requirements con librerías de código abierto (Streamlit, NumPy, NetworkX, SQLAlchemy, Plotly, fpdf2), sin librerías de optimización; entorno web *RNF05 · Restricción tecnológica*	I	R	C	A	Completado
5	Levantamiento de datos regionales de prueba (10 empresas, 4 plantas, distancias estimadas y costo por km-tonelada) *RF06 · Riesgo: datos*	R	A	C	C	Completado
6	Validación de supuestos del modelo (residuo homogéneo, costos lineales, flota no restrictiva) y de restricciones legales/licencias *Restricciones del modelo*	R	A	C	I	Completado
FASE 1 — MODELADO DE LA RED LOGÍSTICA						
7	Definición formal del grafo dirigido G = (V, E): fuentes, nodos de paso y plantas; arcos con distancia y costo *RF03 · RF07*	A	R	C	I	Completado
8	Definición de conjuntos y parámetros (oferta, demanda/capacidad, costos unitarios de transporte) *RF01 · RF02 · RF03 · RF04*	A	R	C	I	Completado

#	Actividad y requisitos asociados	AG	CM	FV	JC	Estado
9	Definición de variables de decisión y función objetivo (costo de transporte, precio de la tonelada y costos de operación por planta) *RF05 · RF09*	A	R	C	I	Completado
10	Formulación de restricciones de oferta, demanda y no negatividad *RF09*	A	R	C	I	Completado
11	Balance oferta–demanda: criterio de nodo ficticio (fuente o destino) cuando el problema está desbalanceado *HU-03 · RF08 · RNF06 · Riesgo: desbalance*	A	R	C	I	Completado
12	Diseño del esquema de datos parametrizable (red ampliable a ≥100 fuentes y 20 plantas sin modificar código) *RF01 · RF02 · RNF02 · RNF10*	C	R	C	A	Completado
13	Redacción de la formulación matemática detallada en el informe técnico *Informe IEEE*	R	A	C	I	Completado
FASE 2 — SOLUCIÓN EXACTA | HU-06						
14	Selección de solucionador exacto (Simplex) por construcción propia *HU-06*	A	R	I	C	Completado
15	Implementación del módulo exacto: genera el modelo PL desde los datos vigentes y entrega costo total, flujo por arco y tiempo *HU-06 · RF09 · RNF01 · RNF04*	A	R	C	C	Completado
16	Manejo de estados de solución (óptima, infactible, no acotada) y validación de entradas sin cierres inesperados *HU-06 · RNF09*	A	R	I	C	Completado
17	Verificación de factibilidad de soluciones (oferta, demanda, no negatividad; tolerancia 1e-6), aplicable al exacto y al AG *HU-06 · RF10 · RNF06*	C	R	A	I	Completado
18	Verificación independiente del modelo exacto con casos pequeños (cálculo manual o método alterno) *RNF06*	I	A	R	I	Completado
19	Documentación del código del módulo exacto *RNF04 · RNF08*	A	R	I	I	Completado
FASE 3 — METAHEURÍSTICA: ALGORITMO GENÉTICO | HU-07						
20	Diseño de la codificación del cromosoma (asignación de flujos fuente → planta) *HU-07 · RF12*	A	C	R	I	Completado
21	Diseño de la función de aptitud (inverso del costo total penalizado por violación de capacidad) *HU-07 · RF12*	C	A	R	I	Completado
22	Operadores de selección (ruleta / torneo), cruce (uno / dos puntos) y mutación *HU-07 · RF11 · RF12*	A	I	R	I	Completado
23	Generación de población inicial y mecanismo de reparación de soluciones infactibles *RF12*	I	A	R	I	Completado
24	Semilla aleatoria configurable y registro de parámetros utilizados (reproducibilidad) *HU-07 · RF11 · RNF07*	A	I	R	C	Completado
25	Implementación del AG: mejor solución, costo, tiempo de ejecución y curva de convergencia (< 60 s con parámetros por defecto) *HU-07 · RF12 · RNF01 · RNF04*	A	C	R	C	Completado
26	Ejecución de múltiples corridas y estadísticas (mejor, promedio, desviación estándar, tiempo promedio) *HU-07 · RF13 · RNF07*	A	I	R	C	Completado
27	Ajuste de parámetros (población, generaciones, cruce, mutación) para evitar convergencia prematura *HU-07 · RF11 · Riesgo: AG inestable*	I	A	R	C	Completado
28	Redacción de la justificación del diseño del AG en el informe técnico *Informe IEEE*	R	C	A	I	Completado
FASE 4 — INTERFAZ, ANÁLISIS DE SENSIBILIDAD Y COMPARACIÓN | HU-01 a HU-05 y HU-07 a HU-11						
29	Arquitectura de software: módulos independientes con bajo acoplamiento (exacto, genético, interfaz, sensibilidad) *RNF04*	A	C	C	R	Completado
30	Diseño de la interfaz: etiquetas claras, valores por defecto, ayudas, legible en 1366x768 *RNF03 · RNF11*	A	C	I	R	Completado
31	HU-01 · Gestión de empresas agropecuarias y plantas (alta, edición, baja con confirmación; ampliación sin reiniciar) *HU-01 · RF01 · RF02 · RNF02 · RNF09*	A	C	I	R	Completado
32	HU-02 · Gestión de costos, distancias y coeficientes (distancia editable por ruta, costo de transporte y su factor, precio por tonelada de cada empresa, costo de operación de cada planta) *HU-02 · RF03 · RF05 · RNF03*	C	A	I	R	Completado
33	HU-03 · Gestión de oferta y demanda (totales, diferencia, advertencia de desbalance y nodo ficticio) *HU-03 · RF04 · RF08*	C	A	I	R	Completado
34	HU-04 · Guardar, duplicar y recuperar escenarios con sus parámetros y resultados; datos regionales del caso base precargados *HU-04 · RF06 · RF17 · RNF10*	A	I	I	R	Completado
35	HU-05 · Visualización gráfica de la red logística (nodos, arcos, costos y datos al seleccionar) *HU-05 · RF07 · RNF11*	A	C	I	R	Completado
36	Interfaz de configuración y ejecución del AG (población, generaciones, cruce, mutación, selección, tipo de cruce, semilla) *HU-07 · RF11 · RNF03*	I	I	A	R	Completado
37	HU-08 · Visualización de resultados: tabla y gráfico de flujos, costo total y utilización de cada planta *HU-08 · RF14 · RNF11*	A	C	C	R	Completado
38	Visualización de la curva de convergencia y de las estadísticas de múltiples corridas del AG *HU-07 · RF13 · RF14*	I	I	A	R	Completado
39	HU-09 · Cálculo de precios sombra, rangos de variación sin cambio de base y restricciones activas (cuellos de botella); manejo de duales degenerados *HU-09 · RF15 · RNF06 · Riesgo: duales*	C	R	I	A	Completado
40	HU-09 · Presentación de la sensibilidad en tablas y gráficos interpretables *HU-09 · RF15 · RNF11*	C	A	I	R	Completado
41	HU-10 · Motor de simulación de escenarios (variación % de costo y oferta, reducción de capacidad de plantas; datos originales intactos) *HU-10 · RF16 · RNF10*	C	R	C	A	Completado
42	HU-10 · Ejecución del método exacto y del AG sobre cada escenario y comparación entre escenarios *HU-10 · RF16*	C	A	R	C	Completado
43	HU-10 · Guardar las simulaciones como escenarios derivados y compararlos entre sí *HU-10 · RF17 · RNF10*	A	I	I	R	Completado
44	HU-11 · Reporte comparativo exacto vs. AG: costo total, tiempo, gap (%) y comportamiento por escenario *HU-11 · RF18 · RNF06 · RNF11*	A	C	C	R	Completado
45	Exportación de resultados, sensibilidad y reporte comparativo en PDF *HU-08 · HU-11 · RF19 · RNF13*	A	I	I	R	Completado
46	Integración de los módulos exacto + AG + interfaz + sensibilidad *RNF04 · Riesgo: integración*	A	C	C	R	Completado
47	Pruebas de rendimiento (exacto < 5 s en 10x4; AG < 60 s; interfaz < 2 s) *RNF01*	A	C	R	C	Completado
48	Pruebas de escalabilidad (≥ 100 fuentes y 20 plantas, solo con cambio de datos) *RNF02*	C	R	A	C	Completado
49	Pruebas de usabilidad y de manejo de errores (entradas inválidas, campos vacíos, problemas infactibles) *RNF03 · RNF09*	R	C	C	A	Completado
50	Modo demostración sin internet con el caso base precargado y escenarios derivados de distintos niveles de oferta y demanda *RNF12*	A	C	C	R	Completado
51	Análisis comparativo: calidad de la función objetivo, tiempo, escalabilidad y respuesta ante sensibilidad *HU-11 · RF18*	R	C	A	C	Completado
ENTREGABLES — INFORME IEEE, CÓDIGO FUENTE Y SUSTENTACIÓN						
52	Redacción de los resultados del análisis de sensibilidad en el informe *Informe IEEE · HU-09*	R	A	C	C	Completado
53	Diagramas de arquitectura del sistema para el informe *Informe IEEE · RNF04*	A	C	C	R	Completado
54	Redacción e integración del Informe Técnico (formato IEEE) *Informe IEEE · RNF08*	R	A	C	C	Completado

#	Actividad y requisitos asociados	AG	CM	FV	JC	Estado
55	Revisión final, control de calidad y formato IEEE del informe *Informe IEEE · RNF08*	A	C	R	C	Completado
56	Documentación del código fuente e instrucciones de instalación y ejecución *RNF08*	A	C	C	R	Completado
57	Entrega del repositorio Git final (código modularizado y documentado) *RNF08*	R	C	C	A	Completado
58	Preparación de la sustentación (guion, escenarios de demostración y casos de prueba) *RNF12*	R	C	A	C	Completado
59	Ensayo de la demostración funcional y de la ejecución en vivo del análisis de sensibilidad *RNF12*	A	C	R	C	Completado
60	Sustentación práctica (demostración funcional ante el docente) *RNF12*	R	A	C	C	Pendiente
GESTIÓN TRANSVERSAL | durante todo el ciclo de vida del proyecto						
61	Seguimiento del cronograma y reuniones de avance del equipo *Req. de proyecto*	R	C	C	A	Completado
62	Gestión de riesgos (datos poco confiables, desbalance, AG inestable, duales degenerados, integración, tiempo) *Riesgos del proyecto*	R	C	A	C	Completado
63	Revisión de código y fusión de ramas por módulo *RNF04 · RNF08*	C	R	A	C	Completado

AG: Angie Gutiérrez (líder y analista). CM: Cristian Muñoz (ingeniero de modelado). FV: Freddy Vega (metaheurísticas). JC: Juan Cabrera (software e interfaz).

La Fig. 1 resume el avance: 62 de 63 actividades completadas (98,4 %) y 1 pendiente, la sustentación práctica. La Fig. 2 muestra la carga por integrante: los cuatro tienen entre 13 y 18 actividades como responsables, lo que indica una distribución equilibrada del trabajo.

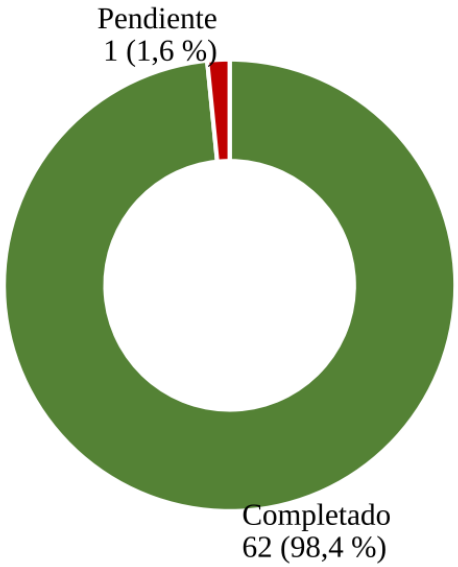


Fig. 1. Estado de las 63 actividades de la matriz RACI después de la actualización.

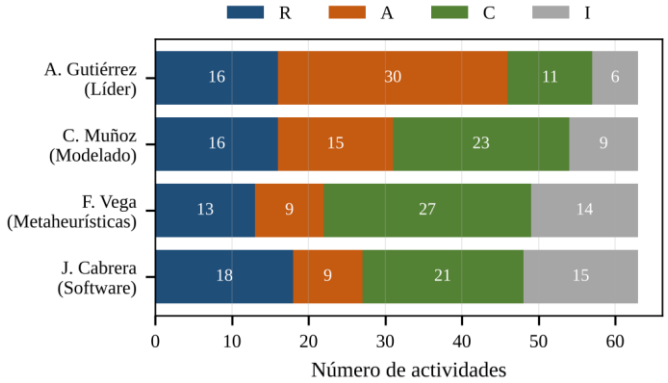


Fig. 2. Número de actividades por integrante y por letra RACI.

III. Requerimientos del Sistema

Los requerimientos se obtuvieron del dominio del problema y del enunciado, siguiendo la distinción entre requerimientos funcionales, que describen los servicios del sistema, y no funcionales, que restringen su calidad y su operación [24]. Se identificaron 19 requerimientos funcionales (RF), 13 no funcionales (RNF) y 11 historias de usuario (HU), además de los requerimientos de proyecto, las restricciones y los riesgos.

A. Identificación de requerimientos

Los requerimientos se documentaron con la plantilla del documento de requerimientos del proyecto, que se conserva en este informe. La Tabla V resume los requerimientos funcionales, los no funcionales y los demás elementos que condicionaron el desarrollo: requerimientos de proyecto, restricciones y riesgos. Los riesgos se trataron explícitamente en el diseño: el desbalance se resolvió con nodos ficticios, la inestabilidad del AG con elitismo, ajuste de parámetros y múltiples corridas, y la degeneración de los duales con un aviso al usuario en el módulo de sensibilidad.

TABLA V
IDENTIFICACIÓN DE REQUERIMIENTOS

REQUERIMIENTOS FUNCIONALES	REQUERIMIENTOS NO FUNCIONALES
• **RF01.** Gestionar empresas agropecuarias (fuentes) • **RF02.** Gestionar plantas de recepción y transformación (destinos) • **RF03.** Gestionar rutas, distancias y costos de transporte • **RF04.** Gestionar oferta y demanda/capacidad • **RF05.** Parametrizar coeficientes del modelo de negocio • **RF06.** Cargar los datos regionales de prueba • **RF07.** Construir y visualizar la red logística • **RF08.** Validar consistencia y balance del problema • **RF09.** Resolver el modelo exacto de transporte / costo mínimo • **RF10.** Verificar factibilidad de las soluciones • **RF11.** Configurar parámetros del Algoritmo Genético • **RF12.** Ejecutar el Algoritmo Genético • **RF13.** Ejecutar múltiples corridas del AG y estadísticas • **RF14.** Visualizar resultados de optimización • **RF15.** Realizar análisis de sensibilidad • **RF16.** Simular escenarios de variación. • **RF17.** Guardar y recuperar escenarios • **RF18.** Generar reporte comparativo exacto vs. AG • **RF19.** Exportar reportes	• **RNF01.** Rendimiento y tiempo de respuesta • **RNF02.** Escalabilidad • **RNF03.** Usabilidad • **RNF04.** Modularidad y mantenibilidad • **RNF05.** Portabilidad y tecnología • **RNF06.** Exactitud y corrección numérica • **RNF07.** Reproducibilidad del AG • **RNF08.** Documentación y control de versiones • **RNF09.** Confiabilidad y manejo de errores • **RNF10.** Integridad de los datos • **RNF11.** Claridad de la visualización • **RNF12.** Capacidad de demostración • **RNF13.** Interoperabilidad de formatos
OTROS ELEMENTOS A CONSIDERAR: REQUERIMIENTOS DE PROYECTO, RESTRICCIONES, RIESGOS	
Requerimientos de proyecto • Trabajo en grupos de 4 integrantes, con roles definidos durante todo el ciclo de desarrollo. • Entregables: informe técnico en formato IEEE, código fuente documentado en repositorio Git y sustentación práctica con demostración en vivo. • Desarrollo por fases: (1) modelado de la red, (2) solución exacta, (3) metaheurística, (4) interfaz, sensibilidad y comparación. • Uso de control de versiones y ramas por módulo; pruebas con datos regionales de prueba. **Restricciones** • Tecnológicas: Python y librerías de código abierto, sin licencias comerciales; el Simplex, el Algoritmo Genético y el análisis de sensibilidad se implementan desde cero, sin librerías de optimización. • Presupuestarias: sin presupuesto asignado; se usan equipos personales del grupo. • Datos: las distancias y costos son estimaciones; la red base se limita a 10 fuentes y 4 plantas. • Legales: respeto de la propiedad intelectual y licencias de las librerías utilizadas. **Riesgos** • Datos reales de oferta, demanda y costos no disponibles o poco confiables; se mitiga con datos de prueba y análisis de sensibilidad. • Desbalance entre oferta y demanda que genere infactibilidad; se mitiga con nodos ficticios y validación previa. • Convergencia prematura o resultados inestables del AG; se mitiga con ajuste de parámetros y múltiples corridas. • Duales degenerados o no únicos que dificulten la interpretación de la sensibilidad. • Poco tiempo del semestre y disponibilidad desigual de los integrantes.	

B. Requerimientos funcionales

Las Tablas VI a XXIV describen cada requerimiento funcional con la plantilla: identificación, nombre, descripción, requerimientos no funcionales relacionados y prioridad.

TABLA VI
REQUERIMIENTO FUNCIONAL RF01

Identificación del Requerimiento:	RF01
Nombre del Requerimiento:	Gestionar empresas agropecuarias (fuentes)
Descripción del Requerimiento:	El sistema permitirá crear, consultar, modificar y eliminar las empresas agropecuarias aliadas (nodos fuente), registrando nombre, municipio y tipo de residuo. Se cargarán inicialmente las 10 empresas y el sistema debe permitir ampliar el número sin modificar el código.
Requerimiento No Funcional:	• RNF02 – Escalabilidad • RNF03 – Usabilidad • RNF09 – Confiabilidad y manejo de errores
Prioridad del Requerimiento: Alta	

TABLA VII
REQUERIMIENTO FUNCIONAL RF02

Identificación del Requerimiento:	RF02
Nombre del Requerimiento:	Gestionar plantas de recepción y transformación (destinos)
Descripción del Requerimiento:	El sistema permitirá crear, consultar, modificar y eliminar plantas (nodos destino). Se cargarán inicialmente las 4 plantas (Villavicencio, Restrepo, Granada y Puerto Gaitán) y se podrán añadir nuevas sedes.
Requerimiento No Funcional:	• RNF02 – Escalabilidad • RNF03 – Usabilidad • RNF09 – Confiabilidad y manejo de errores
Prioridad del Requerimiento: Alta	

TABLA VIII
REQUERIMIENTO FUNCIONAL RF03

Identificación del Requerimiento:	RF03
Nombre del Requerimiento:	Gestionar rutas, distancias y costos de transporte
Descripción del Requerimiento:	El sistema permitirá ingresar y modificar de forma dinámica las distancias estimadas (km) de cada arco (fuente, planta) y el costo de transporte por tonelada-kilómetro, y calculará a partir de ellos la matriz de costos unitarios de la red.
Requerimiento No Funcional:	• RNF03 – Usabilidad • RNF09 – Confiabilidad y manejo de errores
Prioridad del Requerimiento: Alta	

TABLA IX
REQUERIMIENTO FUNCIONAL RF04

Identificación del Requerimiento:	RF04
Nombre del Requerimiento:	Gestionar oferta y demanda/capacidad
Descripción del Requerimiento:	El sistema permitirá definir la oferta semanal de residuos (ton) de cada empresa y la demanda o capacidad de procesamiento (ton) de cada planta, y modificar estos valores en cualquier momento.
Requerimiento No Funcional:	• RNF03 – Usabilidad • RNF09 – Confiabilidad y manejo de errores
Prioridad del Requerimiento: Alta	

TABLA X
REQUERIMIENTO FUNCIONAL RF05

Identificación del Requerimiento:	RF05
Nombre del Requerimiento:	Parametrizar coeficientes del modelo de negocio
Descripción del Requerimiento:	El sistema permitirá ajustar los coeficientes del modelo: el costo de transporte por tonelada-kilómetro (que incluye el combustible) y su factor de ajuste porcentual, el precio por tonelada que se paga a cada empresa y el costo de operación de cada planta, recalculando los costos unitarios del modelo.
Requerimiento No Funcional:	• RNF03 – Usabilidad • RNF09 – Confiabilidad y manejo de errores
Prioridad del Requerimiento: Alta	

TABLA XI
REQUERIMIENTO FUNCIONAL RF06

Identificación del Requerimiento:	RF06
Nombre del Requerimiento:	Cargar los datos regionales de prueba
Descripción del Requerimiento:	El sistema incluirá precargados los datos regionales de prueba del caso base (10 empresas, 4 plantas, distancias, ofertas, demandas, precios y costos de operación), cargados con el script de la base de datos, para poder usarlo y demostrarlo desde la primera ejecución.
Requerimiento No Funcional:	• RNF09 – Confiabilidad y manejo de errores • RNF13 – Interoperabilidad de formatos
Prioridad del Requerimiento: Baja	

TABLA XII
REQUERIMIENTO FUNCIONAL RF07

Identificación del Requerimiento:	RF07
Nombre del Requerimiento:	Construir y visualizar la red logística
Descripción del Requerimiento:	El sistema construirá el grafo dirigido G = (V, E) con nodos fuente, nodos de paso (si aplica) y nodos destino, y lo mostrará gráficamente con sus arcos, distancias y costos asociados.
Requerimiento No Funcional:	• RNF02 – Escalabilidad • RNF11 – Claridad de la visualización
Prioridad del Requerimiento: Media	

TABLA XIII
REQUERIMIENTO FUNCIONAL RF08

Identificación del Requerimiento:	RF08
Nombre del Requerimiento:	Validar consistencia y balance del problema
Descripción del Requerimiento:	El sistema verificará que los datos sean válidos (valores no negativos, completos) y evaluará el balance entre oferta total y demanda total; si están desbalanceados, notificará al usuario y aplicará un nodo ficticio (fuente o destino) para equilibrar el problema.
Requerimiento No Funcional:	• RNF06 – Exactitud y corrección numérica • RNF09 – Confiabilidad y manejo de errores
Prioridad del Requerimiento: Alta	

TABLA XIV
REQUERIMIENTO FUNCIONAL RF09

Identificación del Requerimiento:	RF09
Nombre del Requerimiento:	Resolver el modelo exacto de transporte / costo mínimo
Descripción del Requerimiento:	El sistema implementará el modelo de programación lineal (conjuntos, parámetros, variables de decisión, función objetivo de minimización y restricciones de oferta, demanda y no negatividad) y lo resolverá con un solucionador exacto, entregando el flujo óptimo por arco y el costo total.
Requerimiento No Funcional:	• RNF01 – Rendimiento y tiempo de respuesta • RNF04 – Modularidad y mantenibilidad • RNF06 – Exactitud y corrección numérica
Prioridad del Requerimiento: Alta	

TABLA XV
REQUERIMIENTO FUNCIONAL RF10

Identificación del Requerimiento:	RF10
Nombre del Requerimiento:	Verificar factibilidad de las soluciones
Descripción del Requerimiento:	El sistema validará que toda solución (exacta o del algoritmo genético) cumpla las restricciones de oferta, demanda y no negatividad, y reportará las violaciones encontradas, si las hay.
Requerimiento No Funcional:	• RNF06 – Exactitud y corrección numérica
Prioridad del Requerimiento: Alta	

TABLA XVI
REQUERIMIENTO FUNCIONAL RF11

Identificación del Requerimiento:	RF11
Nombre del Requerimiento:	Configurar parámetros del Algoritmo Genético
Descripción del Requerimiento:	El sistema permitirá definir desde la interfaz el tamaño de población, número de generaciones, tasa de cruce, tasa de mutación, método de selección (ruleta o torneo), tipo de cruce (uno o dos puntos) y semilla aleatoria.
Requerimiento No Funcional:	• RNF03 – Usabilidad • RNF07 – Reproducibilidad del AG
Prioridad del Requerimiento: Alta	

TABLA XVII
REQUERIMIENTO FUNCIONAL RF12

Identificación del Requerimiento:	RF12
Nombre del Requerimiento:	Ejecutar el Algoritmo Genético
Descripción del Requerimiento:	El sistema ejecutará un AG con codificación cromosómica de la asignación de flujos fuente-planta, función de aptitud igual a la inversa del costo total penalizada por violación de restricciones de capacidad, y operadores de selección, cruce y mutación; devolverá la mejor solución, el costo, el tiempo y la curva de convergencia.
Requerimiento No Funcional:	• RNF01 – Rendimiento y tiempo de respuesta • RNF04 – Modularidad y mantenibilidad • RNF07 – Reproducibilidad del AG
Prioridad del Requerimiento: Alta	

TABLA XVIII
REQUERIMIENTO FUNCIONAL RF13

Identificación del Requerimiento:	RF13
Nombre del Requerimiento:	Ejecutar múltiples corridas del AG y estadísticas
Descripción del Requerimiento:	Dado que el AG es estocástico, el sistema permitirá ejecutar N corridas independientes y calcular el mejor costo, el costo promedio, la desviación estándar y el tiempo promedio.
Requerimiento No Funcional:	• RNF01 – Rendimiento y tiempo de respuesta • RNF07 – Reproducibilidad del AG
Prioridad del Requerimiento: Media	

TABLA XIX
REQUERIMIENTO FUNCIONAL RF14

Identificación del Requerimiento:	RF14
Nombre del Requerimiento:	Visualizar resultados de optimización
Descripción del Requerimiento:	El sistema mostrará los flujos óptimos (tabla y gráfico sobre la red), el costo total, la asignación de cada empresa a las plantas y la utilización de capacidad de cada planta, tanto para el método exacto como para el AG.
Requerimiento No Funcional:	• RNF03 – Usabilidad • RNF11 – Claridad de la visualización
Prioridad del Requerimiento: Alta	

TABLA XX
REQUERIMIENTO FUNCIONAL RF15

Identificación del Requerimiento:	RF15
Nombre del Requerimiento:	Realizar análisis de sensibilidad
Descripción del Requerimiento:	El sistema calculará los precios sombra (variables duales) de las restricciones de oferta y demanda, y los rangos de variación permitidos de costos, ofertas y demandas dentro de los cuales la base óptima no cambia.
Requerimiento No Funcional:	• RNF01 – Rendimiento y tiempo de respuesta • RNF06 – Exactitud y corrección numérica
Prioridad del Requerimiento: Alta	

TABLA XXI
REQUERIMIENTO FUNCIONAL RF16

Identificación del Requerimiento:	RF16
Nombre del Requerimiento:	Simular escenarios de variación
Descripción del Requerimiento:	El sistema permitirá simular escenarios con cambios en el costo de combustible/transporte, variaciones porcentuales en la oferta de residuos y cuellos de botella en la demanda de las plantas, y ejecutar ambos métodos sobre cada escenario.
Requerimiento No Funcional:	• RNF01 – Rendimiento y tiempo de respuesta • RNF02 – Escalabilidad • RNF03 – Usabilidad
Prioridad del Requerimiento: Alta	

TABLA XXII
REQUERIMIENTO FUNCIONAL RF17

Identificación del Requerimiento:	RF17
Nombre del Requerimiento:	Guardar y recuperar escenarios
Descripción del Requerimiento:	El sistema permitirá guardar con nombre un conjunto de parámetros y resultados (escenario) y recuperarlo posteriormente para consultarlo o compararlo.
Requerimiento No Funcional:	• RNF09 – Confiabilidad y manejo de errores • RNF13 – Interoperabilidad de formatos
Prioridad del Requerimiento: Media	

TABLA XXIII
REQUERIMIENTO FUNCIONAL RF18

Identificación del Requerimiento:	RF18
Nombre del Requerimiento:	Generar reporte comparativo exacto vs. AG
Descripción del Requerimiento:	El sistema generará un reporte que muestre el costo total de cada método, el tiempo de ejecución, el porcentaje de desviación (gap) del AG respecto al óptimo matemático y el comportamiento de ambos enfoques ante los escenarios del análisis de sensibilidad.
Requerimiento No Funcional:	• RNF06 – Exactitud y corrección numérica • RNF11 – Claridad de la visualización
Prioridad del Requerimiento: Alta	

TABLA XXIV
REQUERIMIENTO FUNCIONAL RF19

Identificación del Requerimiento:	RF19
Nombre del Requerimiento:	Exportar reportes
Descripción del Requerimiento:	El sistema permitirá exportar los resultados, el análisis de sensibilidad y el reporte comparativo en un documento PDF, para su inclusión en el informe técnico.
Requerimiento No Funcional:	• RNF13 – Interoperabilidad de formatos
Prioridad del Requerimiento: Media	

C. Requerimientos no funcionales

Las Tablas XXV a XXXVII describen cada requerimiento no funcional con la misma plantilla, sin la fila de requerimientos relacionados.

TABLA XXV
REQUERIMIENTO NO FUNCIONAL RNF01

Identificación del Requerimiento:	RNF01
Nombre del Requerimiento:	Rendimiento y tiempo de respuesta
Descripción del Requerimiento:	El método exacto debe resolver la instancia base (10 fuentes x 4 plantas) en menos de 5 segundos. El AG con parámetros por defecto debe finalizar en menos de 60 segundos. La interfaz debe responder a las acciones del usuario en menos de 2 segundos (excluyendo ejecuciones de optimización).
Prioridad del Requerimiento: Alta	

TABLA XXVI
REQUERIMIENTO NO FUNCIONAL RNF02

Identificación del Requerimiento:	RNF02
Nombre del Requerimiento:	Escalabilidad
Descripción del Requerimiento:	El sistema debe soportar la ampliación de empresas y sedes (al menos 100 fuentes y 20 plantas) sin modificar el código fuente, solo con cambios de datos, y mantener tiempos de respuesta aceptables.
Prioridad del Requerimiento: Alta	

TABLA XXVII
REQUERIMIENTO NO FUNCIONAL RNF03

Identificación del Requerimiento:	RNF03
Nombre del Requerimiento:	Usabilidad
Descripción del Requerimiento:	La interfaz debe permitir parametrizar datos y ejecutar los métodos sin conocimientos de programación, con etiquetas claras, valores por defecto, mensajes de ayuda y retroalimentación visible del progreso.
Prioridad del Requerimiento: Alta	

TABLA XXVIII
REQUERIMIENTO NO FUNCIONAL RNF04

Identificación del Requerimiento:	RNF04
Nombre del Requerimiento:	Modularidad y mantenibilidad
Descripción del Requerimiento:	El código debe separarse en módulos independientes (exacto, genético, interfaz y análisis de sensibilidad), con bajo acoplamiento, nombres descriptivos y comentarios, de modo que cada integrante pueda trabajar su módulo sin afectar los demás.
Prioridad del Requerimiento: Alta	

TABLA XXIX
REQUERIMIENTO NO FUNCIONAL RNF05

Identificación del Requerimiento:	RNF05
Nombre del Requerimiento:	Portabilidad y tecnología
Descripción del Requerimiento:	El sistema se desarrollará en Python implementando el algoritmo Simplex por construcción propia para el modelo exacto y analisis de sensibilidad orientado a un entorno web y responsive con bases de datos PostgreSQL con las dependencias listadas en un archivo requirements.
Prioridad del Requerimiento: Media	

TABLA XXX
REQUERIMIENTO NO FUNCIONAL RNF06

Identificación del Requerimiento:	RNF06
Nombre del Requerimiento:	Exactitud y corrección numérica
Descripción del Requerimiento:	Las soluciones deben satisfacer las restricciones con una tolerancia máxima de 1e-6, y los resultados del modelo exacto deben coincidir con una verificación independiente (cálculo manual) para casos de prueba pequeños.
Prioridad del Requerimiento: Alta	

TABLA XXXI
REQUERIMIENTO NO FUNCIONAL RNF07

Identificación del Requerimiento:	RNF07
Nombre del Requerimiento:	Reproducibilidad del AG
Descripción del Requerimiento:	Las ejecuciones del AG deben ser reproducibles mediante una semilla aleatoria configurable, registrando los parámetros utilizados junto con los resultados.
Prioridad del Requerimiento: Media	

TABLA XXXII
REQUERIMIENTO NO FUNCIONAL RNF08

Identificación del Requerimiento:	RNF08
Nombre del Requerimiento:	Documentación y control de versiones
Descripción del Requerimiento:	El código fuente debe estar en un repositorio Git, documentado e incluir instrucciones claras de instalación y ejecución; el informe técnico seguirá el formato IEEE.
Prioridad del Requerimiento: Alta	

TABLA XXXIII
REQUERIMIENTO NO FUNCIONAL RNF09

Identificación del Requerimiento:	RNF09
Nombre del Requerimiento:	Confiabilidad y manejo de errores
Descripción del Requerimiento:	El sistema debe validar las entradas del usuario (tipos, rangos, campos vacíos), manejar problemas infactibles o no acotados sin cerrarse inesperadamente y mostrar mensajes de error claros.
Prioridad del Requerimiento: Alta	

TABLA XXXIV
REQUERIMIENTO NO FUNCIONAL RNF10

Identificación del Requerimiento:	RNF10
Nombre del Requerimiento:	Integridad de los datos
Descripción del Requerimiento:	Los datos y escenarios guardados no deben corromperse ni perderse al cerrar el sistema o ante errores de ejecución, y los datos originales no deben alterarse al simular escenarios.
Prioridad del Requerimiento: Media	

TABLA XXXV
REQUERIMIENTO NO FUNCIONAL RNF11

Identificación del Requerimiento:	RNF11
Nombre del Requerimiento:	Claridad de la visualización
Descripción del Requerimiento:	Las gráficas y tablas (red, flujos, convergencia, sensibilidad, comparación) deben tener títulos, ejes y unidades rotuladas, y ser legibles en pantallas de resolución estándar (1366x768).
Prioridad del Requerimiento: Media	

TABLA XXXVI
REQUERIMIENTO NO FUNCIONAL RNF12

Identificación del Requerimiento:	RNF12
Nombre del Requerimiento:	Capacidad de demostración
Descripción del Requerimiento:	El sistema debe poder ejecutarse en vivo durante la sustentación, con el caso base precargado y escenarios derivados creados desde la aplicación, sin depender de conexión a internet.
Prioridad del Requerimiento: Alta	

TABLA XXXVII
REQUERIMIENTO NO FUNCIONAL RNF13

Identificación del Requerimiento:	RNF13
Nombre del Requerimiento:	Interoperabilidad de formatos
Descripción del Requerimiento:	El sistema debe usar formatos estándar y abiertos: el reporte se exporta en PDF y los datos de la red se definen en un script SQL con codificación UTF-8.
Prioridad del Requerimiento: Media	

D. Historias de usuario

Cada historia de usuario se redactó con la forma «como… quiero… para…» y con criterios de aceptación verificables [28]. Las Tablas XXXVIII a XLVIII presentan las 11 historias con la plantilla del proyecto: código, usuario, título, requerimientos funcionales asociados, prioridad, tipo, descripción y criterios de aceptación. Durante el diseño, algunas historias se dividieron en escenarios de secuencia más finos (por ejemplo, HU-01 en gestión de empresas y gestión de plantas), que se documentan en la Sección VII.

TABLA XXXVIII
HISTORIA DE USUARIO HU-01: GESTIÓN DE EMPRESAS AGROPECUARIAS Y PLANTAS

Código: **HU-01**	Usuario: **Analista logístico**
Título: **Gestión de empresas agropecuarias y plantas**	
Requerimiento funcional asociado (Id): **RF01, RF02**	
Prioridad: **Alta**	Tipo: **Funcionalidad**
Descripción: Como analista logístico de EcoResiduos del Llano, quiero registrar, modificar y eliminar empresas agropecuarias y plantas de procesamiento, para mantener actualizada la red y poder ampliarla con nuevas empresas y sedes.	
Criterios de aceptación: • Se cargan por defecto las 10 empresas y las 4 plantas (Villavicencio, Restrepo, Granada y Puerto Gaitán). • Puedo agregar una nueva empresa o planta indicando nombre y municipio, y queda disponible para el modelo sin reiniciar el sistema. • No se permite duplicar nombres ni eliminar un nodo sin confirmación del usuario. • Los cambios se reflejan de inmediato en la matriz de costos, ofertas y demandas.	

TABLA XXXIX
HISTORIA DE USUARIO HU-02: GESTIÓN DE COSTOS, DISTANCIAS Y COEFICIENTES DEL NEGOCIO

Código: **HU-02**	Usuario: **Analista logístico**
Título: **Gestión de costos, distancias y coeficientes del negocio**	
Requerimiento funcional asociado (Id): **RF03, RF05**	
Prioridad: **Alta**	Tipo: **Funcionalidad**
Descripción: Como analista logístico, quiero modificar las distancias, los costos por kilómetro-tonelada, el precio de la tonelada y otros coeficientes del modelo, para evaluar el costo logístico con condiciones actualizadas.	
Criterios de aceptación: • Puedo editar la distancia de cada ruta celda por celda y el sistema recalcula la matriz de costos unitarios. • Puedo modificar el precio por tonelada de cada empresa y el costo de transporte (que incluye el combustible), directamente o con un factor porcentual, y el sistema recalcula los costos unitarios. • El sistema rechaza valores negativos o no numéricos con un mensaje claro. • Los valores originales se conservan en el escenario base: los cambios pueden hacerse sobre un escenario derivado.	

TABLA XL
HISTORIA DE USUARIO HU-03: GESTIÓN DE OFERTA Y DEMANDA

Código: **HU-03**	Usuario: **Analista logístico**
Título: **Gestión de oferta y demanda**	
Requerimiento funcional asociado (Id): **RF04, RF08**	
Prioridad: **Alta**	Tipo: **Funcionalidad**
Descripción: Como analista logístico, quiero ingresar la oferta semanal de cada empresa y la capacidad de cada planta, para que el modelo refleje el volumen real de residuos y la capacidad de procesamiento.	
Criterios de aceptación: • Puedo ingresar y modificar la oferta (ton) de cada empresa y la demanda/capacidad (ton) de cada planta. • El sistema muestra la oferta total, la demanda total y su diferencia. • Si hay desbalance, el sistema advierte y añade un nodo ficticio para equilibrar el problema. • Los datos incompletos o inválidos impiden ejecutar los métodos de solución.	

TABLA XLI
HISTORIA DE USUARIO HU-04: GUARDAR Y RECUPERAR ESCENARIOS

Código: **HU-04**	Usuario: **Analista logístico**
Título: **Guardar y recuperar escenarios**	
Requerimiento funcional asociado (Id): **RF06, RF17**	
Prioridad: **Media**	Tipo: **Mejora**
Descripción: Como analista logístico, quiero contar con los datos regionales precargados y guardar escenarios, para evitar la digitación manual y poder retomar análisis previos.	
Criterios de aceptación: • Puedo crear un escenario nuevo a partir de otro o duplicar uno existente sin modificar el original. • Puedo buscar los escenarios guardados y ver su costo exacto y su variación frente al caso base. • Puedo guardar un escenario con nombre y recuperarlo posteriormente con sus parámetros y resultados. • Los datos de prueba regionales están disponibles como ejemplo precargado.	

TABLA XLII
HISTORIA DE USUARIO HU-05: VISUALIZACIÓN DE LA RED LOGÍSTICA

Código: **HU-05**	Usuario: **Gerente de EcoResiduos**
Título: **Visualización de la red logística**	
Requerimiento funcional asociado (Id): **RF07**	
Prioridad: **Media**	Tipo: **Funcionalidad**
Descripción: Como gerente, quiero ver la red logística como un grafo de empresas, plantas y rutas, para comprender cómo se conectan y cuánto cuesta cada ruta.	
Criterios de aceptación: • El sistema muestra nodos fuente y destino con sus arcos y el costo o la distancia de cada uno. • Al seleccionar un nodo o arco, se muestran sus datos (oferta, demanda, costo). • El grafo se actualiza al agregar o quitar empresas o plantas.	

TABLA XLIII
HISTORIA DE USUARIO HU-06: RESOLUCIÓN CON EL MODELO EXACTO

Código: **HU-06**	Usuario: **Analista de optimización**
Título: **Resolución con el modelo exacto**	
Requerimiento funcional asociado (Id): **RF09, RF10**	
Prioridad: **Alta**	Tipo: **Funcionalidad**
Descripción: Como analista de optimización, quiero resolver el problema de transporte con un método exacto, para obtener la distribución óptima de residuos y el costo mínimo total.	
Criterios de aceptación: • El sistema genera el modelo de programación lineal a partir de los datos vigentes y lo resuelve. • Se muestra el estado de la solución (óptima, infactible o no acotada), el costo total y el flujo por arco. • El sistema verifica que la solución cumpla oferta, demanda y no negatividad. • Se muestra el tiempo de ejecución del método.	

TABLA XLIV
HISTORIA DE USUARIO HU-07: CONFIGURACIÓN Y EJECUCIÓN DEL ALGORITMO GENÉTICO

Código: **HU-07**	Usuario: **Analista de optimización**
Título: **Configuración y ejecución del Algoritmo Genético**	
Requerimiento funcional asociado (Id): **RF11, RF12, RF13**	
Prioridad: **Alta**	Tipo: **Funcionalidad**
Descripción: Como analista de optimización, quiero configurar y ejecutar un algoritmo genético con parámetros ajustables, para obtener soluciones cuasi-óptimas y estudiar su desempeño.	
Criterios de aceptación: • Puedo definir población, generaciones, tasas de cruce y mutación, método de selección y tipo de cruce. • El sistema muestra la mejor solución, su costo, su tiempo y la curva de convergencia. • La penalización de violaciones de capacidad está incluida en la función de aptitud. • Puedo ejecutar varias corridas y ver mejor, promedio y desviación estándar del costo. • Con la misma semilla, el AG produce el mismo resultado.	

TABLA XLV
HISTORIA DE USUARIO HU-08: VISUALIZACIÓN DE RESULTADOS

Código: **HU-08**	Usuario: **Gerente de EcoResiduos**
Título: **Visualización de resultados**	
Requerimiento funcional asociado (Id): **RF14, RF19**	
Prioridad: **Alta**	Tipo: **Funcionalidad**
Descripción: Como gerente, quiero ver de forma clara cuántas toneladas va a cada planta desde cada empresa y cuánto cuesta, para tomar decisiones sobre las rutas y los centros de acopio.	
Criterios de aceptación: • Se muestra una tabla y un gráfico de flujos por empresa y planta. • Se muestra el costo total y el porcentaje de utilización de cada planta. • Puedo exportar los resultados en un reporte PDF.	

TABLA XLVI
HISTORIA DE USUARIO HU-09: ANÁLISIS DE SENSIBILIDAD

Código: **HU-09**	Usuario: **Analista de optimización**
Título: **Análisis de sensibilidad**	
Requerimiento funcional asociado (Id): **RF15**	
Prioridad: **Alta**	Tipo: **Funcionalidad**
Descripción: Como analista de optimización, quiero conocer los precios sombra y los rangos de variación permitidos, para saber qué parámetros afectan más el costo y qué tan estable es la solución óptima.	
Criterios de aceptación: • El sistema muestra los precios sombra de cada restricción de oferta y demanda. • El sistema muestra los rangos de variación de costos, ofertas y demandas que no alteran la base óptima. • Se identifican las restricciones activas (cuellos de botella). • Los resultados se presentan en tablas y gráficos interpretables.	

TABLA XLVII
HISTORIA DE USUARIO HU-10: SIMULACIÓN DE ESCENARIOS

Código: **HU-10**	Usuario: **Analista logístico**
Título: **Simulación de escenarios**	
Requerimiento funcional asociado (Id): **RF16, RF17**	
Prioridad: **Alta**	Tipo: **Funcionalidad**
Descripción: Como analista logístico, quiero simular cambios en el costo de combustible, en la oferta de residuos y en la capacidad de las plantas, para anticipar el impacto sobre el costo logístico.	
Criterios de aceptación: • Puedo definir variaciones porcentuales del costo de transporte y de la oferta, y reducir la capacidad de una planta o de todas. • El sistema ejecuta el método exacto y el AG sobre cada escenario y muestra los resultados de cada uno. • Los datos originales no se modifican al simular. • Puedo comparar varios escenarios entre sí.	

TABLA XLVIII
HISTORIA DE USUARIO HU-11: REPORTE COMPARATIVO EXACTO VS. ALGORITMO GENÉTICO

Código: **HU-11**	Usuario: **Gerente de EcoResiduos**
Título: **Reporte comparativo exacto vs. Algoritmo Genético**	
Requerimiento funcional asociado (Id): **RF18, RF19**	
Prioridad: **Alta**	Tipo: **Funcionalidad**
Descripción: Como gerente y como equipo del proyecto, quiero un reporte que compare el método exacto con el algoritmo genético, para evaluar cuál ofrece mejor calidad de solución y tiempo de cómputo.	
Criterios de aceptación: • El reporte muestra el costo total de ambos métodos y el tiempo de ejecución de cada uno. • El reporte calcula el gap porcentual del AG respecto al óptimo exacto. • El reporte incluye el comportamiento de ambos métodos en cada escenario de sensibilidad. • El reporte se puede exportar para incluirlo en el informe técnico IEEE.	

E. Matrices de trazabilidad

La Tabla XLIX relaciona cada historia de usuario con los requerimientos funcionales que la implementan, y la Tabla L relaciona cada requerimiento funcional con los no funcionales que lo condicionan. Ambas matrices garantizan que todo requerimiento tiene al menos una historia y una actividad de la matriz RACI asociada.

TABLA XLIX
TRAZABILIDAD ENTRE HISTORIAS DE USUARIO Y REQUERIMIENTOS FUNCIONALES

Historia	Título	Requerimientos funcionales	Prioridad
HU-01	Gestión de empresas agropecuarias y plantas	RF01, RF02	Alta
HU-02	Gestión de costos, distancias y coeficientes del negocio	RF03, RF05	Alta
HU-03	Gestión de oferta y demanda	RF04, RF08	Alta
HU-04	Guardar y recuperar escenarios	RF06, RF17	Media
HU-05	Visualización de la red logística	RF07	Media
HU-06	Resolución con el modelo exacto	RF09, RF10	Alta
HU-07	Configuración y ejecución del Algoritmo Genético	RF11, RF12, RF13	Alta
HU-08	Visualización de resultados	RF14, RF19	Alta
HU-09	Análisis de sensibilidad	RF15	Alta
HU-10	Simulación de escenarios	RF16, RF17	Alta
HU-11	Reporte comparativo exacto vs. Algoritmo Genético	RF18, RF19	Alta

TABLA L
TRAZABILIDAD ENTRE REQUERIMIENTOS FUNCIONALES Y NO FUNCIONALES

RF	Nombre	RNF asociados
RF01	Gestionar empresas agropecuarias (fuentes)	RNF02, RNF03, RNF09
RF02	Gestionar plantas de recepción y transformación (destinos)	RNF02, RNF03, RNF09
RF03	Gestionar rutas, distancias y costos de transporte	RNF03, RNF09
RF04	Gestionar oferta y demanda/capacidad	RNF03, RNF09
RF05	Parametrizar coeficientes del modelo de negocio	RNF03, RNF09
RF06	Cargar los datos regionales de prueba	RNF09, RNF13
RF07	Construir y visualizar la red logística	RNF02, RNF11
RF08	Validar consistencia y balance del problema	RNF06, RNF09
RF09	Resolver el modelo exacto de transporte / costo mínimo	RNF01, RNF04, RNF06
RF10	Verificar factibilidad de las soluciones	RNF06
RF11	Configurar parámetros del Algoritmo Genético	RNF03, RNF07
RF12	Ejecutar el Algoritmo Genético	RNF01, RNF04, RNF07
RF13	Ejecutar múltiples corridas del AG y estadísticas	RNF01, RNF07
RF14	Visualizar resultados de optimización	RNF03, RNF11
RF15	Realizar análisis de sensibilidad	RNF01, RNF06
RF16	Simular escenarios de variación	RNF01, RNF02, RNF03

RF	Nombre	RNF asociados
RF17	Guardar y recuperar escenarios	RNF09, RNF13
RF18	Generar reporte comparativo exacto vs. AG	RNF06, RNF11
RF19	Exportar reportes	RNF13

IV. MODELADO DE LA RED LOGÍSTICA (FASE 1)

A. *Descripción del sistema logístico*

La red tiene 10 empresas agropecuarias (fuentes) que generan un volumen constante de residuos orgánicos cada semana. Cada una se ubica en un municipio del Meta y se asocia con la actividad productiva predominante de la zona. EcoResiduos paga a cada empresa un precio por tonelada β_i, que puede depender del tipo de residuo; en el caso base es de 12.000 COP/t para todas (Tabla LI).

TABLA LI
EMPRESAS AGROPECUARIAS ALIADAS (FUENTES)

Nodo	Empresa	Municipio	Residuo predominante	Oferta (t/sem)
E1	Palmeras del Llano S.A.S.	San Carlos de Guaroa	Raquis y fibra de palma	150
E2	Unipalma de Los Llanos S.A.	Cumaral	Cuesco de palmito y raquis de palma	120
E3	Poligrow Colombia S.A.S.	Puerto Gaitán	Biomasa de poda foliar en cultivo	110
E4	Aceites Cumaral S.A.S.	Cumaral	Fibra vegetal y residuos de prensado	140
E5	Comercializadora del Llano S.A.	Villavicencio	Tamo y cascarilla de arroz	100
E6	Molinos Roa S.A.	Villavicencio	Cascarilla de arroz	100
E7	Agropecuaria Aliar S.A.	Puerto López	Estiércol porcino y bovino	150
E8	Refocosta S.A.	Puerto López	Biomasa maderable	120
E9	Bioenergy S.A.S.	Puerto López	Bagazo de caña de azúcar	120
E10	Frigollano S.A.	Villavicencio	Estiércol de bovinos	140
	('b', 'Total')			('b', '1.250')

El precio de compra del residuo es $\beta_i = 12.000$ COP/t para todas las empresas en el caso base.

Las 4 plantas tienen una demanda D_j, que representa el volumen necesario para operar de forma rentable (carga base de compostaje, biodigestión o peletizado), y un costo de operación por tonelada procesada p_j (Tabla LII). La oferta total y la demanda total son iguales a 1.250 t/semana, por lo que el caso base es un problema **balanceado**.

TABLA LII
PLANTAS DE RECEPCIÓN Y TRANSFORMACIÓN (DESTINOS)

Nodo	Municipio	Demanda (t/sem)	Costo de operación (COP/t)
P1	Villavicencio	400	42.000
P2	Restrepo	250	48.000
P3	Granada	300	45.000
P4	Puerto Gaitán	300	38.000
	('b', 'Total')	('b', '1.250')	

B. *Definición formal del grafo*

La red se modela como un grafo bipartito, dirigido y ponderado $G = (V, A)$ [7], con

$$V = V_O \cup V_D, \quad V_O \cap V_D = \emptyset \tag{1}$$

donde $V_O = \{E_1, \dots, E_{10}\}$ son los nodos de origen (empresas, con oferta $O_i > 0$) y $V_D = \{P_1, \dots, P_4\}$ son los nodos de destino (plantas, con demanda $D_j > 0$). Los arcos van siempre de una empresa a una planta, porque el residuo no regresa ni se mueve entre plantas, de modo que el grafo es bipartito dirigido completo con $|A| = m \cdot n = 40$ arcos. No se requieren nodos de paso, ya que cada arco agrega la mejor ruta vial entre su origen y su destino. A cada arco $(i, j) \in A$ se le asocian la distancia vial estimada d_{ij} (km), el costo de transporte por tonelada-kilómetro α (750 COP/(t·km) en el caso base) y el costo unitario total c_{ij} (COP/t).

La Fig. 3 representa el grafo con sus 10 fuentes, 4 plantas y 40 arcos, y la Fig. 4 ubica los nodos en el mapa del Meta. La Tabla LIII contiene la matriz de distancias viales estimadas.

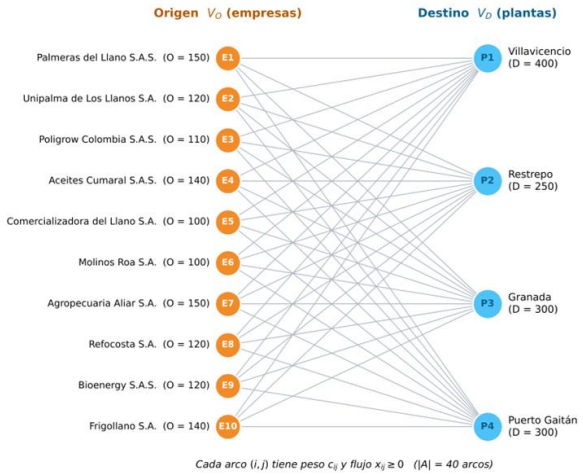


Fig. 3. Representación del grafo G = (V, A): 10 fuentes, 4 plantas y 40 arcos dirigidos de fuente a planta.

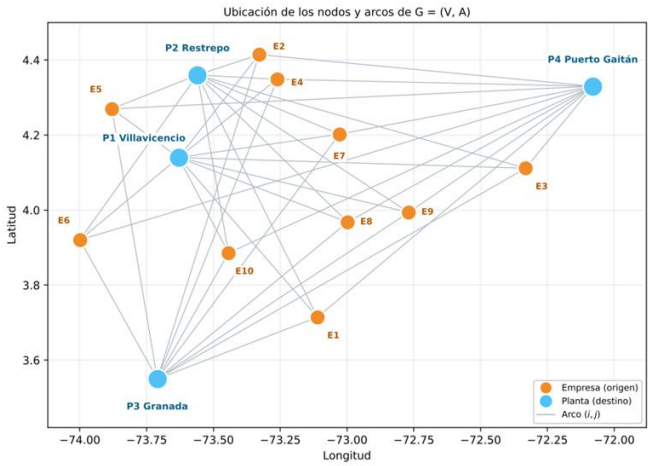


Fig. 4. Ubicación geográfica de los nodos y de los 40 arcos de la red en el departamento del Meta.

TABLA LIII
MATRIZ DE DISTANCIAS VIALES ESTIMADAS (KM)

Empresa	P1	P2	P3	P4
E1 Palmeras del Llano	85	102	92	275
E2 Unipalma de Los Llanos	28	15	113	218
E3 Poligrow Colombia	190	207	275	5
E4 Aceites Cumaral	28	15	113	218
E5 Comercializadora del Llano	5	22	85	190
E6 Molinos Roa	5	22	85	190
E7 Agropecuaria Aliar	85	102	170	105
E8 Refocosta	85	102	170	105
E9 Bioenergy	85	102	170	105

Empresa	P1	P2	P3	P4
E10 Frigollano	5	22	85	190

P1 Villavicencio, P2 Restrepo, P3 Granada, P4 Puerto Gaitán. Las empresas de un mismo municipio comparten distancias.

C. *Parámetros del modelo de negocio*

La Tabla LIV resume los parámetros del modelo. Todos se pueden modificar desde la interfaz sin cambiar el código, como exige el requisito de parametrización del enunciado.

TABLA LIV
PARÁMETROS DEL MODELO DE NEGOCIO

Símbolo	Descripción	Unidad	Caso base
O_i	Oferta semanal de residuos de la fuente i	t/semana	Tabla LI
D_j	Demanda de operación de la planta j	t/semana	Tabla LII
p_j	Costo de operación por tonelada en la planta j	COP/t	38.000 a 48.000
α	Costo base de transporte por tonelada-kilómetro	COP/(t·km)	750
d_{ij}	Distancia vial del arco (i, j)	km	Tabla LIII
β_i	Precio pagado a la empresa i por tonelada de residuo	COP/t	12.000

El costo de enviar una tonelada de la empresa i a la planta j es la suma del costo de transporte, proporcional a la distancia, del costo de procesarla en la planta de destino y del precio que se paga a la empresa de origen:

$$c_{ij} = \alpha d_{ij} + p_j + \beta_i \tag{2}$$

Por ejemplo, para la ruta E1 → P1 se tiene $c = 750 \times 85 + 42.000 + 12.000 = 117.750$ COP/t. La aplicación permite además un factor porcentual sobre α para simular cambios en el precio del combustible. La Tabla LV contiene la matriz de costos unitarios que usan tanto el método exacto como el AG, y la Fig. 5 la muestra como mapa de calor: las rutas más baratas son E3 → P4 (53.750 COP/t, porque Poligrow está en Puerto Gaitán) y las de E5, E6 y E10 hacia P1, y la más cara es E3 → P3 (263.250 COP/t).

TABLA LV
MATRIZ DE COSTOS UNITARIOS CIJ (COP/T)

Empresa	P1	P2	P3	P4	Oferta
E1	117.750	136.500	126.000	256.250	150
E2	75.000	71.250	141.750	213.500	120
E3	196.500	215.250	263.250	53.750	110
E4	75.000	71.250	141.750	213.500	140
E5	57.750	76.500	120.750	192.500	100
E6	57.750	76.500	120.750	192.500	100
E7	117.750	136.500	184.500	128.750	150
E8	117.750	136.500	184.500	128.750	120
E9	117.750	136.500	184.500	128.750	120
E10	57.750	76.500	120.750	192.500	140
('b', 'Demanda')	('b', '400')	('b', '250')	('b', '300')	('b', '300')	('b', '1.250')

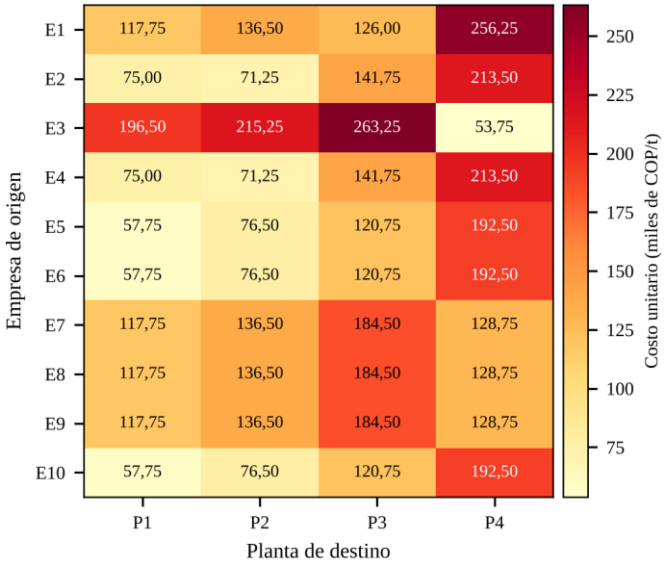


Fig. 5. Mapa de calor de la matriz de costos unitarios (miles de COP por tonelada).

D. *Modelo de programación lineal*

Con los conjuntos $V_O = \{1, \dots, m\}$ de fuentes (índice i, $m = 10$) y $V_D = \{1, \dots, n\}$ de plantas (índice j, $n = 4$), la variable de decisión x_{ij} representa las toneladas de residuo enviadas cada semana desde la fuente i hacia la planta j. El modelo del problema de transporte [1], [3] es:

$$min\ Z = \sum_{i=1}^{m} \sum_{j=1}^{n} c_{ij} x_{ij} \tag{3}$$

sujeto a

$$\sum_{j=1}^{n} x_{ij} = O_i, \quad \forall\, i \in V_O \tag{4}$$

$$\sum_{i=1}^{m} x_{ij} \le D_j, \quad \forall\, j \in V_D \tag{5}$$

$$x_{ij} \ge 0, \quad \forall\, (i,j) \in A \tag{6}$$

La función objetivo (3) minimiza el costo semanal de compra, transporte y operación; como c_{ij} ya incluye β_i, el valor óptimo de Z es directamente el costo total que se reporta al usuario. La restricción (4) obliga a recolectar toda la producción de cada empresa (10 restricciones), la (5) impide superar la demanda de cada planta (4 restricciones) y la (6) es la no negatividad (40 variables). En el caso base toda la oferta se recolecta, de modo que el término $\sum_i \beta_i O_i = 15.000.000$ COP no cambia los flujos óptimos aunque sí el valor de Z; el precio influye en las decisiones cuando la oferta supera la demanda. Explícitamente, para la empresa E1 la restricción de oferta es $x_{1,1} + x_{1,2} + x_{1,3} + x_{1,4} = 150$, y para la planta P1 la de demanda es $\sum_{i=1}^{10} x_{i,1} \le 400$.

E. *Forma no balanceada*

Durante el análisis de sensibilidad la oferta o la demanda cambian y el problema puede dejar de estar balanceado. El sistema detecta el caso y agrega un nodo ficticio de costo cero, como en el método clásico de transporte:

1) **Oferta mayor que la demanda.** No es posible procesar todo el residuo: la oferta pasa a ser de tipo ≤ y la demanda de tipo =. Se agrega una planta ficticia P_{n+1} con demanda

$$D_{n+1} = \sum_{i=1}^{m} O_i - \sum_{j=1}^{n} D_j, \quad c_{i,n+1} = 0 \tag{7}$$

cuyo flujo $x_{i,n+1}$ son las toneladas que quedan sin recolectar en la empresa i. Como esas toneladas no se compran, a igual costo de transporte y operación el modelo deja sin recolectar las de mayor precio. Por ejemplo, si la oferta de E7 sube de 150 a 250 t, la oferta total es 1.350 t y se agrega P_5 con $D_5 = 100$ t.

2) **Oferta menor que la demanda.** Toda la oferta se recolecta, pero las plantas no se llenan. Se agrega una empresa ficticia E_{m+1} con oferta

$$O_{m+1} = \sum_{j=1}^{n} D_j - \sum_{i=1}^{m} O_i, \quad c_{m+1,j} = 0 \tag{8}$$

cuyo flujo $x_{m+1,j}$ es la capacidad no utilizada de la planta j. Por ejemplo, si la oferta total disminuye un 10 % (1.125 t), se agrega E_{11} con $O_{11} = 125$ t.

F. *Supuestos del modelo*

La validación de los supuestos con el líder del proyecto dejó fijados los siguientes, que delimitan el alcance del modelo y deben tenerse en cuenta al interpretar los resultados:

1) El residuo se trata como un producto homogéneo medido en toneladas; cualquier planta puede procesar cualquier tipo de residuo.
2) Los costos son lineales en las toneladas: no hay economías de escala, costos fijos por ruta ni descuentos por volumen.
3) La flota no es restrictiva: no se modela la capacidad de los camiones ni el número de viajes.
4) Las distancias son estimaciones viales entre cabeceras municipales; las empresas de un mismo municipio comparten distancias.
5) La planificación es semanal y estática: la oferta y la demanda se consideran constantes durante la semana.
6) Las rutas sin distancia registrada se consideran deshabilitadas y no generan variable de decisión.

V. Solución exacta (Fase 2)

A. *Método Simplex de dos fases*

Por restricción tecnológica del proyecto, el solucionador exacto se implementó desde cero sobre NumPy [16], sin librerías de optimización. Se eligió el método Simplex de dos fases sobre una tabla densa [4], [5], que resuelve problemas de la forma

$$\min c^T x \quad s.a. \quad A_{=}x = b_{=}, \quad A_{\leq}x \leq b_{\leq}, \quad x \geq 0 \tag{9}$$

El procedimiento implementado es el siguiente:

1) Se llevan los lados derechos a valores no negativos y se agrega una holgura por cada desigualdad y una variable artificial por cada igualdad (y por cada desigualdad invertida).
2) En la fase 1 se minimiza la suma de las variables artificiales. Si el óptimo de esa fase es mayor que una tolerancia relativa de 10^{-7}, el problema es infactible.
3) Las artificiales que quedan en la base con valor cero se retiran pivotando sobre una columna real; si una fila no tiene coeficientes reales, la restricción es redundante (en el problema de transporte siempre hay una dependiente) y se elimina.
4) En la fase 2 se fija la función objetivo original, con la fila de costos reducidos

$$d_{\square} = c - c_B B^{-1} A \tag{10}$$

y se pivotea mientras exista un costo reducido negativo. La columna entrante se elige con la regla de Dantzig (el costo reducido más negativo) y la fila saliente con la razón mínima; los empates se rompen por el menor índice básico. Como el problema de transporte es muy degenerado, si se acumulan 50 pivotes degenerados seguidos el método cambia a la regla de Bland [6], que garantiza que no haya ciclos. El método reporta los estados óptimo, infactible, no acotado o límite de iteraciones (100.000), y los valores menores que 10^{-9} se llevan a cero.

B. *Construcción del modelo a partir de los datos*

El módulo del modelo de transporte toma los datos vigentes del escenario (oferta, demanda, matriz de costos y rutas habilitadas), verifica el balance, agrega el nodo ficticio cuando corresponde y arma el programa lineal: una fila de igualdad por empresa, una fila de desigualdad por planta y una columna por cada ruta habilitada. Así el tamaño del programa crece con los arcos reales de la red, lo que permite ampliar a 100 empresas y 20 plantas sin cambiar el código (RNF02). La solución devuelve el estado, la matriz de flujos sin el nodo ficticio, el costo total con los costos originales, el número de iteraciones, el tiempo de cómputo y, si existe, el residuo sin recolectar o la capacidad ociosa.

C. *Código fuente de la solución exacta*

Los siguientes fragmentos muestran, tal como están en el código, cómo se resuelve el modelo de transporte con el método exacto; los números de línea corresponden a los archivos del repositorio. En el Listado 1, el método resolver verifica el balance y agrega el nodo ficticio si hace falta (líneas 55 y 56), crea una variable por cada ruta habilitada (línea 58), arma el programa lineal (línea 59) y lo resuelve con el Simplex midiendo el tiempo de cómputo (líneas 61 y 62).

Listado 1. Resolución del modelo de transporte (transport_model.py, ModeloTransporte.resolver).

```
52 def resolver(self) -> SolucionTransporte:
53     p = self.problema
54     m, n = p.dimension
55     balance = verificar_balance(p.oferta, p.demanda)
56     balanceado = balancear_problema(p.oferta, p.demanda,
           p.costos, p.habilitadas)
57
58     arcos = np.argwhere(balanceado.habilitadas)
59     c, A_igual, b_igual, A_menor, b_menor =
           self._programa_lineal(balanceado, arcos)
60
61     with Cronometro() as cronometro:
62         resultado = self.simplex.resolver(c, A_igual,
               b_igual, A_menor, b_menor)
```

El Listado 2 construye el programa lineal de la Fase 1: una fila de igualdad por empresa con un 1 en las columnas de sus rutas (líneas 88 y 89), una fila de desigualdad por planta (líneas 90 y 91) y el vector de costos unitarios de las rutas habilitadas (línea 92).

Listado 2. Construcción de las restricciones de oferta y demanda (transport_model.py, ModeloTransporte._programa_lineal).

```
82 @staticmethod
83 def _programa_lineal(balanceado, arcos):
84     num_arcos = len(arcos)
85     filas, columnas = arcos[:, 0], arcos[:, 1]
86     indice = np.arange(num_arcos)
87
88     A_igual = np.zeros((len(balanceado.oferta),
           num_arcos))
89     A_igual[filas, indice] = 1.0
90     A_menor = np.zeros((len(balanceado.demanda),
           num_arcos))
91     A_menor[columnas, indice] = 1.0
92     c = balanceado.costos[filas, columnas]
93     return c, A_igual, balanceado.oferta, A_menor,
           balanceado.demanda
```

El Listado 3 es la fase 1 del Simplex: si hay variables artificiales, minimiza su suma (líneas 75 a 78) y, si esa suma no llega a cero, declara el problema infactible (líneas 81 a 84); luego retira de la base las artificiales que quedaron en cero (línea 85).

Listado 3. Fase 1 del Simplex: búsqueda de una base factible (simplex.py, Simplex.resolver).

```
73 iteraciones = 0
74 if n_artificial:
75     costo_fase1 = np.zeros(n_total)
76     costo_fase1[n + n_holgura:] = 1.0
77     self._fijar_objetivo(tabla, base, costo_fase1)
78     estado, iteraciones = self._iterar(tabla, base,
           iteraciones)
79     if estado is not EstadoSolucion.OPTIMO:
80         return ResultadoSimplex(estado, None, None,
               iteraciones, iteraciones, None)
81     escala = max(1.0, float(np.abs(tabla[:m, -
           1]).max(initial=0.0)))
82     if -tabla[-1, -1] > 1e-7 * escala:
83         return ResultadoSimplex(EstadoSolucion.INFACTIBLE,
               None, None,
84                   iteraciones, iteraciones, None)
85     tabla, base = self._retirar_artificiales(tabla, base,
           n + n_holgura)
```

El Listado 4 es la fase 2: fija la función objetivo original con costo cero para las holguras (líneas 88 y 89), itera hasta el óptimo (línea 90) y lee la solución de la columna de lados derechos (líneas 94 a 98).

Listado 4. Fase 2 del Simplex: optimización de la función objetivo (simplex.py, Simplex.resolver).

```
88 costo_fase2 = np.concatenate([c, np.zeros(n_holgura)])
89 self._fijar_objetivo(tabla, base, costo_fase2)
90 estado, iteraciones = self._iterar(tabla, base,
       iteraciones)
91 if estado is not EstadoSolucion.OPTIMO:
92     return ResultadoSimplex(estado, None, None,
           iteraciones, iteraciones_fase1, None)
93
94 x = np.zeros(n + n_holgura)
95 x[base] = tabla[:-1, -1]
96 x[np.abs(x) < self.tolerancia] = 0.0
97 return ResultadoSimplex(EstadoSolucion.OPTIMO, x[:n],
       float(c @ x[:n]),
98         iteraciones, iteraciones_fase1, base.copy())
```

El Listado 5 es un pivote del método: elige la columna entrante con la regla de Dantzig o, tras 50 pivotes degenerados seguidos, con la de Bland (líneas 125 y 126); aplica la prueba de la razón mínima y rompe los empates por el menor índice básico (líneas 128 a 135), y actualiza la tabla (línea 138).

Listado 5. Selección de las variables entrante y saliente y pivote (simplex.py, Simplex._iterar).

```
125 usar_bland = racha_degenerada >= self.limite_degeneracion
126 entrante = candidatas[0] if usar_bland else
       candidatas[np.argmin(reducidos[candidatas])]
127
128 columna = tabla[:-1, entrante]
129 filas = np.flatnonzero(columna > self.tolerancia)
130 if filas.size == 0:
131     return EstadoSolucion.NO_ACOTADO, iteraciones
132 razones = tabla[filas, -1] / columna[filas]
133 minima = razones.min()
134 empates = filas[razones <= minima + self.tolerancia]
135 saliente = empates[np.argmin(base[empates])]
136
137 racha_degenerada = racha_degenerada + 1 if minima <=
       self.tolerancia else 0
138 self._pivotear(tabla, base, saliente, entrante)
139 iteraciones += 1
```

D. Solución óptima del caso base

El método encuentra la solución óptima en 35 iteraciones, con un costo total de **120.472.500 COP por semana**. La Tabla LVI presenta los flujos óptimos, la Fig. 6 muestra cómo se llena cada planta y la Fig. 7 dibuja la red con las 13 rutas usadas. El costo se descompone en 51.772.500 COP de transporte (43,0 %), 53.700.000 COP de operación (44,6 %) y 15.000.000 COP de compra del residuo (12,5 %); la Fig. 8 lo desagrega por planta y por empresa.

TABLA LVI
FLUJOS ÓPTIMOS DEL MÉTODO EXACTO (T/SEMANA)

Empresa	P1	P2	P3	P4	Oferta
E1	–	–	150	–	150
E2	–	120	–	–	120
E3	–	–	–	110	110

Empresa	P1	P2	P3	P4	Oferta
E4	10	130	–	–	140
E5	50	–	50	–	100
E6	–	–	100	–	100
E7	–	–	–	150	150
E8	80	–	–	40	120
E9	120	–	–	–	120
E10	140	–	–	–	140
('b', 'Recibido')	('b', '400')	('b', '250')	('b', '300')	('b', '300')	('b', '1.250')

Costo total Z* = 120.472.500 COP/semana. Las cuatro plantas reciben exactamente su demanda.

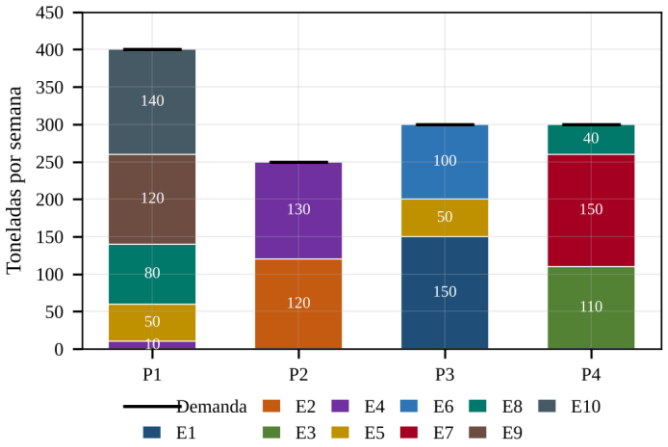


Fig. 6. Toneladas que recibe cada planta en el óptimo, según la empresa de origen; la línea negra es la demanda.

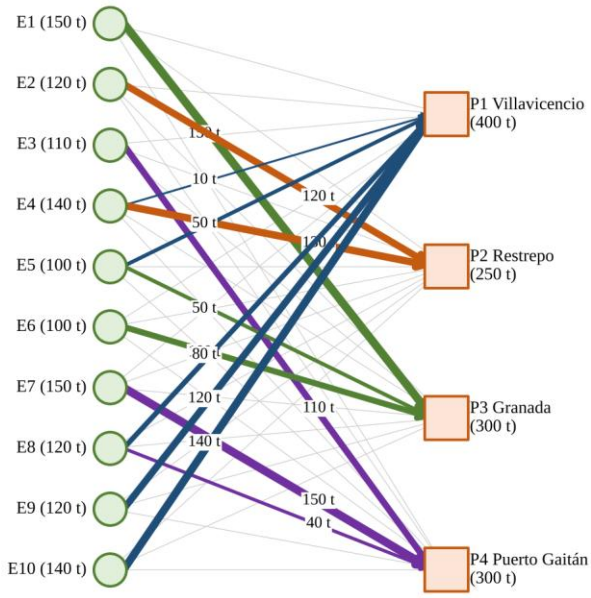


Fig. 7. Red de flujos óptimos: el grosor del arco es proporcional a las toneladas enviadas; en gris, las rutas no usadas.

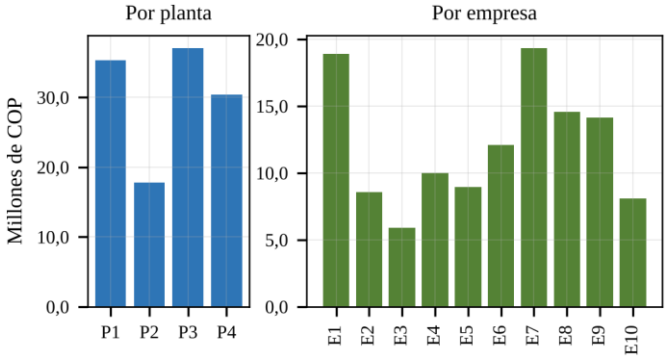


Fig. 8. Costo total del plan óptimo por planta de destino y por empresa de origen (millones de COP por semana).

La solución tiene sentido logístico. Poligrow (E3), ubicada en Puerto Gaitán, envía sus 110 t a P4, en el mismo municipio, y P4 se completa con E7 (150 t) y E8 (40 t), de Puerto López. Las empresas de Cumaral, E2 y E4, llenan P2 en Restrepo, a 15 km, y E4 envía las 10 t que le sobran a P1. Villavicencio (P1) recibe de E10 (140 t), E9 (120 t), E8 (80 t), E5 (50 t) y E4 (10 t). Granada (P3), la planta más alejada de casi todas las fuentes, recibe de E1 (150 t), E6 (100 t) y E5 (50 t): para E1 es la segunda planta más barata y su primera opción, P1, ya está llena con fuentes de menor costo. La base óptima tiene 14 variables, una por restricción, pero solo 13 rutas llevan flujo positivo; la variable básica restante vale cero, por lo que la solución es degenerada, aspecto que se retoma en el análisis de sensibilidad.

E. Validación de factibilidad

Todo plan de envíos, del método exacto o del AG, pasa por un verificador independiente que comprueba la oferta, la demanda y la no negatividad con una tolerancia de 10^{-6} t (RF10, RNF06). Cuando la oferta supera la demanda, la oferta se evalúa como desigualdad. La Tabla LVII muestra el reporte del verificador para la solución del caso base.

TABLA LVII
VERIFICACIÓN DE FACTIBILIDAD DE LA SOLUCIÓN EXACTA

Restricción	Cumplidas	Resultado
Oferta de las empresas (4)	10/10	Cumple
Demanda de las plantas (5)	4/4	Cumple
No negatividad (6)	40/40	Cumple
Violación máxima	0,0e+00 t	Factible

F. Verificación independiente con un segundo solucionador

Para verificar el módulo exacto con un método alterno (actividad 18 de la matriz RACI), la misma formulación se resolvió con el solucionador HiGHS [15] de la librería SciPy [18], usado solo como instrumento de prueba y no dentro de la aplicación. En el caso base ambos obtienen 120.472.500 COP. Además se generaron 200 instancias aleatorias de entre 2 y 8 empresas y entre 2 y 5 plantas, con costos entre 40.000 y 270.000 COP/t: 67 balanceadas, 67 con oferta menor y 66 con oferta mayor que la demanda. En las 200 instancias el costo óptimo del Simplex propio coincide con el de HiGHS; la mayor diferencia relativa fue de 3,1e-16, del orden del error de redondeo de la aritmética de punto flotante (Tabla LVIII).

TABLA LVIII
VERIFICACIÓN DEL SIMPLEX PROPIO FRENTE A HIGHS

Conjunto de prueba	Instancias	Coinciden	Diferencia máx.
Caso base 10 × 4	1	1	0
Aleatorias balanceadas	67	67	$< 10^{-15}$
Aleatorias con oferta menor	67	67	$< 10^{-15}$
Aleatorias con oferta mayor	66	66	$< 10^{-15}$
Casos no balanceados del informe	3	3	0
Instancias de escalabilidad	6	6	$< 10^{-3}$ COP

G. *Desempeño del método exacto*

En 30 ejecuciones del caso base (40 variables y 14 restricciones) el tiempo de solución tuvo una mediana de 0,61 ms, con un mínimo de 0,54 ms y un máximo de 0,87 ms, muy por debajo del límite de 5 s de RNF01. El crecimiento del tiempo con el tamaño de la red se analiza en la Sección IX.

VI. ALGORITMO GENÉTICO (FASE 3)

Un algoritmo genético es una metaheurística poblacional inspirada en la evolución natural: mantiene un conjunto de soluciones candidatas, favorece la reproducción de las mejores y genera variantes mediante recombinación y mutación [8], [9], [14]. No garantiza el óptimo, pero no exige que el modelo sea lineal, explora muchas soluciones en paralelo y su tiempo crece de forma controlada con el tamaño de la red. El AG resuelve exactamente el mismo modelo que el método exacto, incluidos los nodos ficticios, de modo que ambos se comparan en igualdad de condiciones. La Tabla LIX relaciona cada concepto del AG con su significado en el problema.

TABLA LIX
CONCEPTOS DEL ALGORITMO GENÉTICO EN EL PROBLEMA DE TRANSPORTE

Concepto	Significado en el problema
Individuo	Un plan de envíos semanal: cuántas toneladas van de cada empresa a cada planta.
Gen	Las toneladas x_{ij} de una ruta empresa → planta.
Cromosoma	La matriz $X = [x_{ij}]$ de tamaño m × n.
Población	P planes de envío que evolucionan juntos.
Aptitud	Inversa del costo, castigada si una planta recibe más que su demanda.
Generación	Una iteración de selección, cruce, mutación, reparación y reemplazo.
Elitismo	Los mejores planes pasan intactos a la siguiente generación.

A. *Representación de las soluciones*

Cada individuo se codifica directamente como la matriz de flujos del modelo, con valores reales no negativos [10], [11]:

$$X = [x_{ij}] \in \mathbb{R}^{m \times n}, \quad x_{ij} \geq 0 \tag{11}$$

La fila i es el plan completo de la empresa i y la columna j es lo que recibe la planta j. La población se almacena como un arreglo tridimensional de P × m × n números, lo que permite evaluar y transformar todos los individuos a la vez con operaciones vectorizadas. Esta codificación no requiere decodificación: el cromosoma es la solución (Fig. 9).

	P1	P2	P3	P4	$\Sigma_j x_{ij}$	Oferta O_i
E1	$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	=	150
E2	$x_{2,1}$	$x_{2,2}$	$x_{2,3}$	$x_{2,4}$	=	120
⋮	⋮	⋮	⋮	⋮	⋮	⋮
E10	$x_{10,1}$	$x_{10,2}$	$x_{10,3}$	$x_{10,4}$	=	140
$\Sigma_i x_{ij}$	≤ 400	≤ 250	≤ 300	≤ 300	**Demanda D_j**	

Cada fila (gen de empresa) suma su oferta por construcción · cada columna se controla con la penalización

Fig. 9. Estructura del cromosoma: matriz empresa × planta; cada fila suma la oferta de la empresa y cada columna está limitada por la demanda de la planta.

El diseño separa las restricciones según cómo se garantizan. La oferta (4), la no negatividad (6) y las rutas deshabilitadas se cumplen por construcción: todo individuo, después de crearse o modificarse, pasa por la reparación de la oferta. La demanda de las plantas (5) se trata por penalización [12]: un individuo puede exceder la demanda de una planta, pero su aptitud se castiga en proporción al exceso. Con $h_{ij} \in \{0,1\}$ como indicador de ruta habilitada, la reparación anula los valores negativos y las rutas deshabilitadas y reescala cada fila a su oferta:

$$\tilde{x}_{ij} = \max(0, x_{ij}) h_{ij}, \quad x_{ij} \leftarrow O_i \frac{\tilde{x}_{ij}}{\sum_k \tilde{x}_{ik}} \tag{12}$$

Si una fila queda en cero, la oferta se reparte en partes iguales entre las rutas habilitadas de esa empresa. Mantener cada fila igual a la oferta guía el resto del diseño: el cruce intercambia filas completas y la mutación mueve toneladas dentro de una fila, así que ninguno de los dos puede romper la oferta.

B. *Función de aptitud*

El costo de un plan es la función objetivo del modelo y el exceso de capacidad mide las toneladas que superan la demanda de las plantas:

$$Z(X) = \sum_i \sum_j c_{ij} x_{ij}, \quad E(X) = \sum_j \max\left(0, \sum_i x_{ij} - D_j\right) \tag{13}$$

La aptitud que el AG maximiza, como lo pide el enunciado, es la inversa del costo penalizado:

$$f(X) = \frac{1}{Z(X) + \lambda E(X)} \tag{14}$$

donde λ es el factor de penalización (1.000.000 COP por tonelada excedida, por defecto). Un individuo que respeta todas las demandas tiene E(X) = 0 y su aptitud es el inverso de su costo. El valor de λ se eligió mayor que cualquier ahorro posible al cambiar una tonelada de ruta: en el caso base la diferencia entre el costo unitario más alto y el más bajo es 263.250 − 53.750 = 209.500 COP/t, muy inferior a λ, así que exceder la demanda nunca es conveniente y la población converge a planes factibles. Una solución del AG es factible si $E(X) \leq 10^{-6}$ t.

C. *Población inicial*

La población inicial combina diversidad y calidad. La mayoría de los individuos se genera al azar: para cada uno se generan pesos con distribución exponencial de media 1 sobre las rutas habilitadas y la oferta de cada empresa se reparte en proporción a ellos,

$$x_{ij} = O_i \frac{w_{ij} h_{ij}}{\sum_k w_{ik} h_{ik}}, \quad w_{ij} \sim Exp(1) \tag{15}$$

lo que produce repartos muy variados. Una fracción de 0,2 de la población se construye con el método del costo mínimo del problema de transporte [2] sobre costos perturbados $\tilde{c}_{ij} = c_{ij}u_{ij}$, con $u_{ij} \sim U(0{,}9;\ 1{,}1)$: se recorren las rutas de menor a mayor costo perturbado, en cada una se envía el mínimo entre la oferta pendiente de la empresa y la capacidad libre de la planta, y la oferta que sobre se asigna a la ruta más barata de la empresa. Las rutas de nodos ficticios se ordenan con un costo de 10 · máx c + 1 para que se usen de últimas. Todos los individuos pasan por la reparación de la oferta antes de evaluarse.

D. *Operadores genéticos*

Elitismo. En cada generación los e individuos de mayor aptitud (2 por defecto) pasan sin cambios a la siguiente. Así la mejor solución nunca se pierde y la curva del mejor costo nunca empeora.

Selección. Para producir P − e hijos se forman $q = \lceil \frac{P-e}{2} \rceil$ parejas. En la selección por torneo cada padre es el más apto entre k individuos elegidos al azar con reemplazo (k = 3 por defecto) [13]. En la ruleta, la probabilidad de cada individuo es proporcional a su aptitud por encima de la del peor (escalado por ventana):

$$p_r = \frac{f_r - f_{mín}}{\sum_{s=1}^{P}(f_s - f_{mín})} \qquad (16)$$

Como las aptitudes son inversos de costos muy parecidos (del orden de 10^{-8}), una ruleta simple elegiría casi al azar; restar la aptitud mínima amplifica las diferencias. Si todas las aptitudes son iguales, la elección es uniforme.

Cruce por filas. Cada pareja se cruza con probabilidad p_c (0,85 por defecto). Los puntos de corte caen entre empresas: en el cruce de un punto se elige c ∈ {1, …, m − 1} y se intercambian las filas i ≥ c; en el de dos puntos se eligen $c_1 \le c_2$ y se intercambian las filas $c_1 \le i < c_2$. Cada hijo hereda el plan completo de cada empresa de uno de sus padres y la oferta se conserva (Fig. 10).

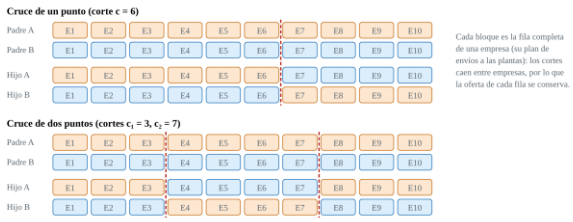


Fig. 10. Cruce de uno y dos puntos por filas: cada bloque es el plan completo de una empresa.

Mutación. Cada empresa de cada hijo muta con probabilidad p_m (0,05 por defecto). Se elige una ruta a con flujo y se aplica uno de dos movimientos (Fig. 11). En la transferencia, si alguna planta b tiene capacidad libre, con probabilidad 0,5 la empresa traslada δ toneladas de a a b. En el intercambio en ciclo, que se usa en los demás casos y en particular cuando el problema está balanceado, una empresa compañera k hace el movimiento contrario, de modo que lo que recibe cada planta no cambia; es el ciclo de cuatro celdas del método MODI [2]. La planta destino o la empresa compañera se eligen con una probabilidad tipo softmax que favorece los cambios que abaratan el envío:

$$P(opción) \propto e^{-\Delta/\sigma},\ \ \Delta = (c_{ib} - c_{ia}) + (c_{ka} - c_{kb}) \qquad (17)$$

donde el segundo término solo aparece en el ciclo, σ es la desviación estándar de los costos de las rutas habilitadas (al menos 1) y el exponente se limita a [−30, 30]. La cantidad movida es el máximo posible con probabilidad 0,7 y una fracción aleatoria de él con probabilidad 0,3. Un movimiento que empeora el costo sigue siendo posible, lo que mantiene la diversidad.

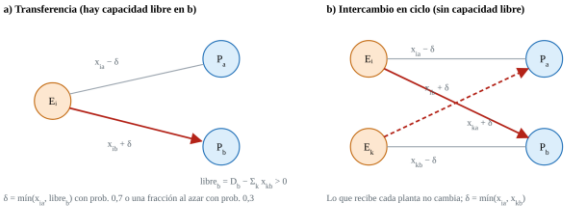


Fig. 11. Movimientos de mutación: transferencia hacia una planta con capacidad libre e intercambio en ciclo entre dos empresas.

Reemplazo y reparación final. La nueva población está formada por la élite y los hijos mutados; toda la población se repara y se evalúa de nuevo, y se conserva aparte la mejor solución de la corrida. Al terminar, a esa solución se le aplica una reparación de capacidad que elimina excesos residuales: mientras alguna planta supere su demanda en más de 10^{-9} t, se mueve el residuo hacia la planta con capacidad libre que menos encarece el envío. Si no queda capacidad libre, el exceso se mantiene y la corrida se reporta como no factible.

E. *Ciclo evolutivo y parámetros*

La Fig. 12 resume el algoritmo. En cada generación se registra el mejor costo, el costo promedio, la mejor aptitud y las plantas excedidas, que forman la curva de convergencia; cada 10 generaciones se informa el avance a la interfaz. El criterio de parada es el número de generaciones G, de modo que el tiempo es predecible. La Tabla LX lista los parámetros que el usuario configura desde la interfaz, como pide el enunciado.

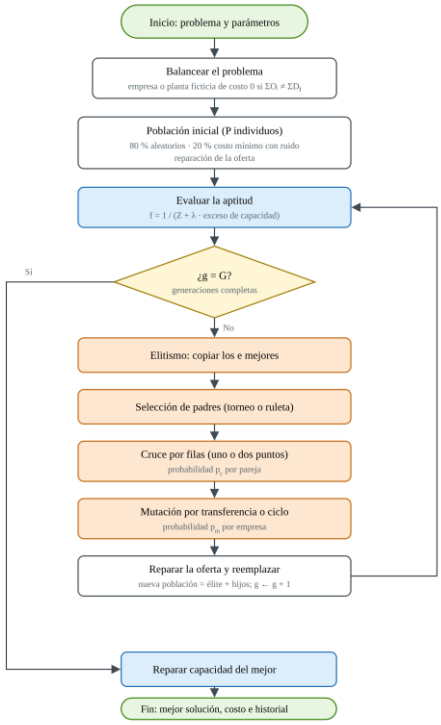


Fig. 12. Diagrama de flujo del algoritmo genético implementado.

TABLA LX
PARÁMETROS DEL ALGORITMO GENÉTICO

Parámetro	Valor por defecto	Rango permitido
Tamaño de población P	100	≥ 2
Generaciones G	300	≥ 1
Tasa de cruce pc	0,85	[0, 1]
Tasa de mutación pm	0,05	[0, 1]
Método de selección	Torneo	Torneo o ruleta
Participantes por torneo k	3	2 a 20
Tipo de cruce	Dos puntos	Uno o dos puntos
Elitismo e	2	$0 \leq e < P$
Penalización λ	1.000.000 COP/t	> 0
Semilla s	42	Entero ≥ 0
Corridas N	30	1 a 50
Fracción heurística inicial	0,2	Fija

Los Listados 6 a 8 muestran cómo se resuelve el mismo modelo con el algoritmo genético (método ejecutar). En el Listado 6 se crea la población inicial, se repara la oferta de cada individuo (líneas 99 a 102), se evalúa la aptitud (línea 103) y se guarda el mejor individuo (líneas 104 y 105).

Listado 6. Población inicial, reparación y evaluación (genetic_algorithm.py, AlgoritmoGenetico.ejecutar).

```
 98 with Cronometro() as cronometro:
 99     poblacion = reparar_oferta(
100         poblacion_inicial(oferta, demanda, costos,
                habilitadas, p.tamano_poblacion, rng,
101                 ficticias=self._celdas_ficticias(bal, m,
                        n)),
102         oferta, habilitadas)
103     evaluacion = evaluar(poblacion, costos, demanda,
            p.penalizacion)
104     mejor =
            poblacion[np.argmax(evaluacion.aptitud)].copy()
105     mejor_aptitud = float(evaluacion.aptitud.max())
```

El Listado 7 es una generación: conserva la élite (líneas 108 y 109), selecciona 2q padres por torneo o ruleta (líneas 111 a 113), cruza las parejas por filas (líneas 114 y 115), recorta y muta los hijos (líneas 116 y 117) y forma la nueva población con la oferta reparada (línea 118).

Listado 7. Operadores genéticos de cada generación (genetic_algorithm.py, AlgoritmoGenetico.ejecutar).

```
107 for generacion in range(1, p.num_generaciones + 1):
108     orden = np.argsort(-evaluacion.aptitud)
109     elite = poblacion[orden[:p.elitismo]]
110
111     faltan = p.tamano_poblacion - p.elitismo
112     pares = (faltan + 1) // 2
113     padres = seleccionar(evaluacion.aptitud, 2 * pares,
            p.seleccion, p.tamano_torneo, rng)
114     hijo_a, hijo_b = cruzar(poblacion[padres[:pares]],
            poblacion[padres[pares:]],
115             p.tasa_cruce, p.cruce, rng)
116     hijos = np.concatenate([hijo_a, hijo_b])[:faltan]
117     hijos = mutar(hijos, p.tasa_mutacion, costos, demanda,
            habilitadas, rng)
118     poblacion = reparar_oferta(np.concatenate([elite,
            hijos]), oferta, habilitadas)
```

El Listado 8 evalúa la nueva población, actualiza la mejor solución de toda la corrida (líneas 121 a 124) y registra el mejor costo, el promedio, la aptitud y las plantas excedidas, que forman la curva de convergencia (líneas 125 a 127). Al terminar las generaciones, a la mejor solución se le aplica la reparación de capacidad.

Listado 8. Evaluación y registro de la curva de convergencia (genetic_algorithm.py, AlgoritmoGenetico.ejecutar).

```
120 evaluacion = evaluar(poblacion, costos, demanda,
        p.penalizacion)
121 indice = int(np.argmax(evaluacion.aptitud))
122 if evaluacion.aptitud[indice] > mejor_aptitud:
123     mejor_aptitud = float(evaluacion.aptitud[indice])
124     mejor = poblacion[indice].copy()
125 historial.append(Generacion(generacion,
        float(evaluacion.costo[indice]),
126         float(evaluacion.costo.mean()),
              float(evaluacion.aptitud[indice]),
127         int(evaluacion.plantas_excedidas[indice])))
```

Todo el azar del algoritmo proviene de un único generador pseudoaleatorio inicializado con la semilla, de modo que con los mismos datos, parámetros y semilla el AG produce exactamente la misma solución y la misma curva (RNF07). Los parámetros se guardan con cada corrida en la base de datos.

F. Múltiples corridas y estadísticas

Como el AG es estocástico, el sistema ejecuta N corridas independientes con semillas consecutivas s, s + 1, …, s + N − 1 (RF13) y calcula el mejor costo entre las corridas factibles, el promedio, la desviación estándar muestral, el peor costo, el tiempo promedio y el número de corridas factibles:

$$Z = \frac{1}{N}\sum_{r=1}^{N} Z_r\,, \quad \sigma = \sqrt{\frac{1}{N-1}\sum_{r=1}^{N}(Z_r - Z)^2} \qquad (18)$$

La calidad frente al óptimo se mide con la desviación porcentual (gap) del AG respecto al método exacto:

$$gap = \frac{Z_{AG} - Z^*}{Z^*} \times 100\ \% \qquad (19)$$

VII. ARQUITECTURA Y DISEÑO DEL SOFTWARE (FASE 4)

A. Selección del estilo arquitectónico

Un estilo arquitectónico es una colección de decisiones de diseño, aplicables en un contexto determinado, que restringen las decisiones específicas y producen beneficios en cada sistema resultante [26]. La elección se hizo a partir de los requerimientos, los atributos de calidad y las restricciones, con los criterios de la guía del curso [34]: sistema monolítico o distribuido, eficiencia, curva de aprendizaje, capacidad de prueba, capacidad de modificación, modularidad, disponibilidad, uso de recursos y adaptabilidad. La Tabla LXI resume la evaluación de los estilos considerados [23], [25].

TABLA LXI
EVALUACIÓN DE ESTILOS ARQUITECTÓNICOS PARA EL SISTEMA

Estilo	Ajuste al sistema	Decisión
Flujo de datos (tubos y filtros)	La optimización no es una cadena fija de transformaciones: el usuario navega, edita y vuelve a resolver.	Descartado
Centrado en datos (pizarrón o repositorio)	La base de datos guarda resultados, pero no coordina a los componentes ni dispara su ejecución.	Descartado
Componentes independientes (eventos)	No hay productores y consumidores de eventos; las operaciones son solicitudes síncronas del analista.	Descartado
Microservicios y orientado a servicios	La aplicación es local y monolítica (RNF12); distribuirla añadiría despliegue y red sin beneficio.	Descartado
Llamada y retorno en capas (multinivel)	Cada nivel pide servicios al de abajo y recibe la respuesta; separa interfaz, servicios, algoritmos y datos (RNF04).	Seleccionado

Se seleccionó la **arquitectura por capas**, de la familia de llamada y retorno. La capa de presentación solo conoce a los servicios; los servicios coordinan la lógica de optimización y el acceso a datos; la lógica de dominio no conoce la base de datos ni la interfaz, lo que permite probar el Simplex, el AG y la sensibilidad de forma aislada, como se hizo en la verificación de la Sección V. Algunas vistas usan directamente funciones puras del dominio, sin estado (por ejemplo, el cálculo del gap o la verificación del balance), lo que corresponde a una variante de capas relajadas que se señala explícitamente en el diagrama.

El patrón de diseño **Singleton** [22] se aplica dentro de las capas y es compatible con el estilo: los siete servicios de aplicación, el gestor de base de datos y la configuración tienen una única instancia en todo el proceso, creada por una metaclase con un candado reentrante, segura entre los hilos con los que Streamlit atiende cada sesión, de modo que todas las vistas comparten el mismo motor de conexiones y la misma configuración sin crear objetos en cada recarga de Streamlit. El Singleton es un patrón de creación de objetos y no un estilo arquitectónico, por lo que no cambia la organización en capas sino que define cómo se instancian los componentes de las capas de aplicación y de datos.

B. Diagrama de arquitectura

La descripción de la arquitectura sigue la práctica recomendada IEEE 1471 [27], que pide documentar el sistema mediante vistas dirigidas a las partes interesadas: la vista de capas de esta sección, la vista lógica del diagrama de clases, la vista de datos del modelo entidad-relación y la vista dinámica de los diagramas de secuencia.

La Fig. 13 presenta la vista completa de la arquitectura y las Figs. 14 a 18 amplían cada capa para que los componentes sean legibles a ancho de columna; en ellas cada tramo de una capa ancha se dibuja debajo del anterior, separado por una línea punteada. Las flechas de color representan las llamadas de cada vista al servicio del mismo color; las flechas oscuras, las llamadas de los servicios a las capas inferiores; y las punteadas, los accesos directos de la variante relajada. El distintivo rojo con el número 1 marca las clases con una sola instancia.

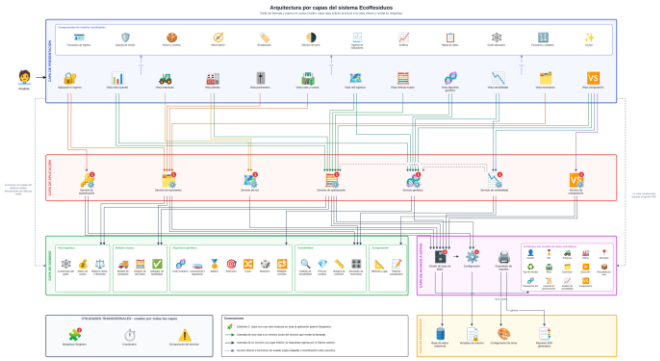


Fig. 13. Arquitectura por capas del sistema EcoResiduos (vista completa).

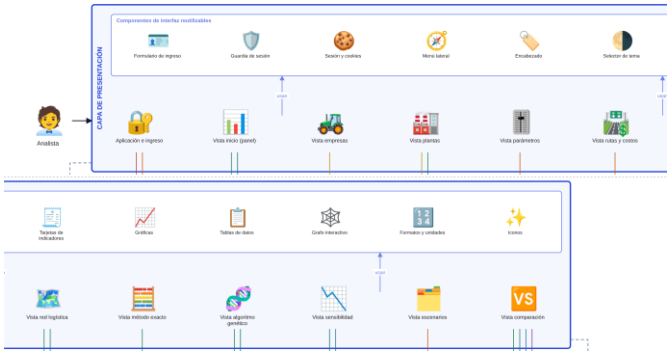


Fig. 14. Capa de presentación: el analista, las 12 vistas y los 12 componentes de interfaz reutilizables (dos tramos).

La capa de presentación contiene las 12 vistas de la aplicación (ingreso, inicio, empresas, plantas, parámetros, rutas y costos, red logística, método exacto, algoritmo genético, sensibilidad, escenarios y comparación) y los componentes reutilizables: formulario de ingreso, guardia de sesión, sesión y cookies, menú lateral, encabezado, selector de tema, tarjetas de indicadores, gráficas, tablas, grafo interactivo, formatos con unidades e iconos.

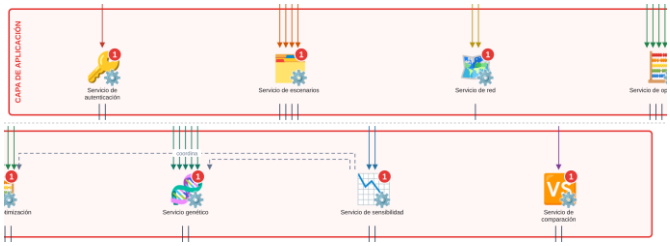


Fig. 15. Capa de aplicación: los siete servicios de instancia única y las llamadas que reciben de las vistas (dos tramos).

La capa de aplicación tiene un servicio por área funcional: autenticación, escenarios, red, optimización, genético, sensibilidad y comparación. Cada servicio abre una sesión de base de datos por operación, ejecuta la lógica de dominio y devuelve objetos de transferencia de datos a la vista. El servicio de sensibilidad coordina a los servicios de optimización y genético para las simulaciones.

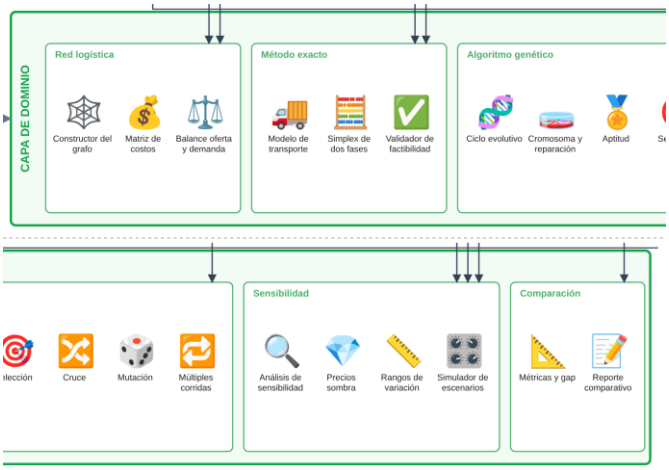


Fig. 16. Capa de dominio: paquetes de red logística, método exacto, algoritmo genético, sensibilidad y comparación (dos tramos).

La capa de dominio agrupa los 19 módulos de la lógica de optimización en cinco paquetes: red logística (constructor del grafo, matriz de costos y balance), método exacto (modelo de transporte, Simplex de dos fases y validador), algoritmo genético (ciclo evolutivo, cromosoma y reparación, aptitud, selección, cruce, mutación y múltiples corridas), sensibilidad (análisis, precios sombra, rangos y simulador) y comparación (métricas y reporte).

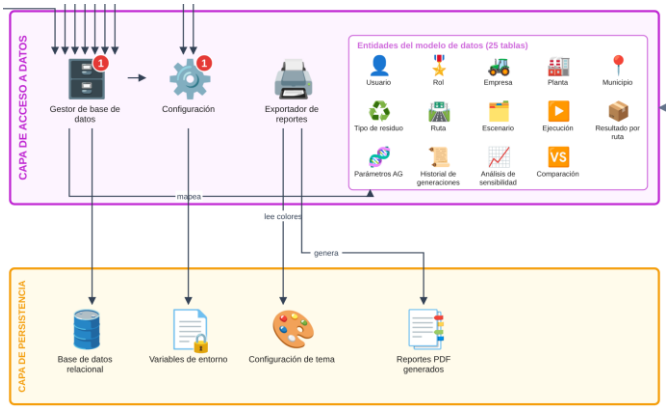


Fig. 17. Capas de acceso a datos y de persistencia: gestor de base de datos, configuración, exportador de reportes, entidades y almacenamiento.

La capa de acceso a datos contiene el gestor de base de datos (motor y sesiones), la configuración leída de las variables de entorno, el exportador de reportes PDF y las 25 entidades del mapeo objeto-relacional. La capa de persistencia está formada por la base de datos relacional, el archivo de variables de entorno, la configuración del tema de colores y los reportes PDF generados.

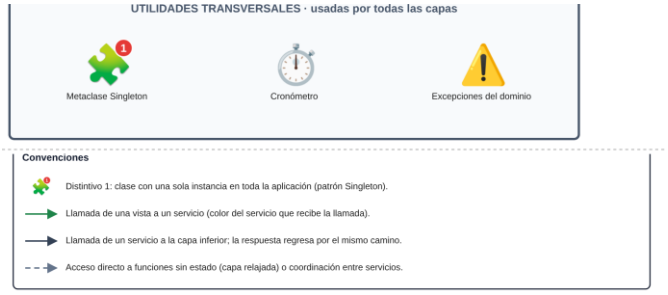


Fig. 18. Utilidades transversales (metaclase Singleton, cronómetro y excepciones del dominio) y convenciones del diagrama.

C. Tecnologías y organización del código

La Tabla LXII lista las tecnologías, todas de código abierto. Los algoritmos de optimización (Simplex, AG y sensibilidad) se implementaron desde cero sobre NumPy [16]; NetworkX [17] se usa solo para construir y resumir el grafo. La interfaz se construyó con Streamlit [19], la persistencia con SQLAlchemy [20] sobre PostgreSQL [21], y la Tabla LXIII describe los paquetes del código.

TABLA LXII
TECNOLOGÍAS UTILIZADAS

Componente	Tecnología	Uso en el sistema
Lenguaje	Python 3.11 o superior	Todo el sistema
Interfaz web	Streamlit 1.66	Vistas, navegación, sesión y tema claro u oscuro
Cálculo numérico	NumPy	Simplex, AG, sensibilidad y simulación
Grafos	NetworkX	Construcción del grafo G = (V, A)
Visualización	Plotly	Gráficas interactivas y dibujo de la red
Persistencia	SQLAlchemy 2.0 y psycopg2	Mapeo objeto-relacional y conexión
Base de datos	PostgreSQL 14 o superior	25 tablas en tercera forma normal y 2 vistas
Seguridad	bcrypt, HMAC-SHA256 y python-dotenv	Contraseñas cifradas, token de sesión y credenciales fuera del código
Reportes	fpdf2	Reporte comparativo en PDF
Control de versiones	Git	Repositorio con ramas por módulo

TABLA LXIII
PAQUETES DEL CÓDIGO Y RESPONSABILIDADES

Paquete	Responsabilidad
frontend/pages	Las 12 vistas de la aplicación (una por página).
frontend/components	Componentes de interfaz reutilizables, sesión, tema y estilos.
backend/services	Servicios de instancia única que coordinan dominio y datos.
backend/exact	Modelo de transporte, Simplex de dos fases y validador de factibilidad.
backend/genetic	Algoritmo genético: cromosoma, aptitud, selección, cruce, mutación y múltiples corridas.
backend/network	Grafo, matriz de costos unitarios y balance de oferta y demanda.
backend/sensitivity	Precios sombra, rangos de variación y simulación de escenarios.
backend/comparison	Gap, escalabilidad y estructura del reporte comparativo.
backend/data_io	Exportación del reporte en PDF.
backend (raíz)	Configuración, base de datos (entidades) y autenticación.
backend/utils	Metaclase Singleton, cronómetro y jerarquía de excepciones.
database	Script SQL que crea la base y carga el caso base.

D. Diagrama de clases

El diagrama de clases se obtuvo directamente del código y contiene 133 elementos: 25 entidades, 9 clases de instancia única, 40 clases de datos, 2 enumeraciones, una metaclase, 22 módulos de funciones del backend y 24 módulos de interfaz, cada uno con el tipo de sus atributos, parámetros y retornos. Se organiza en cuatro franjas: interfaz, servicios, algoritmos y entidades. La Fig. 19 muestra la vista completa y las Figs. 20 a 24 amplían cada franja en tramos apilados; las imágenes conservan alta resolución para ampliarlas en el documento. Las Tablas LXIV a LXVII listan los miembros públicos de las clases principales con sus firmas.

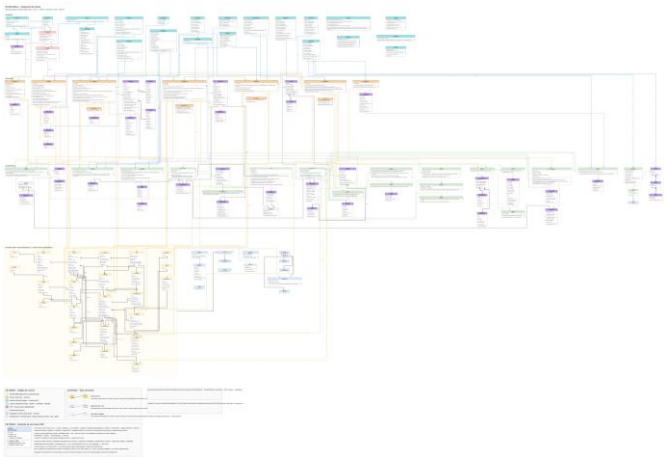


Fig. 19. Diagrama de clases completo (vista general de las cuatro franjas y las leyendas).

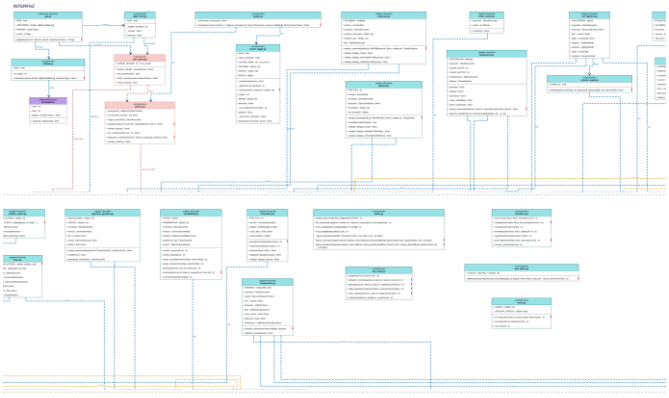


Fig. 20. Franja de interfaz del diagrama de clases: aplicación, páginas y componentes de Streamlit (tres tramos).

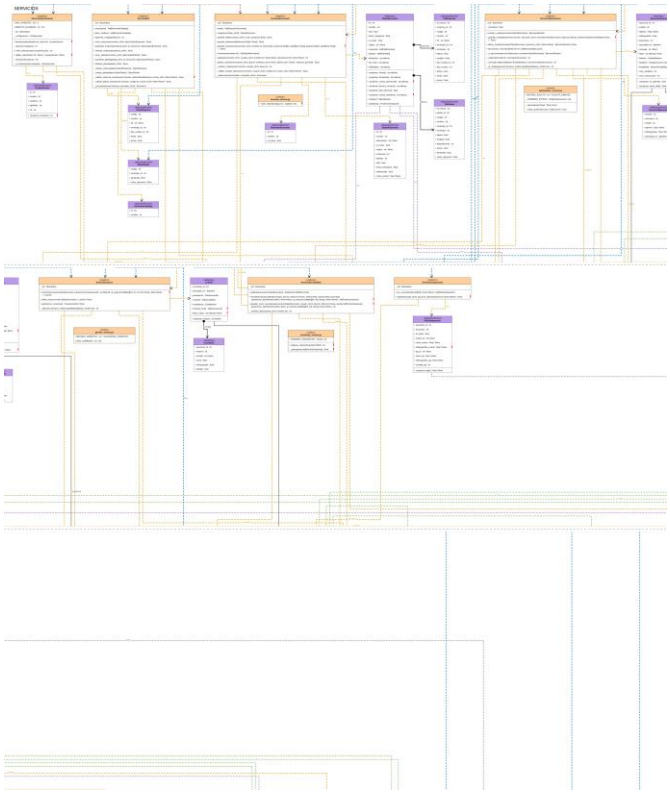


Fig. 21. Franja de servicios: los siete servicios de instancia única y sus objetos de transferencia de datos (tres tramos).

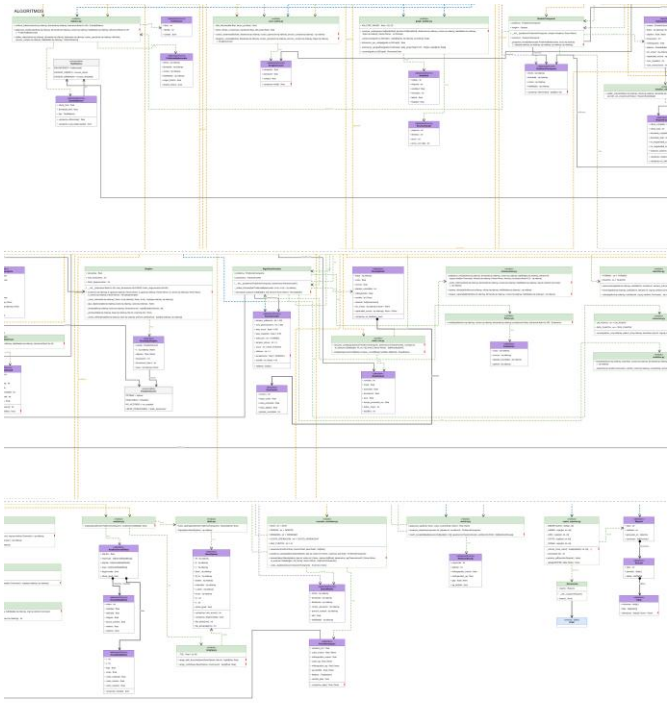
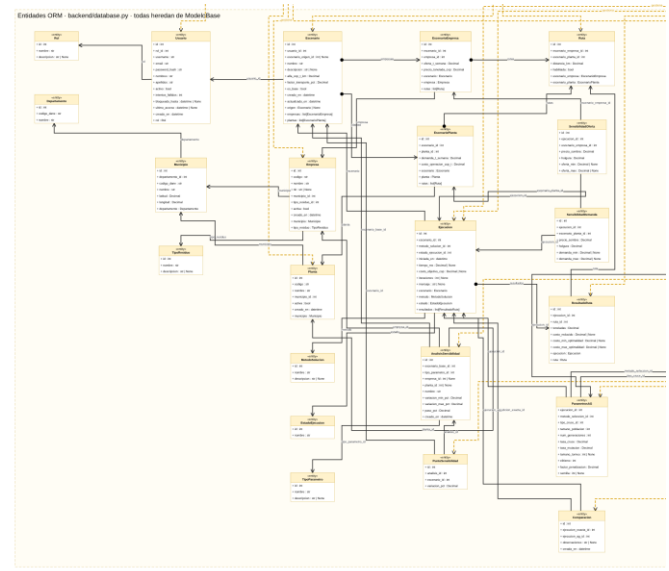


Fig. 22. Franja de algoritmos: método exacto, algoritmo genético, red, sensibilidad, comparación y utilidades (tres tramos).



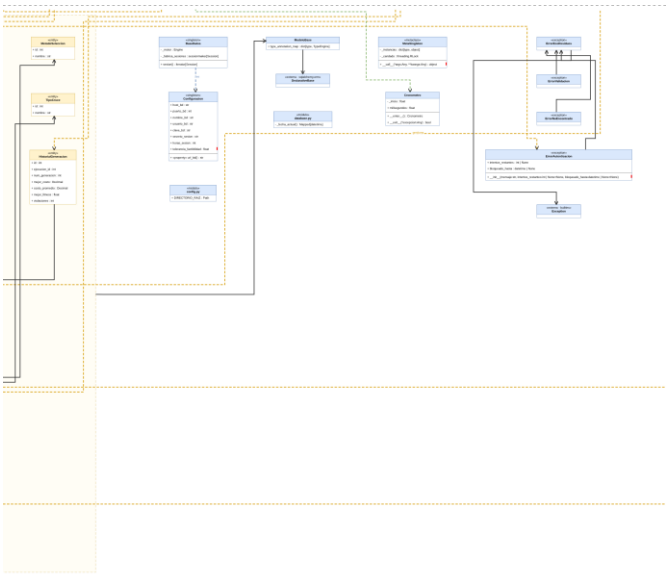


Fig. 23. Franja de entidades del mapeo objeto-relacional, que heredan de ModeloBase (dos tramos).

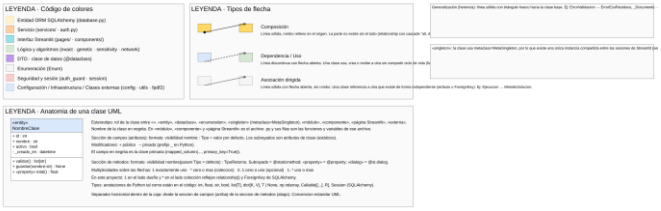


Fig. 24. Leyendas del diagrama de clases: código de colores, tipos de flecha y anatomía de una clase.

TABLA LXIV
CLASES DE INSTANCIA ÚNICA Y SUS MÉTODOS PÚBLICOS

Clase	Miembros públicos (firma con tipos)
ServicioAutenticacion *backend/auth.py*	+ autenticar(identificador: str, clave: str): UsuarioSesion + emitir_token(usuario: UsuarioSesion): str + validar_token(token: str | None): UsuarioSesion | None

Clase	Miembros públicos (firma con tipos)
ServicioEscenarios *backend/services/scenario_service.py*	+ listar(): list[ResumenEscenario] + cargar(escenario_id: int): DatosEscenario + guardar_alfa(escenario_id: int, costo_transporte: float): None + guardar_distancias(distancias: dict[int, float]): None + guardar_parametros(escenario_id: int, nombre: str, factor: float, empresas: dict[int, tuple[float, float]], plantas: dict[int, tuple[float, float]]): None + resumenes(metodo: str): list[TarjetaEscenario] + duplicar(escenario_id: int, usuario_id: int, nombre: str | None=None, descripcion: str | None=None): int + aplicar_variacion(escenario_id: int, tipo: str, empresa_id: int | None, planta_id: int | None, variacion_pct: float): None
ServicioRed *backend/services/network_service.py*	+ municipios(): list[ElementoCatalogo] + tipos_residuo(): list[ElementoCatalogo] + siguiente_codigo(prefijo: str): str + crear_empresa(escenario_id: int, datos: DatosEmpresa): None + actualizar_empresa(empresa_id: int, id_vinculo: int, datos: DatosEmpresa): None + eliminar_empresa(empresa_id: int): None + crear_planta(escenario_id: int, datos: DatosPlanta): None + actualizar_planta(planta_id: int, id_vinculo: int, datos: DatosPlanta): None + eliminar_planta(planta_id: int): None
ServicioOptimizacion *backend/services/optimization_service.py*	+ resolver_exacto(escenario: DatosEscenario): EjecucionExacta + ultima_exacta(escenario: DatosEscenario, ejecucion_id: int | None=None): EjecucionExacta | None + ejecuciones_recientes(limite: int=4): list[ResumenEjecucion]
ServicioGenetico *backend/services/genetic_service.py*	+ resolver(escenario: DatosEscenario, parametros: ParametrosAG, corridas: int, al_avanzar: Callable[[int, int, int, int], None] | None=None): LoteAG + ultimo_lote(escenario: DatosEscenario): LoteAG | None + parametros_recientes(): ParametrosAG | None
ServicioSensibilidad *backend/services/sensitivity_service.py*	+ analizar(escenario: DatosEscenario): AnalisisSensibilidad | None + simular(escenario: DatosEscenario, tipo: str, indice: int | None, minimo: float, maximo: float, paso: float, parametros_ag: ParametrosAG | None=None, al_avanzar: Callable[[int, int], None] | None=None): list[PuntoSimulacion] + guardar_como_escenarios(escenario: DatosEscenario, usuario_id: int, tipo: str, indice: int | None, puntos: list[PuntoSimulacion], parametros_ag: ParametrosAG | None, al_avanzar: Callable[[int, int], None] | None=None): int
ServicioComparacion *backend/services/comparison_service.py*	+ por_escenarios(ids: list[int] | None=None): list[FilaComparacion] + registrar(exacta_id: int, ag_id: int, observaciones: str | None=None): None
BaseDatos *backend/database.py*	+ sesion(): Iterator[Session]
Configuracion *backend/config.py*	+ host_bd: str + puerto_bd: int + nombre_bd: str + usuario_bd: str + clave_bd: str + secreto_sesion: str + horas_sesion: int + tolerancia_factibilidad: float + «property» url_bd(): str
MetaSingleton *backend/utils/singleton.py*	+ __call__(*args: Any, **kwargs: Any): object

TABLA LXV
CLASES PRINCIPALES DEL DOMINIO

Clase	Atributos y métodos públicos
ProblemaTransporte *dataclass(frozen)*	+ oferta: np.ndarray + demanda: np.ndarray + costos: np.ndarray + habilitadas: np.ndarray + «property» dimension(): tuple[int, int]
SolucionTransporte *dataclass*	+ estado: EstadoSolucion + flujos: np.ndarray | None + objetivo: float | None + iteraciones: int + milisegundos: float + balance: EstadoBalance + sin_enviar: np.ndarray | None + capacidad_ociosa: np.ndarray | None + num_variables: int + num_restricciones: int
ModeloTransporte *clase*	+ problema: ProblemaTransporte + simplex: Simplex + __init__(problema: ProblemaTransporte, simplex: Simplex | None=None): None + resolver(): SolucionTransporte
Simplex *clase*	+ tolerancia: float + max_iteraciones: int + limite_degeneracion: int + __init__(tolerancia: float=1e-09, max_iteraciones: int=100000, limite_degeneracion: int=50): None + resolver(c: np.ndarray, A_igual: np.ndarray | None=None, b_igual: np.ndarray | None=None, A_menor: np.ndarray | None=None, b_menor: np.ndarray | None=None): ResultadoSimplex
ReporteFactibilidad *dataclass(frozen)*	+ oferta_cumplida: int + oferta_total: int + demanda_cumplida: int + demanda_total: int + no_negatividad_cumplida: int + no_negatividad_total: int + violacion_maxima: float + «property» violaciones(): int + «property» es_factible(): bool
ParametrosAG *dataclass(frozen)*	+ tamano_poblacion: int + num_generaciones: int + tasa_cruce: float + tasa_mutacion: float + seleccion: str + tamano_torneo: int + cruce: str + elitismo: int + penalizacion: float + semilla: int | None + validar(): list[str]
ResultadoAG *dataclass*	+ flujos: np.ndarray + costo: float + exceso: float + plantas_excedidas: int + milisegundos: float + semilla: int | None + historial: list[Generacion] + sin_enviar: np.ndarray | None + capacidad_ociosa: np.ndarray | None + «property» es_factible(): bool
AlgoritmoGenetico *clase*	+ problema: ProblemaTransporte + parametros: ParametrosAG + __init__(problema: ProblemaTransporte, parametros: ParametrosAG): None + ejecutar(al_avanzar: Callable[[int, int], None] | None=None): ResultadoAG
Estadisticas *dataclass*	+ corridas: int + mejor: float + promedio: float + desviacion: float + peor: float + tiempo_promedio_ms: float + indice_mejor: int + factibles: int
EstadoBalance *dataclass(frozen)*	+ oferta_total: float + demanda_total: float + tipo: TipoBalance + «property» diferencia(): float + «property» esta_balanceado(): bool
BaseOptima *dataclass*	+ M: np.ndarray + b: np.ndarray + c: np.ndarray + base: np.ndarray + B_inv: np.ndarray + duales: np.ndarray + reducidos: np.ndarray + x_base: np.ndarray + arcos: np.ndarray + m: int + n: int + oferta_igual: bool + «property» num_arcos(): int + «property» degenerada(): bool + fila_oferta(i: int): int + fila_demanda(j: int): int
AnalisisSensibilidad *dataclass*	+ objetivo: float + empresas: list[SensibilidadNodo] + plantas: list[SensibilidadNodo] + rutas: list[SensibilidadRuta] + degenerada: bool + oferta_igual: bool
PuntoSimulacion *dataclass*	+ variacion_pct: float + costo_exacto: float | None + milisegundos_exacto: float + costo_ag: float | None + milisegundos_ag: float | None + ag_factible: bool | None + balance: TipoBalance + cambia_plan: bool + «property» gap(): float | None

Clase	Atributos y métodos públicos
MedicionEscala *dataclass(frozen)*	+ empresas: int + plantas: int + milisegundos_exacto: float + milisegundos_ag: float + gap: float | None + ag_factible: bool

TABLA LXVI

MÓDULOS DE FUNCIONES DEL BACKEND

Módulo	Funciones públicas (firma con tipos)
chromosome.py *backend/genetic*	+ poblacion_inicial(oferta: np.ndarray, demanda: np.ndarray, costos: np.ndarray, habilitadas: np.ndarray, tamano: int, rng: np.random.Generator, ficticias: np.ndarray | None=None, fraccion_heuristica: float=0.2): np.ndarray + reparar_oferta(individuos: np.ndarray, oferta: np.ndarray, habilitadas: np.ndarray): np.ndarray + reparar_capacidad(individuo: np.ndarray, demanda: np.ndarray, costos: np.ndarray, habilitadas: np.ndarray): np.ndarray
crossover.py *backend/genetic*	+ UN_PUNTO: str + DOS_PUNTOS: str + cruzar(padres_a: np.ndarray, padres_b: np.ndarray, tasa: float, tipo: str, rng: np.random.Generator): tuple[np.ndarray, np.ndarray]
fitness.py *backend/genetic*	+ evaluar(poblacion: np.ndarray, costos: np.ndarray, demanda: np.ndarray, penalizacion: float, tolerancia: float=1e-06): Evaluacion
mutation.py *backend/genetic*	+ mutar(poblacion: np.ndarray, tasa: float, costos: np.ndarray, demanda: np.ndarray, habilitadas: np.ndarray, rng: np.random.Generator): np.ndarray
selection.py *backend/genetic*	+ TORNEO: str + RULETA: str + seleccionar(aptitud: np.ndarray, cantidad: int, metodo: str, tamano_torneo: int, rng: np.random.Generator): np.ndarray + torneo(aptitud: np.ndarray, cantidad: int, tamano: int, rng: np.random.Generator): np.ndarray + ruleta(aptitud: np.ndarray, cantidad: int, rng: np.random.Generator): np.ndarray
multi_run.py *backend/genetic*	+ ejecutar_corridas(problema: ProblemaTransporte, parametros: ParametrosAG, corridas: int, al_avanzar: Callable[[int, int, int, int], None] | None=None): list[ResultadoAG] + estadisticas(costos: list[float], tiempos_ms: list[float], factibles: list[bool]): Estadisticas
balance.py *backend/network*	+ verificar_balance(oferta: np.ndarray, demanda: np.ndarray, tolerancia: float=1e-06): EstadoBalance + balancear_problema(oferta: np.ndarray, demanda: np.ndarray, costos: np.ndarray, habilitadas: np.ndarray, tolerancia: float=1e-06): ProblemaBalanceado + validar_datos(oferta: np.ndarray, demanda: np.ndarray, distancias: np.ndarray, costos_operacion: np.ndarray, alfa: float, precios_compra: np.ndarray, habilitadas: np.ndarray): list[Verificacion]
cost_matrix.py *backend/network*	+ alfa_efectivo(alfa: float, factor_pct: float): float + factor_desde_costo(costo_transporte: float, alfa_base: float): float + costos_unitarios(alfa: float, distancias: np.ndarray, costos_operacion: np.ndarray, precios_compra: np.ndarray): np.ndarray + desglose_costo(alfa: float, distancias: np.ndarray, costos_operacion: np.ndarray, precios_compra: np.ndarray, flujos: np.ndarray): DesgloseCosto
graph_builder.py *backend/network*	+ KM_POR_GRADO: float + construir_red(origenes: list[NodoRed], destinos: list[NodoRed], distancias: np.ndarray, costos: np.ndarray, habilitadas: np.ndarray, flujos: np.ndarray | None=None): nx.DiGraph + kilometros_por_unidad(grafo: nx.DiGraph): float + posiciones_geograficas(grafo: nx.DiGraph, radio_grupo: float=0.07): dict[str, tuple[float, float]] + resumir(grafo: nx.DiGraph): ResumenGrafo
duals.py *backend/sensitivity*	+ base_optima(problema: ProblemaTransporte): BaseOptima | None + flujos(base: BaseOptima): np.ndarray
ranging.py *backend/sensitivity*	+ rango_lado_derecho(base: BaseOptima, fila: int): tuple[float, float] + rango_costo(base: BaseOptima, columna: int): tuple[float, float]
analysis.py *backend/sensitivity*	+ analizar(problema: ProblemaTransporte): AnalisisSensibilidad | None
scenario_simulator.py *backend/sensitivity*	+ ALFA: str + OFERTA: str + DEMANDA: str + COSTO_OPERACION: str + MAX_PUNTOS: int + variaciones(minimo: float, maximo: float, paso: float): list[float] + problema_variado(datos: DatosModelo, tipo: str, indice: int | None, variacion_pct: float): ProblemaTransporte + simular(datos: DatosModelo, tipo: str, indice: int | None, valores: list[float], parametros_ag: ParametrosAG | None=None, al_avanzar: Callable[[int, int], None] | None=None): list[PuntoSimulacion] + gap(costo_ag: float | None, costo_exacto: float | None): float | None
metrics.py *backend/comparison*	+ instancia_aleatoria(empresas: int, plantas: int, semilla: int): ProblemaTransporte + medir_escalabilidad(tamanos: list[tuple[int, int]], parametros: ParametrosAG, semilla: int=2026): list[MedicionEscala]
solution_validator.py *backend/exact*	+ validar_solucion(flujos: np.ndarray, oferta: np.ndarray, demanda: np.ndarray, habilitadas: np.ndarray, tolerancia: float=1e-06, permitir_sin_enviar: bool=False): ReporteFactibilidad
report_exporter.py *backend/data_io*	+ generar_pdf(reporte: Reporte): bytes

TABLA LXVII

MÓDULOS DE LA INTERFAZ Y SUS FUNCIONES PÚBLICAS

Módulo	Funciones públicas
app.py *aplicación Streamlit*	+ RAIZ: Path + SECCIONES: dict[str, list[EntradaMenu]] + PAGINAS: list[st.Page] + actual: st.Page + pagina(archivo: str, titulo: str, ruta: str, inicial: bool=False): st.Page
auth_guard.py *componente*	+ COOKIE_SESION: str + usuario_actual(): UsuarioSesion | None + esta_autenticado(): bool + iniciar_sesion(usuario: UsuarioSesion): None + cerrar_sesion(): None
charts.py *componente*	+ progreso(porcentaje: float, delgada: bool=False): str + fila_utilizacion(codigo: str, nombre: str, usado: str, capacidad: str, porcentaje: float): str + barra_apilada(partes: list[tuple[float, str, float]]): str + red_portada(paleta: dict[str, str]): str + figura_convergencia(generaciones: list[int], mejor: list[float], promedio: list[float], optimo: float | None, paleta: dict[str, str]): go.Figure + figura_simulacion(etiquetas: list[str], exacto: list[float | None], genetico: list[float | None] | None, cambia_plan: list[bool], paleta: dict[str, str]): go.Figure
data_tables.py *componente*	+ Columna: TypeAlias = tuple[str, str] + tabla(columnas: list[Columna], filas: list[list[str]], pie: list[str] | None=None, clase: str='', ancho_minimo: int=420): str
formatters.py *componente*	+ numero(valor: float | None, decimales: int=0): str + cantidad(valor: float | None, max_decimales: int=2): str + moneda(valor: float | None): str + toneladas(valor: float | None, unidad: str='t'): str + segundos(milisegundos: float | None): str + porcentaje(valor: float | None, decimales: int=0): str + nombre_corto(nombre: str): str
header.py *componente*	+ mostrar(seccion: str, titulo: str, *, etiqueta_escenario: str | None='Escenario', acciones: Callable[[], None] | None=None): None
icons.py *componente*	+ TRAZOS: dict[str, str] + url_mascara(nombre: str, grosor: float | None=None): str + icono(nombre: str, tamano: int=18): str + css_iconos(): str
kpi_cards.py *componente*	+ insignia(texto: str, tipo: str='ok'): str + indicador_icono(etiqueta: str, valor: str, nota: str, icono: str): str + dato(etiqueta: str, valor: str, nota: str, resaltado: bool=False): str + rejilla_indicadores(tarjetas: list[str], compacta: bool=False): str + clave_valor(etiqueta: str, valor: str, simple: bool=False): str + verificacion(titulo: str, detalle: str, cumple: bool): str
login_form.py *componente*	+ RAIZ: Path + mostrar(): None
network_graph.py *componente*	+ PLANTILLA: Path + mostrar(grafo: nx.DiGraph, ver_flujos: bool, paleta: dict[str, str], alto: int=600): None
session.py *componente*	+ escenarios(): list[ResumenEscenario] + id_escenario_activo(): int | None + cargar_escenario(): DatosEscenario + avisar(mensaje: str, icono: str=': material/check_circle: '): None + mostrar_avisos(): None + leer_cookie(nombre: str): str | None + programar_cookie(nombre: str, valor: str, segundos_vida: int): None + escribir_cookies(): None

Módulo	Funciones públicas
sidebar.py *componente*	+ RAIZ: Path + uri_logo(): str + mostrar(secciones: dict[str, list[EntradaMenu]], actual: st.Page): None
theme_toggle.py *componente*	+ RAIZ: Path + HOJA_ESTILOS: Path + COOKIE_TEMA: str + COLORES: dict[str, str] + MODOS: dict[str, str] + RUTAS: list[str] + colores(nombre_modo: str): dict[str, str] + modo(): str + paleta(): dict[str, str] + alternar(): None + aplicar(): None + boton(clave: str='boton_tema'): None
algoritmo_genetico.py *página Streamlit*	+ SELECCIONES: dict[str, str] + CRUCES: dict[str, str] + escenario: DatosEscenario + servicio: ServicioGenetico + lote: LoteAG | None + exacta: EjecucionExacta | None + optimo: float | None + cargar_parametros(parametros: ParametrosAG, corridas: int=30): None + restablecer(): None + parametros_formulario(): ParametrosAG
comparacion.py *página Streamlit*	+ TAMANOS: list[tuple[int, int]] + escenario: DatosEscenario + exacta: EjecucionExacta | None + lote: LoteAG | None + seleccion: list[int] | None + filas: list[FilaComparacion] + costo_exacto: float | None + diferencia: float | None + mediciones: list[MedicionEscala] | None + construir_reporte(secciones: list[str]): Reporte + registrar_comparacion(): None
empresas.py *página Streamlit*	+ COLUMNAS: list[float] + servicio: ServicioRed + escenario: DatosEscenario + nombres_municipio: dict[int, str] + nombres_tipo: dict[int, str] + filas: list[FilaEmpresa] + campos_empresa(prefijo: str, fila: FilaEmpresa | None, codigo: str): DatosEmpresa + dialogo_crear(): None + dialogo_eliminar(fila: FilaEmpresa): None + dialogo_editar(fila: FilaEmpresa): None + POR_FILA: int
escenarios.py *página Streamlit*	+ servicio: ServicioEscenarios + tarjetas: list[TarjetaEscenario] + costo_base: float | None + seleccionados: list[int] + descripcion(t: TarjetaEscenario): str + variacion(t: TarjetaEscenario): str + abrir(escenario_id: int): None + duplicar(t: TarjetaEscenario): None + dialogo_nuevo(): None + INSIGNIAS_ESTADO: dict[str, tuple[str, str]]
inicio.py *página Streamlit*	+ MODULOS: list[tuple[str, str, str]] + escenario: DatosEscenario + servicio: ServicioOptimizacion + ejecucion: EjecucionExacta | None + lote: LoteAG | None + diferencia: float | None + balance: EstadoBalance + ETIQUETAS_ESTADO: dict[str, str]
metodo_exacto.py *página Streamlit*	+ COLORES_COSTO: tuple[tuple[str, str, float], ...] + escenario: DatosEscenario + servicio: ServicioOptimizacion + ejecucion: EjecucionExacta | None + PROPORCION: list[float]
parametros.py *página Streamlit*	+ escenario: DatosEscenario + CLAVE_COSTO: str + CLAVE_FACTOR: str + verificaciones: list[Verificacion] + balance: EstadoBalance + guardar(): None + validar(): None + acciones(): None + costo_cambiado(): None + factor_cambiado(): None + campo_numero(etiqueta: str, clave: str, valor: float, paso: float, caja: str): None + cabecera_tabla(titulo: str, columnas: tuple[tuple[str, str], ...]): list
plantas.py *página Streamlit*	+ POR_FILA: int + servicio: ServicioRed + escenario: DatosEscenario + ejecucion: EjecucionExacta | None + municipios: dict[int, str] + ids_municipio: list[int] + campos_planta(prefijo: str, fila: FilaPlanta | None, codigo: str): DatosPlanta + crear(datos: DatosPlanta): bool + dialogo_crear(): None + dialogo_editar(fila: FilaPlanta): None + dialogo_eliminar(fila: FilaPlanta): None
red_logistica.py *página Streamlit*	+ SOLUCIONES: list[str] + escenario: DatosEscenario + ejecucion: EjecucionExacta | None + lote: LoteAG | None + flujos: np.ndarray | None + origenes: list[NodoRed] + destinos: list[NodoRed] + grafo: nx.DiGraph + resumen: ResumenGrafo
rutas_costos.py *página Streamlit*	+ escenario: DatosEscenario + costos: np.ndarray + recalcular(): None + VISTAS: list[str]
sensibilidad.py *página Streamlit*	+ PARAMETROS: dict[str, str] + escenario: DatosEscenario + servicio: ServicioSensibilidad + analisis: AnalisisSensibilidad | None + parametros_ag: ParametrosAG + puntos: list[PuntoSimulacion] + nombre_empresa(i: int): str + nombre_planta(j: int): str + rango_toneladas(minimo: float, maximo: float): str + rango_costo(minimo: float, maximo: float): str + efecto(valor: float, mas: str, menos: str): str + explicacion(icono: str, titulo: str, pregunta: str, texto: str): str + conclusiones(frases: list[str]): str

Las páginas de Streamlit se ejecutan de arriba abajo en cada interacción; sus funciones privadas (prefijo _) dibujan secciones de la vista y no se listan.

E. Modelo de datos

La base de datos se diseñó con un modelo entidad-relación normalizado hasta tercera forma normal [30], con 25 tablas agrupadas en ocho módulos (Tabla LXVIII). Los datos que cambian entre escenarios (oferta, precio, demanda, costo de operación, α y distancias) viven en las tablas del escenario y no en los datos maestros, de modo que un escenario derivado nunca altera el caso base. Los valores derivados no se almacenan: el costo unitario se calcula en la vista v_costo_unitario y la comparación en la vista v_comparacion. Las Figs. 25 a 27 presentan el diagrama entidad-relación en dos mitades y sus convenciones.

TABLA LXVIII
TABLAS DE LA BASE DE DATOS POR MÓDULO

Módulo	Tablas	Contenido
Seguridad	rol, usuario	Usuarios con contraseña cifrada, intentos fallidos y bloqueo.
Datos maestros	departamento, municipio, tipo_residuo, empresa, planta	Datos fijos de la red y su ubicación.
Escenarios	escenario, escenario_empresa, escenario_planta, ruta	Parámetros del modelo por escenario.
Ejecuciones	ejecucion, resultado_ruta	Estado, costo, tiempo y flujos de cada solución.
Algoritmo genético	parametros_ag, historial_generacion	Parámetros de cada corrida y su curva de convergencia.
Sensibilidad	sensibilidad_oferta, sensibilidad_demanda, analisis_sensibilidad, punto_sensibilidad	Precios sombra, rangos y simulaciones.
Comparación	comparacion	Pares de ejecuciones exacta y genética comparadas.
Catálogos	metodo_solucion, estado_ejecucion, metodo_seleccion, tipo_cruce, tipo_parametro	Valores de dominio.

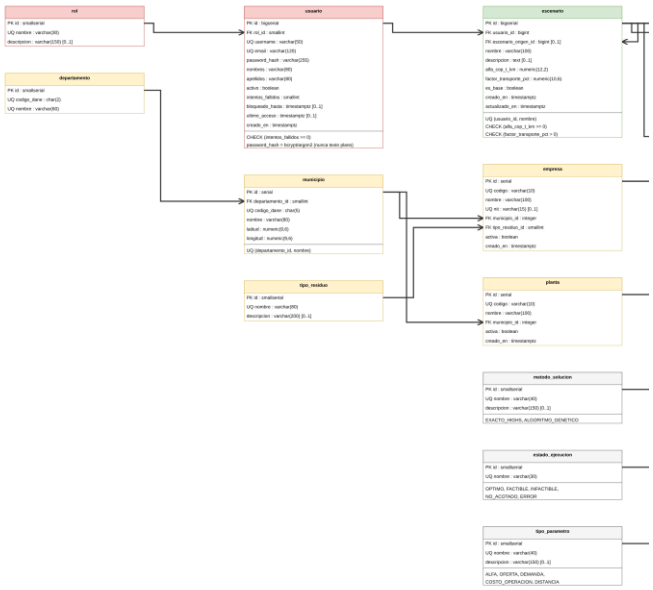


Fig. 25. Diagrama entidad-relación, mitad izquierda: seguridad, datos maestros, escenario y catálogos.

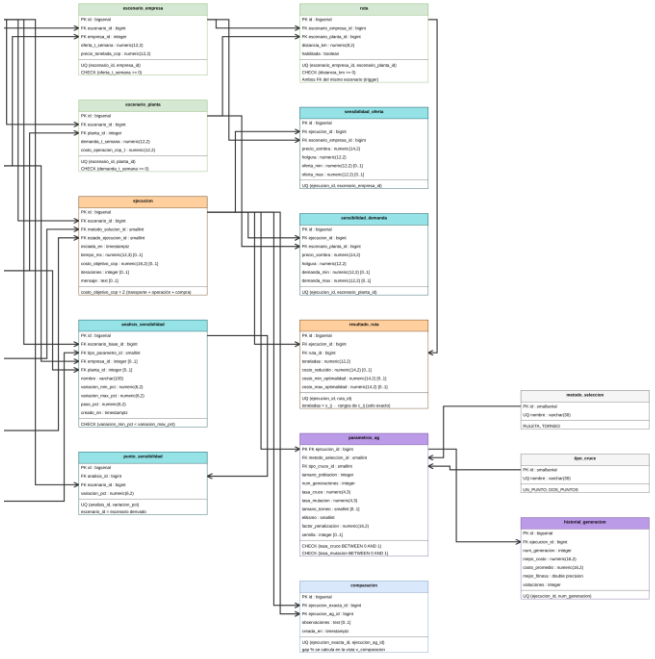


Fig. 26. Diagrama entidad-relación, mitad derecha: escenarios, ejecuciones, sensibilidad, algoritmo genético y comparación.

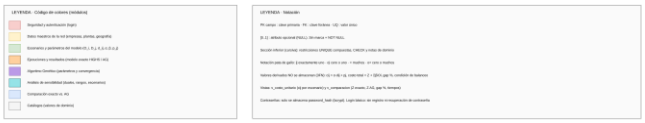


Fig. 27. Convenciones del diagrama entidad-relación: código de colores por módulo y notación.

Como el diagrama entidad-relación no es legible en su totalidad a ancho de columna, la Tabla LXIX presenta el diccionario de datos completo, con todas las columnas de las 25 tablas, su tipo y sus restricciones. Los archivos fuente editables de todos los diagramas de este informe (.drawio y .mmd) se entregan en la carpeta docs/diagramas del repositorio.

TABLA LXIX
DICCIONARIO DE DATOS DE LA BASE DE DATOS

Tabla	Columnas, tipos y restricciones
('b', 'rol')	PK id: smallserial UQ nombre: varchar(30) descripcion: varchar(150) [0..1]
('b', 'usuario')	PK id: bigserial FK→rol rol_id: smallint UQ username: varchar(50) UQ email: varchar(120) password_hash: varchar(255) nombres: varchar(80) apellidos: varchar(80) activo: boolean intentos_fallidos: smallint bloqueado_hasta: timestamptz [0..1] ultimo_acceso: timestamptz [0..1] creado_en: timestamptz
('b', 'departamento')	PK id: smallserial UQ codigo_dane: char(2) UQ nombre: varchar(60)
('b', 'municipio')	PK id: serial FK→departamento departamento_id: smallint UQ codigo_dane: char(5) nombre: varchar(80) latitud: numeric(9,6) longitud: numeric(9,6) *UNIQUE (departamento_id, nombre)*
('b', 'tipo_residuo')	PK id: smallserial UQ nombre: varchar(80) descripcion: varchar(200) [0..1]
('b', 'empresa')	PK id: serial UQ codigo: varchar(10) nombre: varchar(100) UQ nit: varchar(15) [0..1] FK→municipio municipio_id: integer FK→tipo_residuo tipo_residuo_id: smallint activa: boolean creado_en: timestamptz
('b', 'planta')	PK id: serial UQ codigo: varchar(10) nombre: varchar(100) FK→municipio municipio_id: integer activa: boolean creado_en: timestamptz
('b', 'escenario')	PK id: bigserial FK→usuario usuario_id: bigint FK→escenario escenario_origen_id: bigint [0..1] nombre: varchar(100) descripcion: text [0..1] alfa_cop_t_km: numeric(12,2) factor_transporte_pct: numeric(10,6) es_base: boolean creado_en: timestamptz actualizado_en: timestamptz *UNIQUE (usuario_id, nombre)*
('b', 'escenario_empresa')	PK id: bigserial FK→escenario escenario_id: bigint FK→empresa empresa_id: integer oferta_t_semana: numeric(12,2) precio_tonelada_cop: numeric(12,2) *UNIQUE (escenario_id, empresa_id)*
('b', 'escenario_planta')	PK id: bigserial FK→escenario escenario_id: bigint FK→planta planta_id: integer demanda_t_semana: numeric(12,2) costo_operacion_cop_t: numeric(12,2) *UNIQUE (escenario_id, planta_id)*
('b', 'ruta')	PK id: bigserial FK→escenario_empresa escenario_empresa_id: bigint FK→escenario_planta escenario_planta_id: bigint distancia_km: numeric(8,2) habilitada: boolean *UNIQUE (escenario_empresa_id, escenario_planta_id)*
('b', 'metodo_solucion')	PK id: smallserial UQ nombre: varchar(40) descripcion: varchar(150) [0..1]
('b', 'estado_ejecucion')	PK id: smallserial UQ nombre: varchar(30)
('b', 'metodo_seleccion')	PK id: smallserial UQ nombre: varchar(30)

Tabla	Columnas, tipos y restricciones
('b', 'tipo_cruce')	PK id: smallserial UQ nombre: varchar(30)
('b', 'tipo_parametro')	PK id: smallserial UQ nombre: varchar(40) descripcion: varchar(150) [0..1]
('b', 'ejecucion')	PK id: bigserial FK→escenario escenario_id: bigint FK→metodo_solucion metodo_solucion_id: smallint FK→estado_ejecucion estado_ejecucion_id: smallint iniciada_en: timestamptz tiempo_ms: numeric(12,3) [0..1] costo_objetivo_cop: numeric(16,2) [0..1] iteraciones: integer [0..1] mensaje: text [0..1]
('b', 'resultado_ruta')	PK id: bigserial FK→ejecucion ejecucion_id: bigint FK→ruta ruta_id: bigint toneladas: numeric(12,2) costo_reducido: numeric(14,2) [0..1] costo_min_optimalidad: numeric(14,2) [0..1] costo_max_optimalidad: numeric(14,2) [0..1] *UNIQUE (ejecucion_id, ruta_id)*
('b', 'sensibilidad_oferta')	PK id: bigserial FK→ejecucion ejecucion_id: bigint FK→escenario_empresa escenario_empresa_id: bigint precio_sombra: numeric(14,2) holgura: numeric(12,2) oferta_min: numeric(12,2) [0..1] oferta_max: numeric(12,2) [0..1] *UNIQUE (ejecucion_id, escenario_empresa_id)*
('b', 'sensibilidad_demanda')	PK id: bigserial FK→ejecucion ejecucion_id: bigint FK→escenario_planta escenario_planta_id: bigint precio_sombra: numeric(14,2) holgura: numeric(12,2) demanda_min: numeric(12,2) [0..1] demanda_max: numeric(12,2) [0..1] *UNIQUE (ejecucion_id, escenario_planta_id)*
('b', 'analisis_sensibilidad')	PK id: bigserial FK→escenario escenario_base_id: bigint FK→tipo_parametro tipo_parametro_id: smallint FK→empresa empresa_id: integer [0..1] FK→planta planta_id: integer [0..1] nombre: varchar(100) variacion_min_pct: numeric(6,2) variacion_max_pct: numeric(6,2) paso_pct: numeric(6,2) creado_en: timestamptz
('b', 'punto_sensibilidad')	PK id: bigserial FK→analisis_sensibilidad analisis_id: bigint FK→escenario escenario_id: bigint variacion_pct: numeric(6,2) *UNIQUE (analisis_id, variacion_pct)*
('b', 'parametros_ag')	PK FK→ejecucion ejecucion_id: bigint FK→metodo_seleccion metodo_seleccion_id: smallint FK→tipo_cruce tipo_cruce_id: smallint tamano_poblacion: integer num_generaciones: integer tasa_cruce: numeric(4,3) tasa_mutacion: numeric(4,3) tamano_torneo: smallint [0..1] elitismo: smallint factor_penalizacion: numeric(16,2) semilla: integer [0..1]
('b', 'historial_generacion')	PK id: bigserial FK→parametros_ag ejecucion_id: bigint num_generacion: integer mejor_costo: numeric(16,2) costo_promedio: numeric(16,2) mejor_fitness: double violaciones: integer *UNIQUE (ejecucion_id, num_generacion)*
('b', 'comparacion')	PK id: bigserial FK→ejecucion ejecucion_exacta_id: bigint FK→ejecucion ejecucion_ag_id: bigint observaciones: text [0..1] creada_en: timestamptz *UNIQUE (ejecucion_exacta_id, ejecucion_ag_id)*

PK: clave primaria. FK→tabla: clave foránea. UQ: valor único. [0..1]: admite nulo; las demás columnas son obligatorias.

El script de la base de datos crea las 25 tablas con sus claves primarias, foráneas, restricciones de unicidad y de verificación (por ejemplo, oferta y distancias no negativas, tasas del AG entre 0 y 1), las dos vistas, los catálogos, el escenario del caso base con sus 10 empresas, 4 plantas y 40 rutas, y un usuario analista inicial. Es reejecutable y usa codificación UTF-8 (RNF13).

F. Diagramas de secuencia

Se elaboró un diagrama de secuencia por cada historia de usuario implementada, más uno transversal para el inicio de sesión; las historias más amplias se dividieron en dos diagramas. Las líneas de vida son únicamente clases u objetos, encabezadas por el actor analista, y los mensajes son exclusivamente métodos del código con sus parámetros tipados y su tipo de retorno; las funciones de módulo se indican con notas. Las Figs. 28 a 43 presentan los 16 diagramas.

1) Inicio de sesión (transversal)

El analista envía su usuario o correo y su contraseña; ServicioAutenticacion consulta el usuario mediante BaseDatos, verifica el bloqueo temporal y la contraseña con bcrypt, actualiza los intentos fallidos y, si las credenciales son válidas, construye un UsuarioSesion y emite un token firmado que conserva la sesión entre recargas de la página.

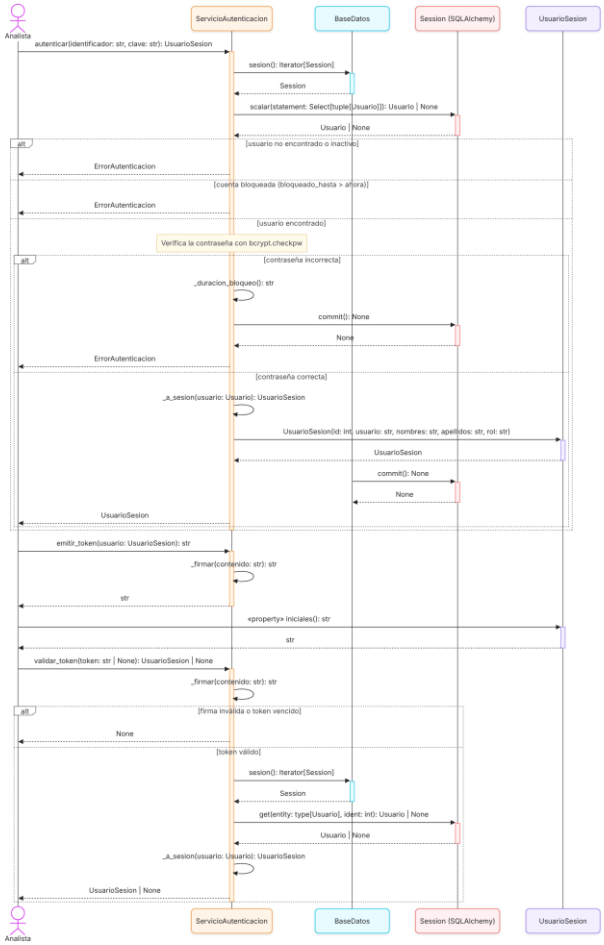


Fig. 28. Diagrama de secuencia: inicio de sesión (transversal).

2) HU-01: gestión de empresas agropecuarias

La vista carga el escenario con ServicioEscenarios y los catálogos de municipios y tipos de residuo con ServicioRed. Al crear o editar una empresa, ServicioRed limpia y valida los datos (nombre y NIT únicos), la registra en el escenario con su oferta y precio por tonelada, y crea automáticamente las rutas

hacia todas las plantas.

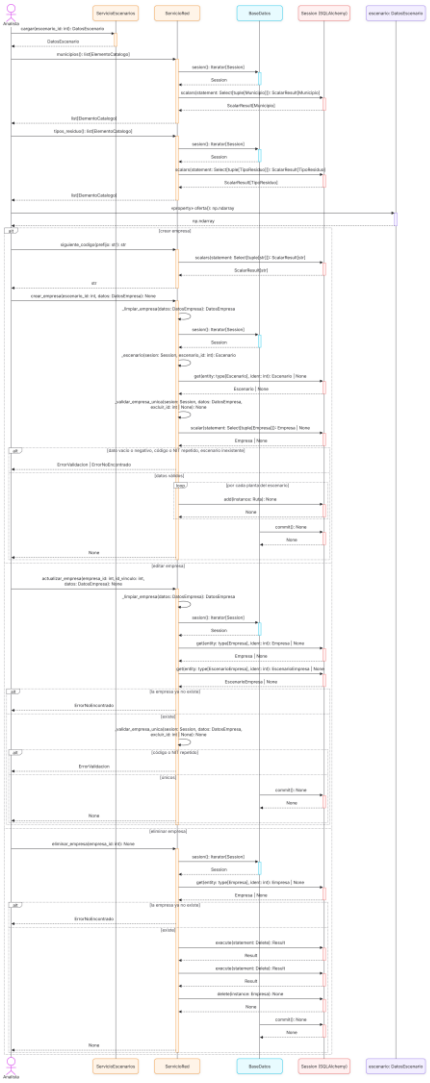


Fig. 29. Diagrama de secuencia: hU-01: gestión de empresas agropecuarias.

3) HU-01: gestión de plantas

Además de los datos del escenario, la vista consulta la última solución exacta para mostrar la utilización de cada planta. ServicioRed valida que la planta no exista, la crea con su demanda y costo de operación y genera las rutas desde todas las empresas; la edición y la baja siguen el mismo patrón con confirmación.

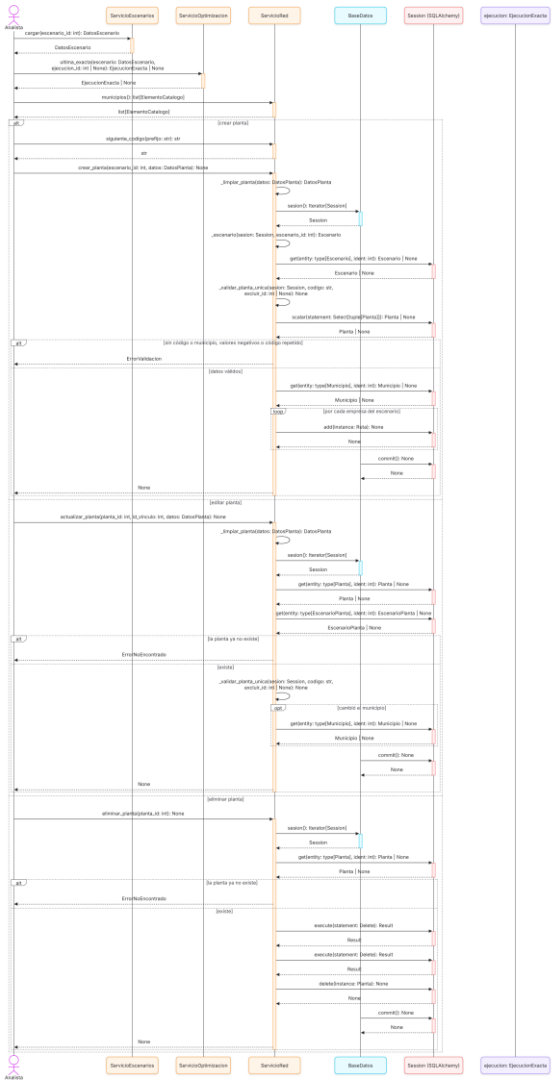


Fig. 30. Diagrama de secuencia: hU-01: gestión de plantas.

4) HU-02: costos, distancias y coeficientes

DatosEscenario expone como propiedades la matriz de costos unitarios, el costo de transporte efectivo (con su factor), los costos de operación y los precios de compra. Los cambios de α, de distancias o de parámetros se guardan con ServicioEscenarios en una transacción y la matriz se recalcula al recargar.

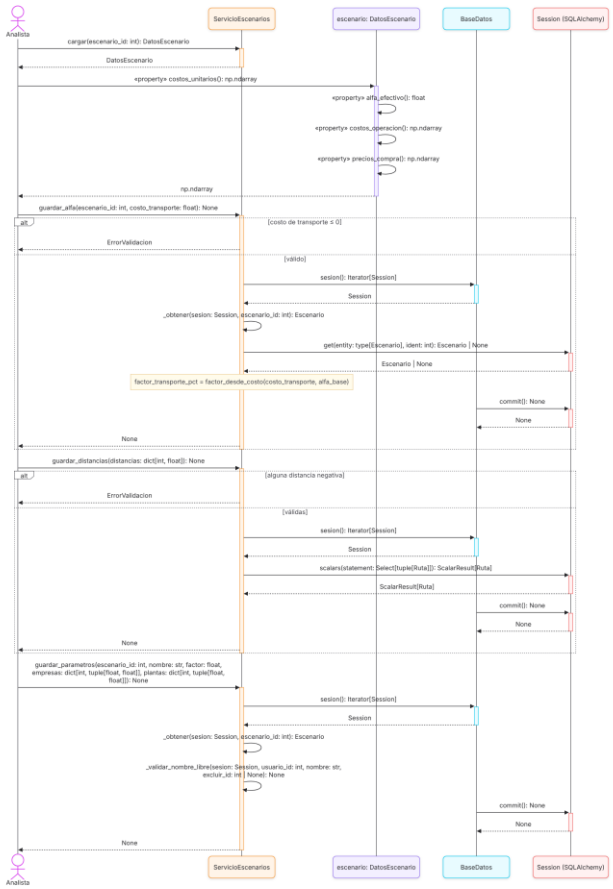


Fig. 31. Diagrama de secuencia: hU-02: costos, distancias y coeficientes.

5) HU-03: oferta y demanda

La vista lee la oferta y la demanda del escenario, muestra los totales y la diferencia, y advierte si el problema no está balanceado y qué nodo ficticio se agregará. Al guardar, ServicioEscenarios valida el nombre del escenario y registra los nuevos valores.

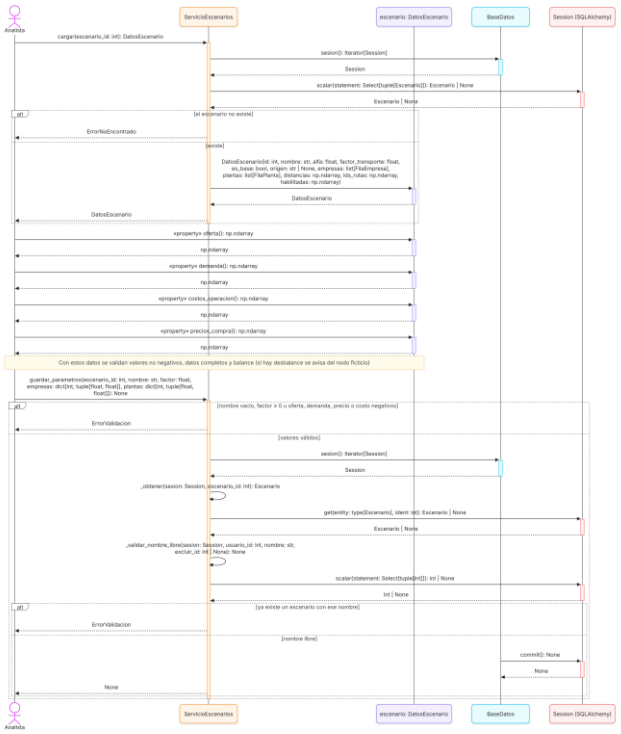


Fig. 32. Diagrama de secuencia: hU-03: oferta y demanda.

6) HU-04: guardar y recuperar escenarios

ServicioEscenarios lista los escenarios con su resumen (costos de la última ejecución exacta y del AG) y permite duplicar uno: copia el escenario, sus empresas, plantas y rutas con un nombre libre, de modo que los datos originales nunca se alteran (RNF10).

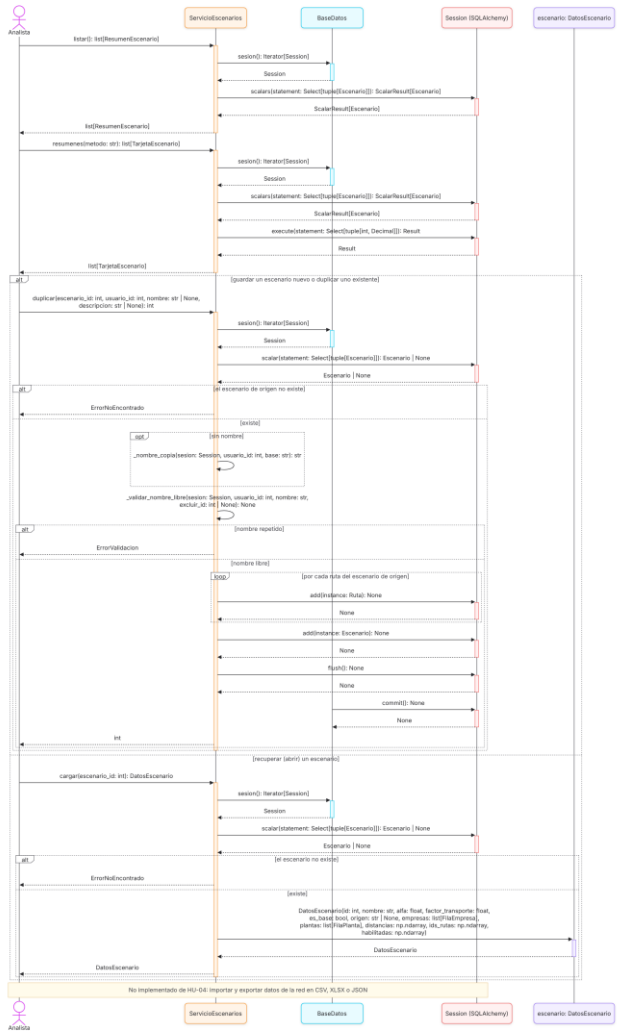


Fig. 33. Diagrama de secuencia: hU-04: guardar y recuperar escenarios.

7) *HU-05: visualización de la red logística*

La vista obtiene el escenario, la última solución exacta y el último lote del AG; con ellos crea los NodoRed y un grafo dirigido de NetworkX con un nodo por empresa y planta y un arco por ruta, y calcula el ResumenGrafo (nodos, arcos y arcos con flujo) que acompaña al dibujo interactivo.

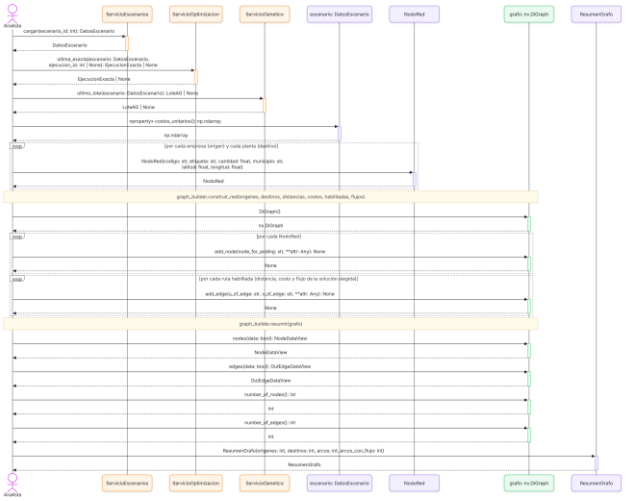


Fig. 34. Diagrama de secuencia: hU-05: visualización de la red logística.

8) *HU-06: resolución con el modelo exacto*

ServicioOptimizacion arma el ProblemaTransporte con los costos unitarios, resuelve con ModeloTransporte, verifica el balance y la factibilidad, ejecuta el análisis de sensibilidad y guarda en una sola transacción la ejecución, los flujos por ruta, los precios sombra y los rangos; la vista recibe una EjecucionExacta con el reporte de factibilidad.

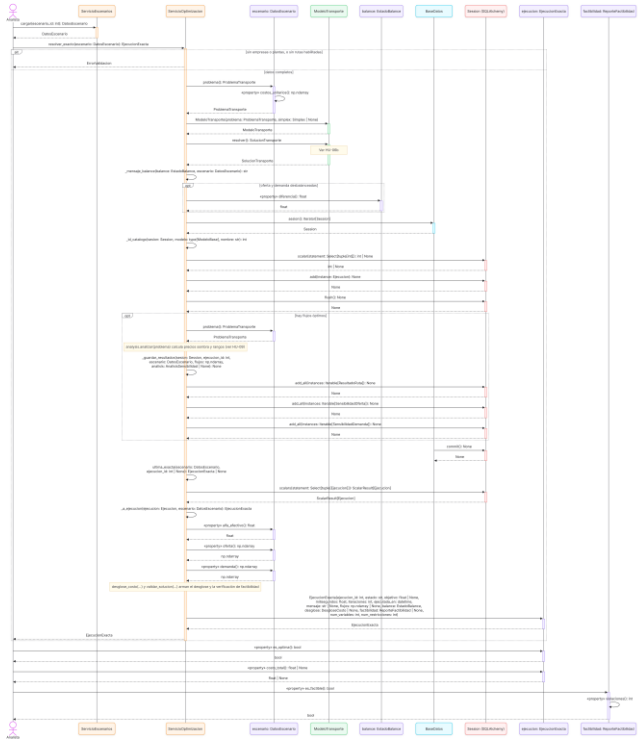


Fig. 35. Diagrama de secuencia: hU-06: resolución con el modelo exacto.

9) *HU-06: modelo de transporte y Simplex de dos fases*

Detalle interno de la resolución: ModeloTransporte balancea el problema, construye el programa lineal y mide el tiempo con Cronometro mientras Simplex convierte el sistema, fija la función objetivo de la fase 1, itera y pivotea, retira las artificiales y repite el ciclo en la fase 2.

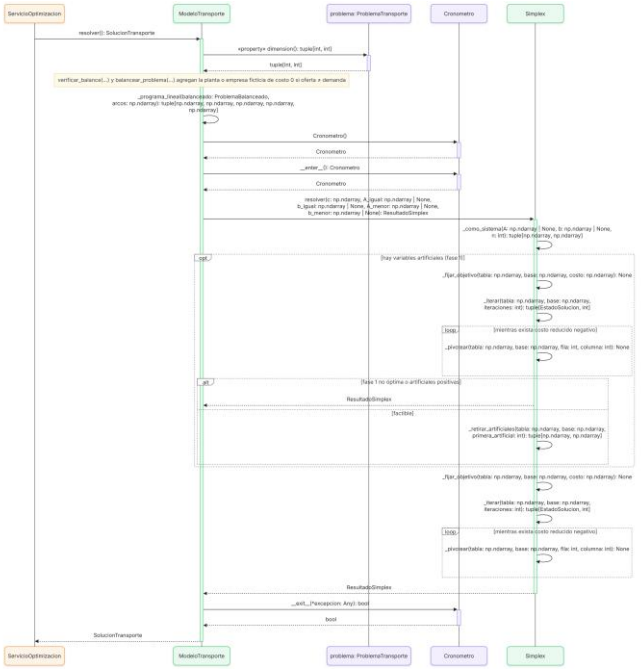


Fig. 36. Diagrama de secuencia: hU-06: modelo de transporte y Simplex de dos fases.

10) HU-07: configuración y ejecución del AG

La vista propone los últimos parámetros usados. ServicioGenetico valida ParametrosAG, crea el AlgoritmoGenetico sobre el problema del escenario y lo ejecuta una o N veces informando el avance; cada corrida se guarda con sus parámetros, su historial de generaciones y sus flujos, y el lote resultante incluye las estadísticas.

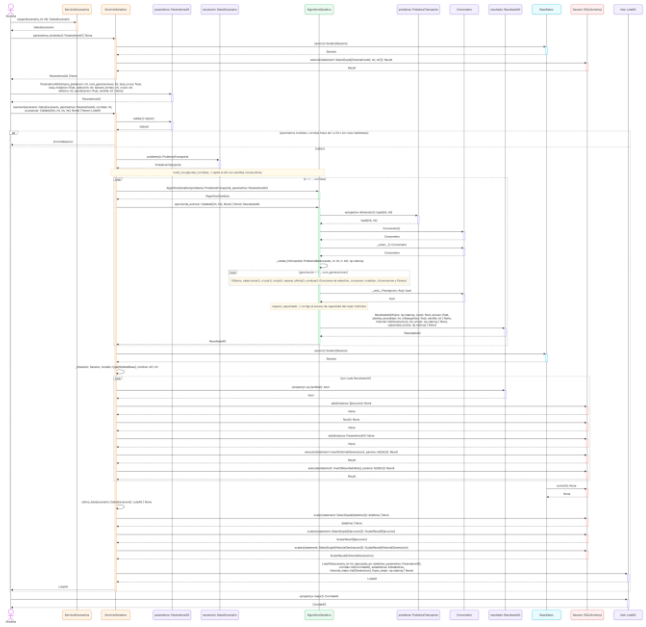


Fig. 37. Diagrama de secuencia: hU-07: configuración y ejecución del AG.

11) HU-08: visualización de resultados

La vista de inicio combina la última solución exacta, el último lote del AG y las ejecuciones recientes para mostrar el costo total, la utilización de cada planta, el desglose del costo (transporte, operación y compra), la verificación de factibilidad y el estado de balance.

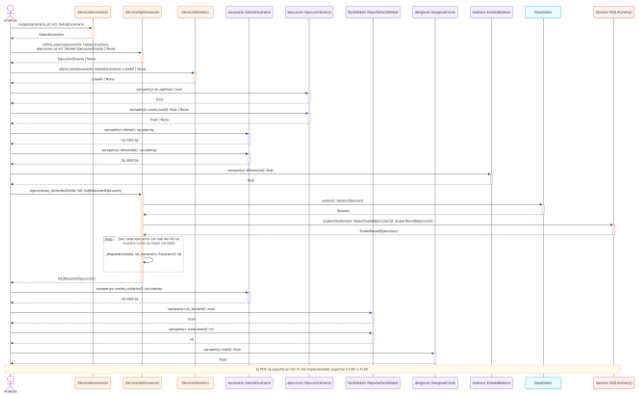


Fig. 38. Diagrama de secuencia: hU-08: visualización de resultados.

12) HU-09: análisis de sensibilidad

ServicioSensibilidad resuelve la formulación natural con Simplex, construye la BaseOptima con la inversa de la base, y obtiene para cada empresa y planta el precio sombra y su rango, y para cada ruta el costo reducido y su rango de optimalidad; el resultado indica si la solución es degenerada.

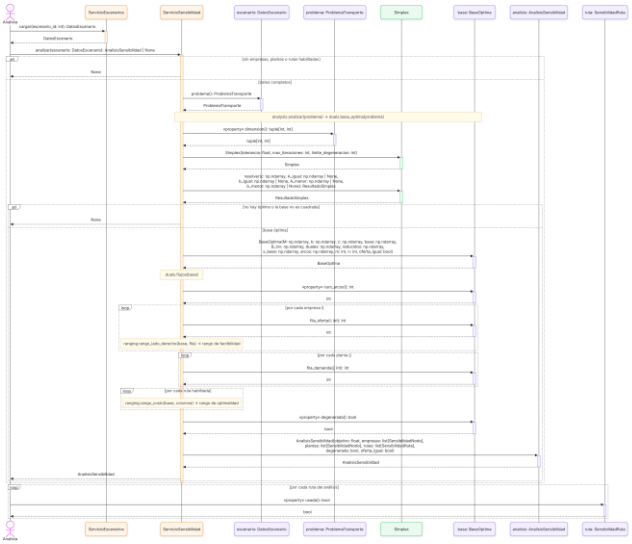


Fig. 39. Diagrama de secuencia: hU-09: análisis de sensibilidad.

13) HU-10: simulación de escenarios

Para cada valor de la variación elegida (costo de transporte, oferta, demanda o costo de operación) se resuelve el método exacto y, si el usuario lo pide, el AG con los últimos parámetros; cada punto de simulación registra ambos costos, tiempos, el gap y si cambió el plan de rutas.

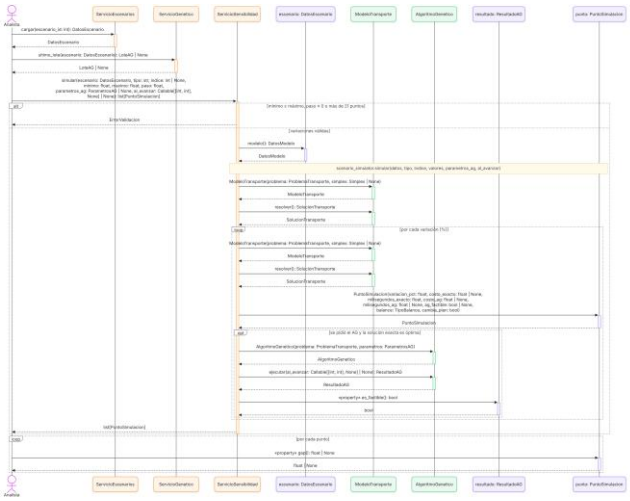


Fig. 40. Diagrama de secuencia: hU-10: simulación de escenarios.

14) *HU-10: guardar la simulación como escenarios*

Los puntos simulados se guardan como escenarios derivados: se duplica el escenario base, se aplica la variación y se resuelven ambos métodos; después ServicioComparacion los presenta lado a lado con sus costos y su gap.

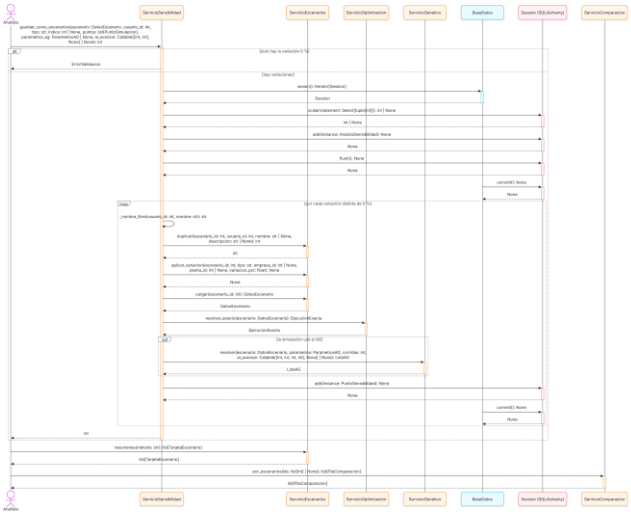


Fig. 41. Diagrama de secuencia: hU-10: guardar la simulación como escenarios.

15) *HU-11: comparación exacto frente a AG*

La vista toma la última ejecución exacta y el mejor resultado del último lote del AG de cada escenario, construye las FilaComparacion con el gap y, para la escalabilidad, resuelve instancias aleatorias con ambos métodos.

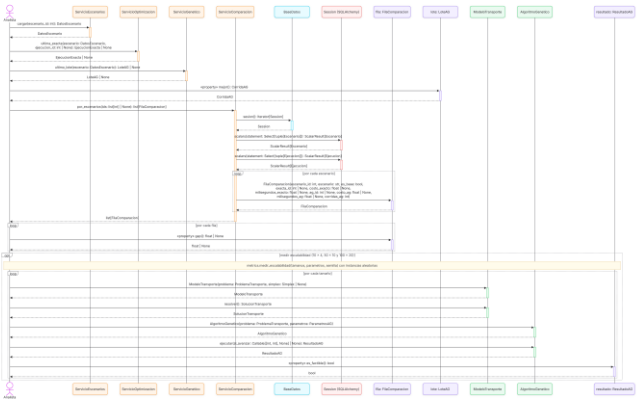


Fig. 42. Diagrama de secuencia: hU-11: comparación exacto frente a AG.

16) *HU-11: exportar el reporte comparativo en PDF*

El reporte se arma con secciones y tablas (resultados, sensibilidad y comparación) y se dibuja con el documento PDF con los colores del tema; ServicioComparacion registra la comparación en la base de datos.

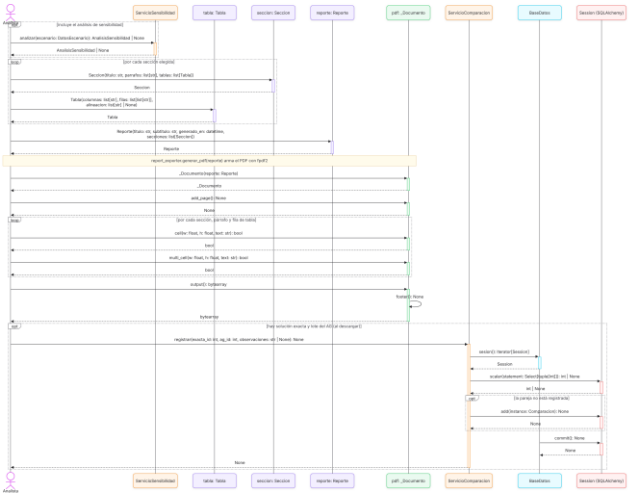


Fig. 43. Diagrama de secuencia: hU-11: exportar el reporte comparativo en PDF.

Las historias HU-04 y HU-08 se marcaron como parciales. En HU-04 están disponibles crear, duplicar, buscar, guardar y recuperar escenarios con sus parámetros y resultados, pero los datos regionales de prueba se precargan con el script de la base de datos y no se importan desde la interfaz. En HU-08 se muestran la tabla y la gráfica de flujos, el costo total y la utilización de cada planta, pero la exportación a PDF se hace desde la vista de comparación (HU-11) y no desde la vista de resultados.

G. *Interfaz de usuario*

La interfaz se diseñó primero como prototipos de baja fidelidad (wireframes) y después se implementó en Streamlit con un menú lateral agrupado en datos de la red, optimización y análisis, un selector de escenario activo y un tema claro u oscuro. Todos los colores de la interfaz, de las gráficas, del grafo y del reporte PDF se leen de un único archivo de configuración del tema. Las etiquetas muestran siempre las unidades (t/semana, COP/t, km) y la interfaz es legible desde una resolución de 1366 × 768 (RNF03, RNF11). Las Figs. 44 a

55 presentan las 12 pantallas.

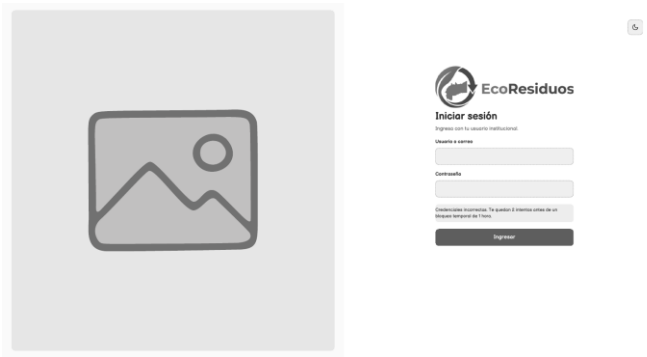


Fig. 44. *Pantalla inicio de sesión.* Formulario de usuario y contraseña con el mensaje de intentos restantes antes del bloqueo temporal de una hora.

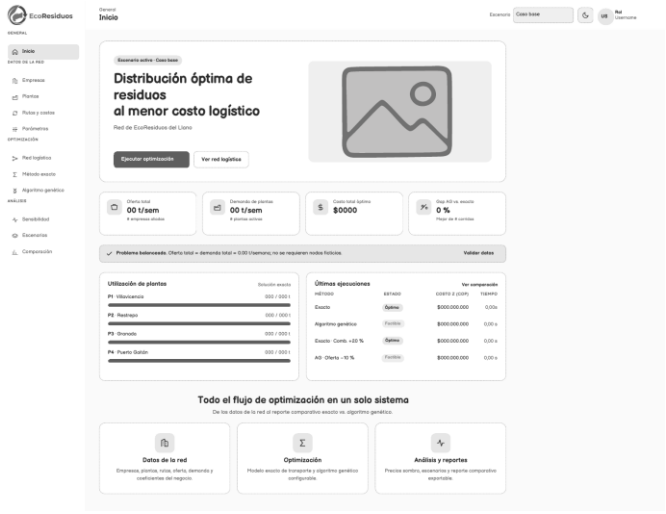


Fig. 45. *Pantalla inicio.* Panel principal con el escenario activo, los indicadores de oferta, demanda, costo óptimo y gap, el estado de balance, la utilización de las plantas y las últimas ejecuciones.

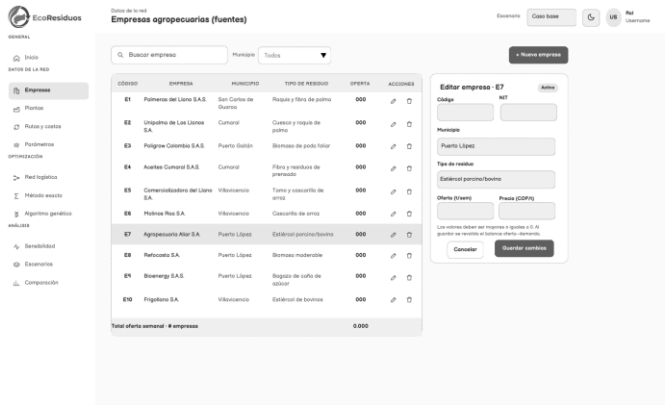


Fig. 46. *Pantalla empresas.* Tabla de empresas con búsqueda y filtro por municipio, y panel lateral para crear o editar una empresa con su oferta y precio por tonelada.

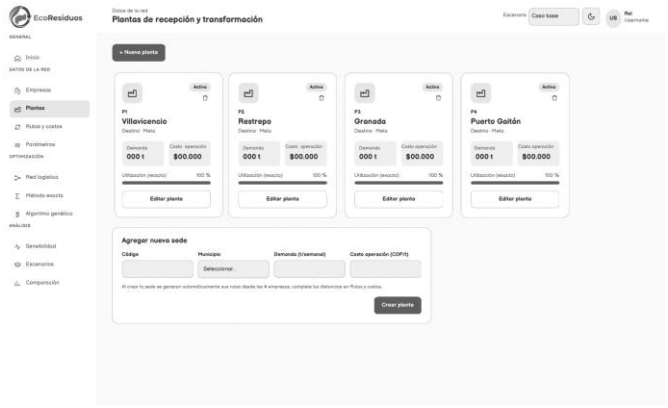


Fig. 47. *Pantalla plantas.* Tarjetas por planta con su demanda, costo de operación y utilización según la solución exacta, y formulario para agregar una nueva sede.

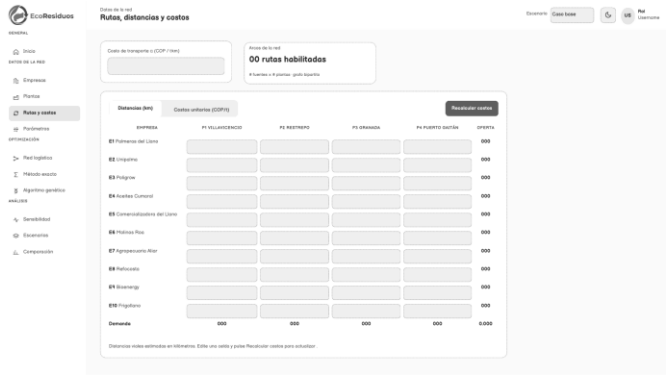


Fig. 48. *Pantalla rutas y costos.* Matriz editable de distancias y de costos unitarios por empresa y planta, con el costo de transporte por tonelada-kilómetro.

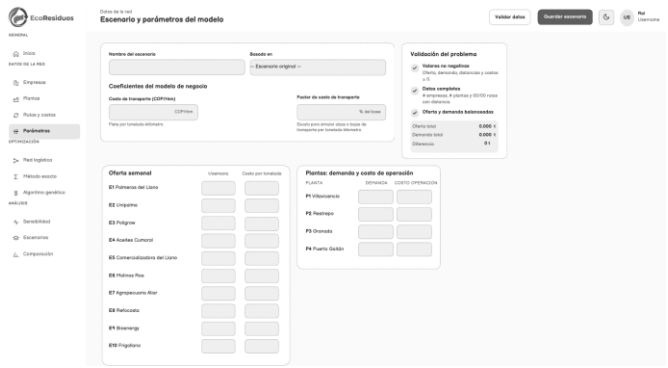


Fig. 49. *Pantalla parámetros.* Datos del escenario, coeficientes del modelo de negocio, oferta y demanda por nodo y la validación del problema (valores no negativos, datos completos y balance).



Fig. 50. *Pantalla red logística.* Grafo interactivo G = (V, A) con las rutas disponibles y los flujos óptimos, el resumen del grafo y el detalle del arco seleccionado.

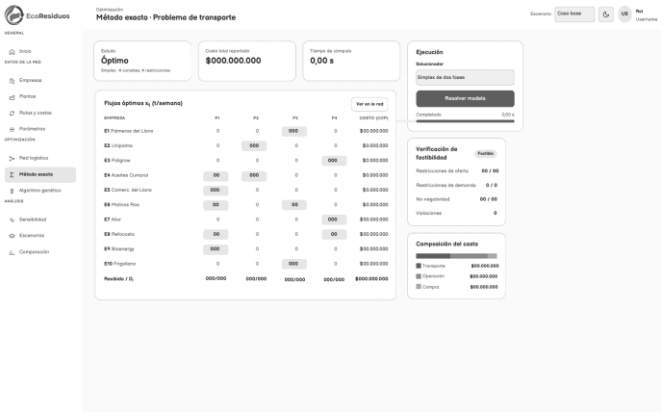


Fig. 51. *Pantalla método exacto.* Estado de la solución, costo total, tiempo, tabla de flujos óptimos, verificación de factibilidad y composición del costo.

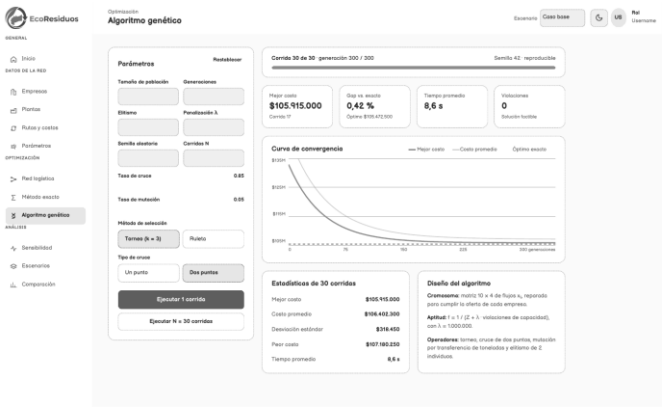


Fig. 52. *Pantalla algoritmo genético.* Parámetros configurables, progreso de la corrida, indicadores, curva de convergencia, estadísticas de las N corridas y resumen del diseño del algoritmo.

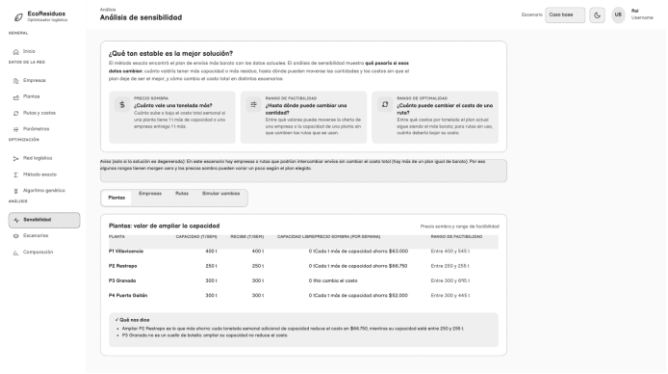


Fig. 53. *Pantalla sensibilidad.* Explicación de precios sombra, rangos de factibilidad y de optimalidad, aviso de degeneración y tablas por planta, empresa y ruta, con la pestaña de simulación.

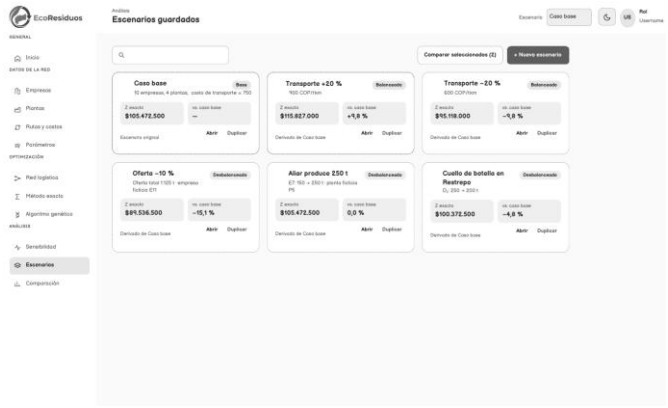


Fig. 54. *Pantalla escenarios.* Tarjetas de los escenarios guardados con su costo exacto y la variación frente al caso base, con opciones para abrir, duplicar y comparar.

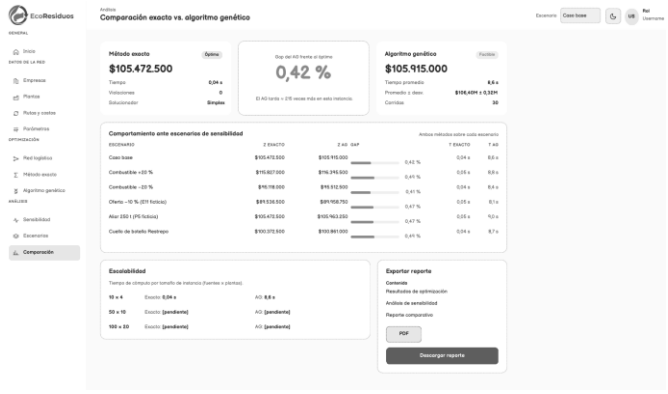


Fig. 55. *Pantalla comparación.* Costos de ambos métodos, gap, comportamiento por escenario de sensibilidad, escalabilidad y exportación del reporte en PDF.

H. Seguridad y manejo de errores

El acceso exige inicio de sesión con usuario o correo y contraseña. Las contraseñas se almacenan cifradas con bcrypt [31]; después de tres intentos fallidos la cuenta se bloquea durante una hora. La sesión se conserva entre recargas mediante un token firmado con HMAC-SHA256 [32] guardado en una cookie, y la clave de firma y las credenciales de la base de datos se leen de variables de entorno que no se suben al repositorio. Todas las operaciones de escritura se hacen en transacciones; las entradas inválidas (valores negativos, campos vacíos, nombres duplicados) se rechazan con mensajes claros mediante

una jerarquía de excepciones del dominio (validación, no encontrado y autenticación), y un problema infactible o no acotado se informa como estado de la ejecución en lugar de cerrar la aplicación (RNF09).

VIII. ANÁLISIS DE SENSIBILIDAD

A. *Fundamento*

El análisis de sensibilidad estudia cómo cambia la solución óptima cuando cambian los datos, sin volver a resolver [1], [4]. Se calcula sobre la formulación natural de la Fase 1, sin nodos ficticios: con oferta igual o menor que la demanda total la oferta es de igualdad y la demanda de desigualdad, y con oferta mayor ocurre lo contrario. En forma estándar, con la matriz M de restricciones con sus holguras y la base óptima B que entrega el Simplex, los precios sombra y los costos reducidos son

$$y = c_B B^{-1}, \quad d = c - yM \tag{20}$$

Para el problema de transporte los precios sombra se separan en u_i para la oferta de cada empresa y v_j para la demanda de cada planta, y el costo reducido de una ruta es

$$d_{ij} = c_{ij} - u_i - v_j \tag{21}$$

que vale cero en las rutas básicas y es no negativo en el óptimo. El precio sombra u_i es el cambio del costo total por cada tonelada adicional de oferta de la empresa i, y v_j es el cambio del costo por cada tonelada adicional de demanda de la planta j; como la demanda es una desigualdad de tipo $\leq$, $v_j \leq 0$ y su valor absoluto es el ahorro que produce ampliar la capacidad de la planta. Los rangos de factibilidad indican hasta dónde puede moverse el lado derecho b_k sin que cambie la base, es decir, mientras $x_B + \Delta B^{-1} e_k \geq 0$; dentro de ellos el precio sombra es válido. Los rangos de optimalidad indican entre qué valores puede moverse el costo de una ruta sin que cambie la solución óptima: una ruta no básica puede subir sin límite y bajar hasta su costo reducido, y para una ruta básica en la fila r el cambio δ debe mantener $d_k - \delta \alpha_{rk} \geq 0$ en todas las columnas no básicas.

B. *Precios sombra y rangos de factibilidad*

TABLA LXX
PRECIOS SOMBRA Y RANGOS DE LA OFERTA DE LAS EMPRESAS

Empresa	Oferta (t)	Precio sombra (COP/t)	Rango sin cambio de base (t)
E1	150	126.000	0 a 150
E2	120	138.000	110 a 120
E3	110	105.750	60 a 110
E4	140	138.000	130 a 140
E5	100	120.750	50 a 100
E6	100	120.750	0 a 100
E7	150	180.750	100 a 150
E8	120	180.750	70 a 120
E9	120	180.750	70 a 120
E10	140	120.750	90 a 140

TABLA LXXI
PRECIOS SOMBRA Y RANGOS DE LA DEMANDA DE LAS PLANTAS

Planta	Demanda (t)	Recibido (t)	Precio sombra (COP/t)	Rango (t)
P1 Villavicencio	400	400	-63.000	400 a 450
P2 Restrepo	250	250	-66.750	250 a 260
P3 Granada	300	300	0	300 a ∞
P4 Puerto Gaitán	300	300	-52.000	300 a 350

Las Tablas LXX y LXXI y la Fig. 56 presentan los resultados del caso base. En las plantas, ampliar la capacidad de P2 Restrepo es lo más valioso: cada tonelada adicional reduce el costo en 66.750 COP, hasta 260 t; le siguen P1 Villavicencio (63.000 COP/t hasta 450 t) y P4 Puerto Gaitán (52.000 COP/t hasta 350 t). P3 Granada tiene precio sombra cero: aunque recibe exactamente su demanda, ampliarla no reduce el costo, porque Granada es la planta más cara de abastecer y no hay rutas que se abaraten al enviarle más residuo; desde el punto de vista económico no es un cuello de botella. Las plantas P1, P2 y P4 sí lo son, y su ampliación es la recomendación principal para la gerencia.

En las empresas, el precio sombra es el costo marginal de recolectar una tonelada más de cada fuente. Las de Puerto López (E7, E8 y E9) tienen el mayor valor, 180.750 COP/t, y Poligrow (E3) el menor, 105.750 COP/t, porque está junto a la planta de Puerto Gaitán. Como el problema está balanceado y las plantas están llenas, los rangos de oferta solo permiten disminuir la oferta (el máximo coincide con el valor actual): reducir una tonelada en la empresa i ahorra exactamente su precio sombra, y aumentar la oferta exige ampliar alguna planta o dejar residuo sin recolectar, lo que cambia la base.

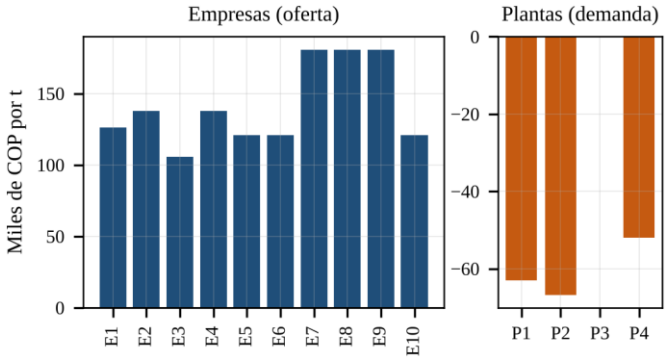


Fig. 56. Precios sombra de la oferta de las empresas y de la demanda de las plantas en el caso base.

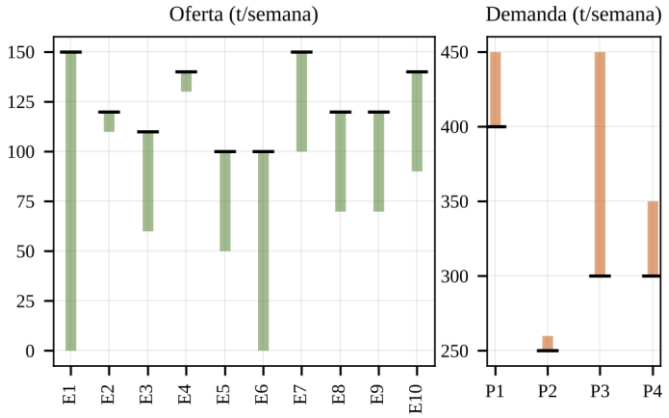


Fig. 57. Rangos de factibilidad de la oferta y de la demanda; la marca negra es el valor actual y la flecha indica un límite superior infinito.

C. *Costos reducidos y rangos de optimalidad*

TABLA LXXII
COSTOS REDUCIDOS Y RANGOS DE OPTIMALIDAD DE LAS 40 RUTAS (COP/T)

Ruta	Flujo (t)	Costo c	Costo reducido	Mínimo	Máximo
E1→P1	–	117.750	54.750	63.000	∞
E1→P2	–	136.500	77.250	59.250	∞
E1→P3	150	126.000	0	−∞	180.750
E1→P4	–	256.250	182.250	74.000	∞
E2→P1	–	75.000	0	75.000	∞
E2→P2	120	71.250	0	−∞	71.250
E2→P3	–	141.750	3.750	138.000	∞
E2→P4	–	213.500	127.500	86.000	∞
E3→P1	–	196.500	153.750	42.750	∞
E3→P2	–	215.250	176.250	39.000	∞
E3→P3	–	263.250	157.500	105.750	∞
E3→P4	110	53.750	0	−∞	207.500
E4→P1	10	75.000	0	52.500	75.000
E4→P2	130	71.250	0	71.250	93.750
E4→P3	–	141.750	3.750	138.000	∞
E4→P4	–	213.500	127.500	86.000	∞
E5→P1	50	57.750	0	57.750	57.750
E5→P2	–	76.500	22.500	54.000	∞
E5→P3	50	120.750	0	120.750	120.750
E5→P4	–	192.500	123.750	68.750	∞
E6→P1	–	57.750	0	57.750	∞
E6→P2	–	76.500	22.500	54.000	∞
E6→P3	100	120.750	0	−∞	120.750
E6→P4	–	192.500	123.750	68.750	∞
E7→P1	–	117.750	0	117.750	∞
E7→P2	–	136.500	22.500	114.000	∞
E7→P3	–	184.500	3.750	180.750	∞
E7→P4	150	128.750	0	−∞	128.750
E8→P1	80	117.750	0	117.750	117.750
E8→P2	–	136.500	22.500	114.000	∞
E8→P3	–	184.500	3.750	180.750	∞
E8→P4	40	128.750	0	128.750	128.750
E9→P1	120	117.750	0	−∞	117.750
E9→P2	–	136.500	22.500	114.000	∞
E9→P3	–	184.500	3.750	180.750	∞
E9→P4	–	128.750	0	128.750	∞
E10→P1	140	57.750	0	−∞	57.750
E10→P2	–	76.500	22.500	54.000	∞
E10→P3	–	120.750	0	120.750	∞
E10→P4	–	192.500	123.750	68.750	∞

El rango indica entre qué valores puede moverse el costo unitario de la ruta sin que cambie la solución óptima.

La Tabla LXXII y la Fig. 58 muestran los resultados de las rutas. Las rutas no usadas con costo reducido alto, como E1→P4 (182.250 COP/t) o E3→P2 (176.250 COP/t), tendrían que abaratarse mucho para entrar en la solución. En cambio, las rutas no usadas con costo reducido cero (E2→P1, E6→P1, E7→P1, E9→P4, E10→P3) pueden entrar en la base sin cambiar el costo total: el problema tiene **óptimos alternativos**. Esto explica que el AG encuentre un plan distinto del Simplex con el mismo costo (Sección IX). Además, la solución es degenerada, y por eso algunas rutas usadas tienen un rango de optimalidad de ancho cero (E5→P1, E5→P3, E8→P1, E8→P4): cualquier cambio de su costo cambia la base, aunque el plan óptimo pueda seguir siendo uno de los alternativos. La aplicación advierte esta situación al usuario, porque en una solución degenerada los precios sombra no son únicos y deben interpretarse junto con sus rangos.

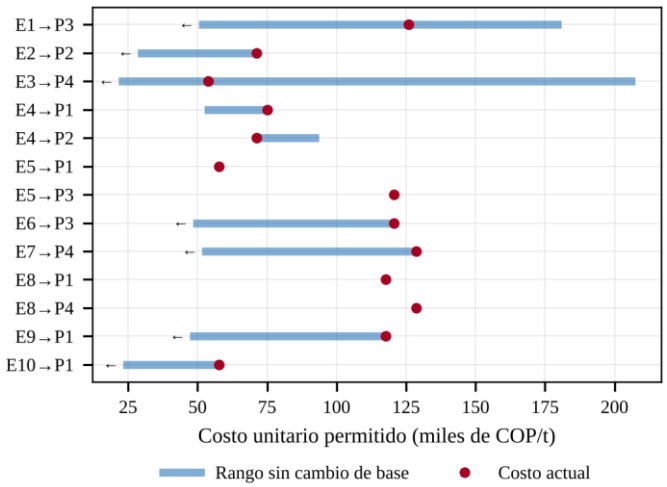


Fig. 58. Rangos de optimalidad del costo unitario de las rutas usadas; el punto es el costo actual y la flecha indica un límite inferior infinito.

D. *Simulación de escenarios*

El módulo de simulación varía un parámetro en porcentaje, resuelve el método exacto y el AG en cada punto y marca dónde cambia el conjunto de rutas usadas, sin modificar los datos originales (RF16, RNF10). Se simularon los cuatro tipos de escenario que pide el enunciado: cambios del costo de combustible y transporte, variaciones de la oferta de residuos, cuellos de botella en la demanda de una planta y cambios del costo de operación.

TABLA LXXIII
VARIACIÓN DEL COSTO DE TRANSPORTE A

Variación	Z exacto (COP)	Z AG (COP)	Gap	Balance	Cambia plan
-30 %	104.940.750	104.940.750	0,00 %	Balanceado	No
-20 %	110.118.000	110.118.000	0,00 %	Balanceado	No
-10 %	115.295.250	115.295.250	0,00 %	Balanceado	No
0 %	120.472.500	120.472.500	0,00 %	Balanceado	No
10 %	125.649.750	125.649.750	0,00 %	Balanceado	Sí
20 %	130.827.000	130.827.000	0,00 %	Balanceado	No
30 %	136.004.250	136.004.250	0,00 %	Balanceado	No

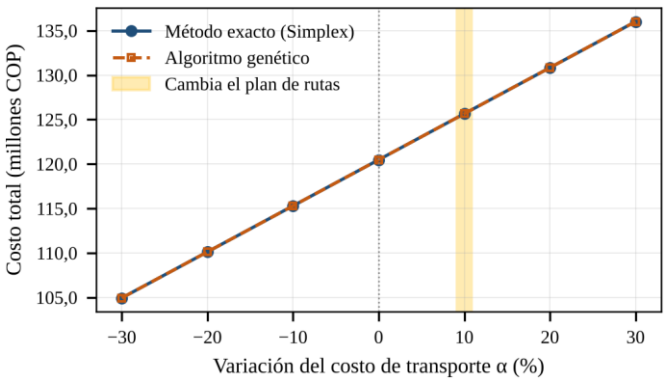


Fig. 59. Variación del costo de transporte α: costo total del método exacto y del AG; en amarillo, los puntos donde cambia el plan de rutas.

Costo de transporte. El costo total cambia linealmente con α: cada 10 % de aumento suma 5.177.250 COP por semana, que es el 10 % del costo de transporte del plan óptimo. Entre −30 % y +30 % el plan se mantiene salvo en +10 %, donde, por la degeneración, el Simplex llega a otro conjunto de rutas con un costo que sigue la misma recta; el gap del AG es 0 % en todos

los puntos (Tabla LXXIII, Fig. 59).

TABLA LXXIV
VARIACIÓN DE LA OFERTA TOTAL

Variación	Z exacto (COP)	Z AG (COP)	Gap	Balance	Cambia plan
-20 %	89.140.000	89.140.000	0,00 %	Dem. > of.	Sí
-15 %	95.094.750	95.094.750	0,00 %	Dem. > of.	Sí
-10 %	103.036.500	103.036.500	0,00 %	Dem. > of.	Sí
-5 %	111.642.000	111.642.000	0,00 %	Dem. > of.	Sí
0 %	120.472.500	120.472.500	0,00 %	Balanceado	No
5 %	118.073.625	118.073.625	0,00 %	Of. > dem.	Sí
10 %	115.674.750	115.674.750	0,00 %	Of. > dem.	Sí
15 %	113.275.875	113.275.875	0,00 %	Of. > dem.	Sí
20 %	110.877.000	110.877.000	0,00 %	Of. > dem.	Sí

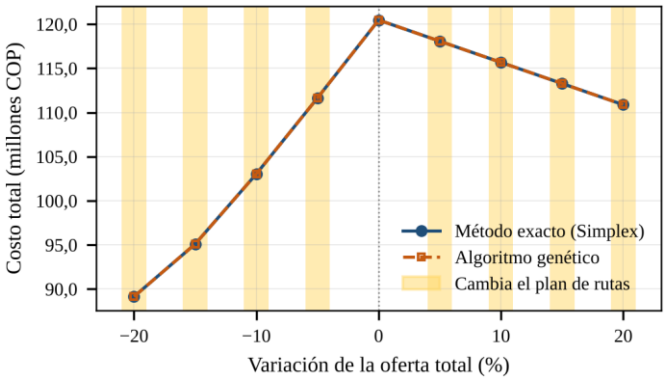


Fig. 60. Variación de la oferta total: costo total del método exacto y del AG; en amarillo, los puntos donde cambia el plan de rutas.

Oferta de residuos. Al disminuir la oferta, el costo baja porque se compra, transporta y procesa menos residuo: con −10 % el costo es 103.036.500 COP y aparece una empresa ficticia que representa la capacidad ociosa. Al aumentar la oferta el costo también baja, de 120.472.500 a 110.877.000 COP con +20 %, porque las plantas ya están llenas y el excedente permite dejar sin recolectar las toneladas más caras (las de Puerto López) y reemplazarlas por fuentes más baratas, sin pagar su precio de compra. Todos los puntos distintos del caso base cambian el plan, como se espera al cambiar el tipo de balance (Tabla LXXIV, Fig. 60).

TABLA LXXV
VARIACIÓN DE LA DEMANDA DE P1 VILLAVICENCIO

Variación	Z exacto (COP)	Z AG (COP)	Gap	Balance	Cambia plan
-40 %	101.632.500	101.632.500	0,00 %	Of. > dem.	Sí
-30 %	106.342.500	106.342.500	0,00 %	Of. > dem.	Sí
-20 %	111.052.500	111.052.500	0,00 %	Of. > dem.	Sí
-10 %	115.762.500	115.762.500	0,00 %	Of. > dem.	Sí
0 %	120.472.500	120.472.500	0,00 %	Balanceado	No
10 %	117.952.500	117.952.500	0,00 %	Dem. > of.	No
20 %	115.432.500	115.432.500	0,00 %	Dem. > of.	Sí

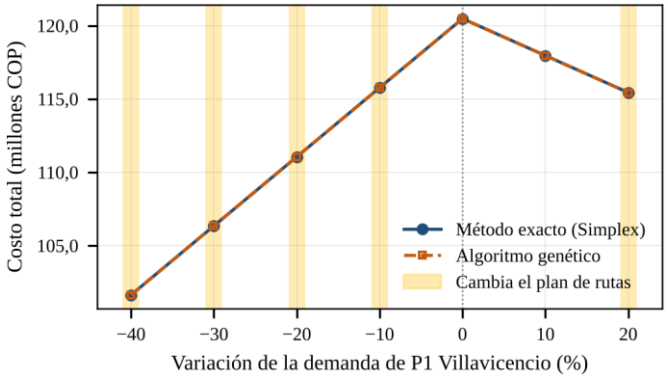


Fig. 61. Variación de la demanda de P1 Villavicencio: costo total del método exacto y del AG; en amarillo, los puntos donde cambia el plan de rutas.

Cuello de botella en P1. Reducir la demanda de Villavicencio en un 10 % (40 t) baja el costo en 117.750 COP por tonelada, que es el costo de las rutas desde Puerto López hacia P1: el residuo que deja de procesarse es el más caro. Ampliar P1 en un 10 % ahorra 63.000 COP por tonelada, exactamente el valor absoluto de su precio sombra (Tabla LXXI), lo que valida el análisis dual con una simulación independiente. Con +10 % (440 t) la base no cambia porque está dentro del rango de 400 a 450 t; con +20 % (480 t) se sale del rango y el plan cambia (Tabla LXXV, Fig. 61).

TABLA LXXVI
VARIACIÓN DEL COSTO DE OPERACIÓN DE P4 PUERTO GAITÁN

Variación	Z exacto (COP)	Z AG (COP)	Gap	Balance	Cambia plan
-30 %	117.052.500	117.052.500	0,00 %	Balanceado	No
-20 %	118.192.500	118.192.500	0,00 %	Balanceado	No
-10 %	119.332.500	119.332.500	0,00 %	Balanceado	No
0 %	120.472.500	120.472.500	0,00 %	Balanceado	No
10 %	121.612.500	121.612.500	0,00 %	Balanceado	No
20 %	122.752.500	122.752.500	0,00 %	Balanceado	No
30 %	123.892.500	123.892.500	0,00 %	Balanceado	No
40 %	125.032.500	125.032.500	0,00 %	Balanceado	No

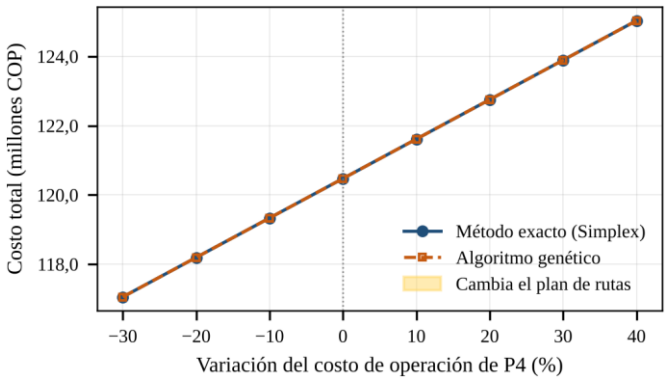


Fig. 62. Variación del costo de operación de P4 Puerto Gaitán: costo total del método exacto y del AG; en amarillo, los puntos donde cambia el plan de rutas.

Costo de operación de P4. El costo total crece 1.140.000 COP por cada 10 % de aumento del costo de operación de Puerto Gaitán (3.800 COP/t sobre 300 t) y el plan no cambia en ningún punto entre −30 % y +40 %: P4 sigue siendo la mejor planta para Poligrow y para las empresas de Puerto López, de modo que la solución es estable ante este parámetro (Tabla LXXVI, Fig. 62).

E. Casos no balanceados

TABLA LXXVII
RESULTADOS DE LOS CASOS NO BALANCEADOS

Caso	Oferta / demanda (t)	Z exacto (COP)	Z AG (COP)	Nodo ficticio en el exacto
Oferta de E7 = 250 t	1.350 / 1.250	120.472.500	120.472.500	Sin recolectar: E7 100 t
Oferta total -10 %	1.125 / 1.250	103.036.500	103.036.500	Ociosa: P3 125 t
Demanda de P1 = 300 t	1.250 / 1.150	108.697.500	108.697.500	Sin recolectar: E7 100 t

La Tabla LXXVII y la Fig. 63 muestran tres casos en los que el sistema agrega un nodo ficticio. Con la oferta de E7 en 250 t el costo no cambia, porque las 100 t adicionales quedan sin recolectar en la propia E7. Con la oferta total reducida un 10 % quedan 125 t de capacidad ociosa en Granada, la planta más cara de abastecer. Con la demanda de P1 reducida a 300 t quedan 100 t sin recolectar; el método exacto las deja en E7 y el AG en E8 y E9, empresas con costos idénticos, de modo que ambos planes cuestan lo mismo. En los tres casos el AG llega al costo del método exacto.

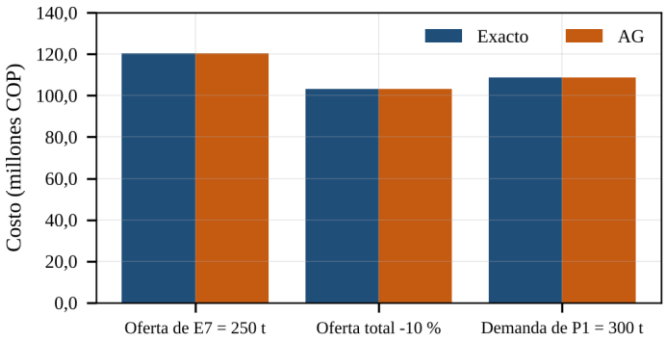


Fig. 63. Costo total del método exacto y del AG en los tres casos no balanceados.

IX. ANÁLISIS COMPARATIVO: MÉTODO EXACTO FRENTE A ALGORITMO GENÉTICO

A. Calidad de la función objetivo

Con los parámetros por defecto (Tabla LX) y semillas 42 a 71 se ejecutaron 30 corridas del AG sobre el caso base. La Tabla LXXVIII resume las estadísticas: las 30 corridas alcanzaron el óptimo del método exacto, por lo que el gap es 0 % y la desviación estándar es cero. El óptimo se alcanzó entre la generación 6 y la 27, con una media de 14,5 generaciones (Fig. 65).

TABLA LXXVIII
ESTADÍSTICAS DE 30 CORRIDAS DEL AG EN EL CASO BASE

Indicador	Valor
Mejor costo	120.472.500 COP
Costo promedio	120.472.500 COP
Desviación estándar	0 COP
Peor costo	120.472.500 COP
Corridas factibles	30 de 30
Óptimo del método exacto	120.472.500 COP
Gap del mejor costo	0,00 %
Generación del óptimo	6 a 27 (media 14,5)
Tiempo promedio por corrida	0,86 s

La Fig. 64 muestra la curva de convergencia de la corrida con semilla 42. Gracias a los individuos construidos con el método del costo mínimo, el mejor individuo de la primera generación ya está a 0,03 % del óptimo (120.510.000 COP); el intercambio en ciclo corrige esa diferencia y el óptimo se alcanza en la generación 15. El costo promedio de la población baja de 149,6 a cerca de 123 millones y se mantiene por encima del mejor: la mutación conserva diversidad mientras el elitismo protege la mejor solución.

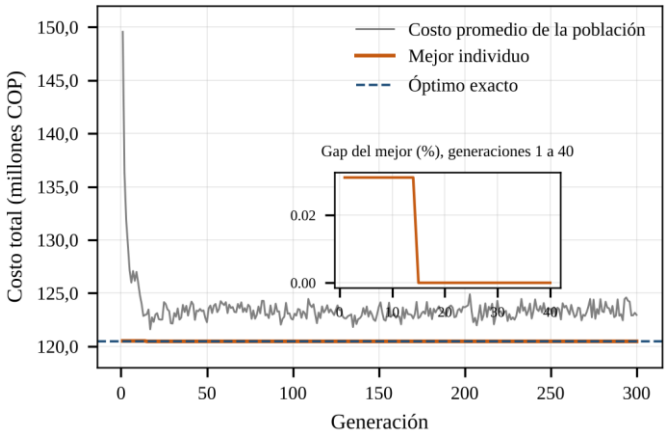


Fig. 64. Curva de convergencia del AG en el caso base (semilla 42): mejor individuo, costo promedio y óptimo exacto; el recuadro amplía el gap de las primeras 40 generaciones.

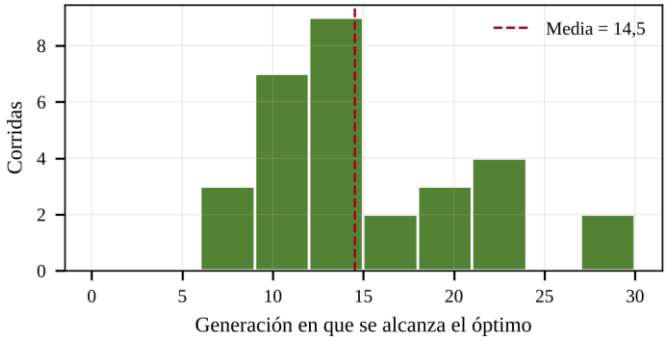


Fig. 65. Generación en la que cada una de las 30 corridas alcanzó el óptimo exacto.

TABLA LXXIX
MEJOR PLAN DEL AG EN EL CASO BASE (T/SEMANA)

Empresa	P1	P2	P3	P4
E1	–	–	150	–
E2	–	120	–	–
E3	–	–	–	110
E4	10	130	–	–
E5	90	–	10	–
E6	100	–	–	–
E7	150	–	–	–
E8	50	–	–	70
E9	–	–	–	120
E10	–	–	140	–
('b', 'Recibido')	('b', '400')	('b', '250')	('b', '300')	('b', '300')

Costo total 120.472.500 COP/semana, igual al óptimo del Simplex.

La Tabla LXXIX presenta el mejor plan del AG. Todas las filas suman su oferta, todas las plantas reciben exactamente su demanda y el costo es igual al óptimo, pero el plan es distinto del Simplex (Tabla LVI): por ejemplo, el AG envía E7 a P1 en lugar de P4 y E10 a P3 en lugar de P1. Es uno de los óptimos alternativos que anticipaban los costos reducidos nulos de la Tabla LXXII.

B. *Tiempo de cómputo*

En el caso base el Simplex tarda una mediana de 0,61 ms y el AG 0,86 s por corrida, es decir, el método exacto es unas 1.413 veces más rápido. Ambos cumplen holgadamente RNF01 (5 s y 60 s). La Fig. 66 muestra que el tiempo del AG es estable entre corridas, porque depende del número fijo de generaciones y no de los datos.

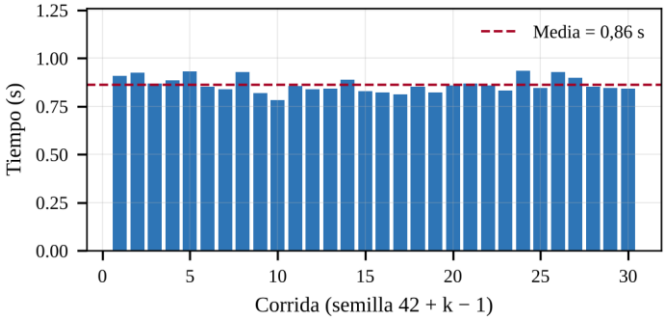


Fig. 66. Tiempo de cada una de las 30 corridas del AG en el caso base.

C. *Escalabilidad*

TABLA LXXX
ESCALABILIDAD DE AMBOS MÉTODOS EN INSTANCIAS ALEATORIAS BALANCEADAS

Instancia	Variables	Iter. Simplex	Exacto (ms)	AG (s)	Gap AG	Factible
10 × 4	40	38	0,7	0,80	0,00 %	Sí
20 × 6	120	107	2,4	1,49	0,13 %	Sí
30 × 8	240	197	7,3	2,43	0,71 %	Sí
50 × 10	500	399	28,3	3,87	3,96 %	Sí
75 × 15	1.125	785	229,0	6,33	5,22 %	Sí
100 × 20	2.000	1.308	1127,5	8,92	8,15 %	Sí

Instancias generadas con la semilla 2026 + k, ofertas entre 50 y 200 t y costos entre 50.000 y 270.000 COP/t. AG con parámetros por defecto.

Para medir el crecimiento del esfuerzo se resolvieron instancias balanceadas aleatorias desde 10 × 4 hasta 100 × 20 (2.000 variables), el tamaño mínimo que exige RNF02 (Tabla LXXX). El tiempo del Simplex crece de forma superlineal, de 0,7 ms a 1,13 s, porque el número de iteraciones y el tamaño de la tabla densa crecen a la vez; el del AG crece casi linealmente, de 0,80 s a 8,92 s (Fig. 67). En todos los tamaños el método exacto sigue siendo más rápido y siempre obtiene el óptimo, mientras que el gap del AG crece de 0 % a 8,15 % (Fig. 68): con 100 individuos y 300 generaciones el AG explora una fracción cada vez menor de un espacio mucho más grande. El AG fue factible en todas las instancias.

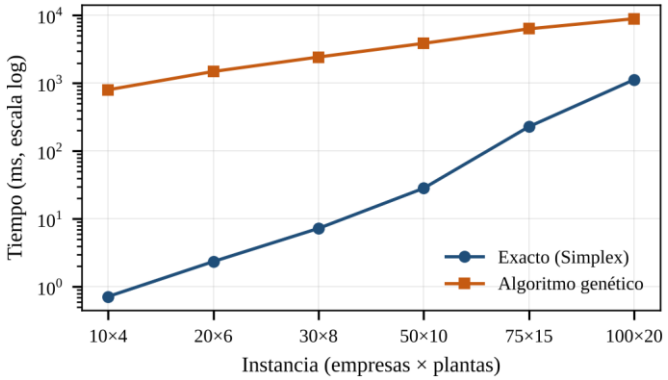


Fig. 67. Tiempo de cómputo de ambos métodos según el tamaño de la instancia (escala logarítmica).

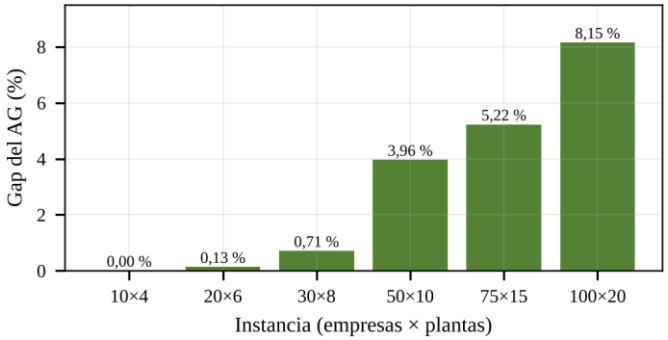


Fig. 68. Gap del AG frente al óptimo según el tamaño de la instancia.

D. *Ajuste de parámetros del AG*

TABLA LXXXI
EFECTO DE LOS PARÁMETROS DEL AG EN UNA INSTANCIA DE 50 × 10

Configuración	Gap medio	Rango del gap	Tiempo (s)	Factibles
Por defecto	1,97 %	1,55 % a 2,35 %	4,04	3/3
Población 50	2,00 %	1,38 % a 3,01 %	1,96	3/3
Población 200	2,33 %	1,30 % a 3,66 %	7,97	3/3
Generaciones 150	2,28 %	1,71 % a 2,71 %	1,98	3/3
Generaciones 600	1,74 %	1,24 % a 2,32 %	7,84	3/3
Cruce 0,60	2,37 %	1,21 % a 3,66 %	4,30	3/3
Mutación 0,01	1,27 %	0,95 % a 1,48 %	1,11	3/3
Mutación 0,20	4,28 %	1,96 % a 5,93 %	15,08	3/3
Ruleta	4,48 %	4,11 % a 4,94 %	3,91	3/3
Un punto	2,01 %	1,77 % a 2,22 %	3,96	3/3
Torneo k = 5	2,02 %	1,39 % a 2,42 %	4,09	3/3
Sin elitismo	5,52 %	4,94 % a 5,93 %	4,09	3/3

Instancia aleatoria con semilla 2027, óptimo 450.637.260 COP; tres corridas por configuración (semillas 1 a 3). Cada fila cambia un solo parámetro respecto a los valores por defecto.

Como el caso base es demasiado pequeño para diferenciar configuraciones, el efecto de los parámetros se estudió en una instancia de 50 × 10, cambiando un parámetro a la vez (Tabla LXXXI, Figs. 69 a 71). Con los valores por defecto el gap medio fue 1,97 %. Los hallazgos principales son: el elitismo es indispensable (sin él el gap sube a 5,52 % y la curva deja de ser monótona); el torneo supera a la ruleta (4,48 %), porque la presión selectiva de la ruleta se diluye cuando las aptitudes son muy parecidas; una mutación baja (0,01) dio el mejor gap, 1,27 %, y además el menor tiempo, mientras que una mutación alta (0,20) degrada la calidad (4,28 %) y multiplica el tiempo, porque cada movimiento recorre las empresas del hijo; duplicar las generaciones mejora el gap a 1,74 % a costa del doble de tiempo, y duplicar la población no lo mejoró (2,33 %). El tipo de cruce (uno o dos puntos) y el tamaño del torneo tuvieron

poco efecto. Los valores por defecto se mantienen como un equilibrio razonable entre calidad y tiempo; para redes grandes se recomienda reducir la tasa de mutación y aumentar las generaciones.

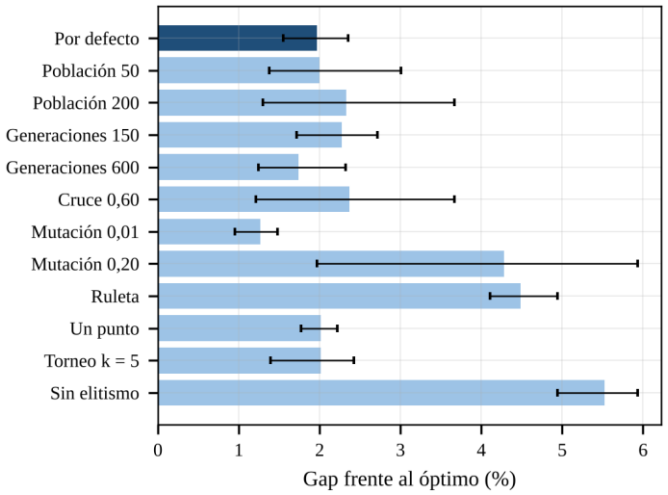


Fig. 69. Gap medio del AG por configuración en la instancia de 50 × 10; las barras de error indican el mínimo y el máximo de tres corridas.

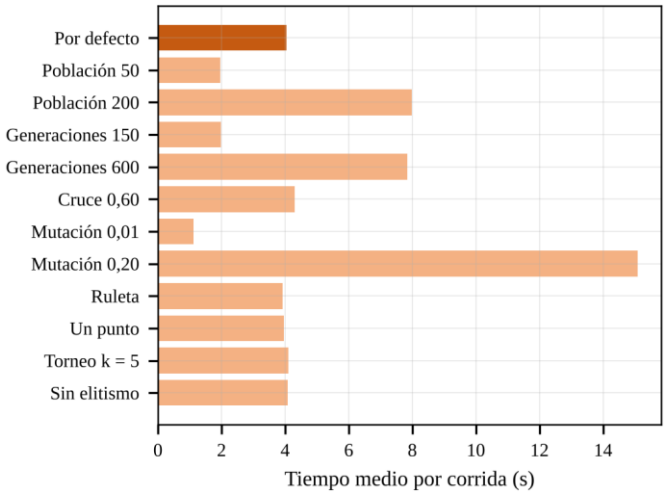


Fig. 70. Tiempo medio por corrida del AG para cada configuración en la instancia de 50 × 10.

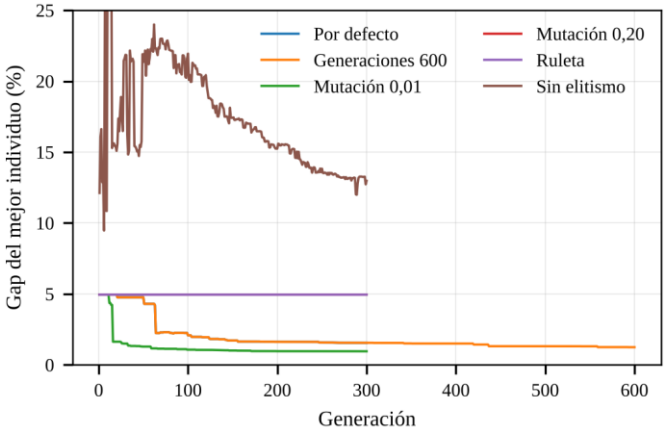


Fig. 71. Curvas de convergencia (gap del mejor individuo de cada generación) de seis configuraciones en la instancia de 50 × 10.

E. Desempeño ante los escenarios de sensibilidad

En los 31 puntos simulados de la Sección VIII el AG obtuvo el mismo costo que el método exacto (gap de 0 % en todos) y fue factible en todos, incluidos los puntos no balanceados con nodos ficticios. La Fig. 72 compara los tiempos medios por punto: el método exacto resuelve cada escenario en menos de 1 ms y el AG en menos de 1 s. Para la red regional ambos enfoques son confiables ante los cambios de parámetros; el método exacto es preferible para el análisis de sensibilidad porque además entrega los precios sombra y los rangos, que el AG no produce, mientras que el AG aporta valor como verificación independiente y como alternativa para extensiones no lineales del modelo (costos fijos o economías de escala) en las que el Simplex ya no aplica.

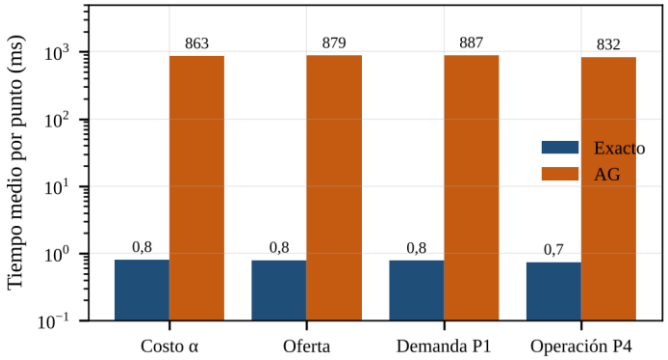


Fig. 72. Tiempo medio por punto de simulación de ambos métodos en los cuatro tipos de escenario (escala logarítmica).

TABLA LXXXII
RESUMEN COMPARATIVO DE LOS DOS ENFOQUES

Criterio	Método exacto (Simplex)	Algoritmo genético
Calidad en el caso base	Óptimo global garantizado	Óptimo en 30 de 30 corridas
Calidad en 100 × 20	Óptimo global	Gap de 8,15 %
Tiempo en el caso base	0,61 ms	0,86 s
Tiempo en 100 × 20	1,13 s	8,92 s
Crecimiento del tiempo	Superlineal	Casi lineal
Escenarios de sensibilidad	31 de 31 óptimos	31 de 31 con gap 0 %
Información dual	Precios sombra y rangos	No disponible
Reproducibilidad	Determinista	Determinista con semilla
Modelos no lineales	No aplica	Aplicable cambiando la aptitud

X. PRUEBAS Y VERIFICACIÓN DE REQUERIMIENTOS

La Tabla LXXXIII resume las pruebas realizadas y la Tabla LXXXIV el estado de cumplimiento de los requerimientos no funcionales con la evidencia correspondiente.

TABLA LXXXIII
PRUEBAS REALIZADAS

Prueba	Resultado
Factibilidad de la solución exacta del caso base	Oferta 10/10, demanda 4/4, no negatividad 40/40.
Verificación independiente con HiGHS (201 instancias)	Mismo costo óptimo en todas; diferencia máxima del orden de 10^{-16}.
Casos no balanceados (3)	Nodo ficticio correcto; mismo costo en Simplex, HiGHS y AG.
30 corridas del AG en el caso base	30 factibles, gap 0 %, desviación estándar 0.
Reproducibilidad del AG	Misma semilla: mismo costo, mismos flujos y misma curva; otra semilla: resultado distinto.
Escalabilidad hasta 100 × 20	Ambos métodos resuelven; AG factible en todas las instancias.
Simulación de escenarios (31 puntos)	Gap 0 % y factibilidad en todos los puntos.

Prueba	Resultado
Coherencia entre duales y simulación	Ampliar P1 en 40 t ahorra 63.000 COP/t, igual a su precio sombra.
Validación de entradas e inicio de sesión (revisión del código)	Rechazo de valores negativos y duplicados; bloqueo tras 3 intentos fallidos.

TABLA LXXXIV
CUMPLIMIENTO DE LOS REQUERIMIENTOS NO FUNCIONALES

RNF	Evidencia	Estado
RNF01	Exacto 0,61 ms (< 5 s); AG 0,86 s (< 60 s). Pruebas de rendimiento completadas (actividad 47).	Cumple
RNF02	Instancias de 100 × 20 resueltas solo con cambio de datos.	Cumple
RNF03	Etiquetas con unidades, valores por defecto y ayudas; pruebas de usabilidad y de manejo de errores completadas (actividad 49).	Cumple
RNF04	Arquitectura por capas con módulos exacto, genético, interfaz y sensibilidad.	Cumple
RNF05	Python, librerías abiertas, Simplex propio sin librerías de optimización, web y requirements.	Cumple
RNF06	Verificador con tolerancia de 10^{-6} y verificación con HiGHS.	Cumple
RNF07	Semilla configurable, parámetros guardados y prueba de reproducibilidad.	Cumple
RNF08	Repositorio Git, README con instalación y uso, código documentado y este informe.	Cumple
RNF09	Excepciones del dominio, transacciones y estados de solución.	Cumple
RNF10	Escenarios derivados; las simulaciones no alteran los datos originales.	Cumple
RNF11	Gráficas con títulos, ejes y unidades; tema claro u oscuro.	Cumple
RNF12	Ejecución local sin internet con el caso base precargado y escenarios de demostración (actividad 50).	Cumple
RNF13	Reporte en PDF y script SQL en UTF-8.	Cumple

XI. CONCLUSIONES

Se diseñó y desarrolló una solución computacional completa para la red logística de EcoResiduos del Llano. El modelo de transporte, que integra el costo de compra, de transporte y de operación, determinó que el costo mínimo semanal del caso base es de 120.472.500 COP, con un plan de 13 rutas que aprovecha la cercanía entre fuentes y plantas: Poligrow abastece a Puerto Gaitán, las empresas de Cumaral a Restrepo y las de Villavicencio y Puerto López a Villavicencio y Puerto Gaitán.

El Simplex de dos fases implementado desde cero obtuvo el óptimo en menos de un milisegundo, cumplió todas las restricciones y coincidió con el solucionador de referencia HiGHS en 201 instancias, lo que da confianza en su corrección. El algoritmo genético, con cromosoma matricial, aptitud penalizada, cruce por filas y mutación por transferencia o ciclo, alcanzó el óptimo en las 30 corridas del caso base y en todos los escenarios simulados, y su diseño garantiza la oferta por construcción.

El análisis de sensibilidad mostró que las plantas de Restrepo, Villavicencio y Puerto Gaitán son los cuellos de botella de la red, con ahorros de entre 52.000 y 66.750 COP por cada tonelada de capacidad adicional, mientras que ampliar Granada no reduce el costo. También mostró que la solución es degenerada y tiene óptimos alternativos, lo que exige interpretar los precios sombra junto con sus rangos. Las simulaciones confirmaron los resultados duales y la estabilidad del plan ante cambios del costo de operación.

La comparación permite concluir que, para el tamaño regional del problema, el método exacto es la mejor opción: es más rápido, garantiza el óptimo y entrega información dual. El AG es competitivo en calidad en redes pequeñas, pero su gap crece con el tamaño de la red (hasta 8,15 % en 100 × 20), por lo que su valor está en la verificación independiente y en futuras extensiones no lineales del modelo. Como trabajo futuro se propone incorporar costos fijos de apertura de rutas, capacidad de la flota y varios periodos.

REFERENCIAS

[1] F. S. Hillier y G. J. Lieberman, *Introducción a la investigación de operaciones*, 10.ª ed. Ciudad de México, México: McGraw-Hill, 2015.

[2] H. A. Taha, *Investigación de operaciones*, 10.ª ed. Ciudad de México, México: Pearson, 2017.

[3] W. L. Winston, *Investigación de operaciones: aplicaciones y algoritmos*, 4.ª ed. Ciudad de México, México: Thomson, 2005.

[4] M. S. Bazaraa, J. J. Jarvis y H. D. Sherali, *Linear Programming and Network Flows*, 4.ª ed. Hoboken, NJ, EE. UU.: Wiley, 2010.

[5] G. B. Dantzig, *Linear Programming and Extensions*. Princeton, NJ, EE. UU.: Princeton University Press, 1963.

[6] R. G. Bland, "New finite pivoting rules for the simplex method," *Mathematics of Operations Research*, vol. 2, no. 2, pp. 103–107, 1977.

[7] R. K. Ahuja, T. L. Magnanti y J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Englewood Cliffs, NJ, EE. UU.: Prentice Hall, 1993.

[8] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, EE. UU.: University of Michigan Press, 1975.

[9] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA, EE. UU.: Addison-Wesley, 1989.

[10] Z. Michalewicz, *Genetic Algorithms + Data Structures = Evolution Programs*, 3.ª ed. Berlín, Alemania: Springer, 1996.

[11] G. A. Vignaux y Z. Michalewicz, "A genetic algorithm for the linear transportation problem," *IEEE Trans. Syst., Man, Cybern.*, vol. 21, no. 2, pp. 445–452, 1991.

[12] K. Deb, "An efficient constraint handling method for genetic algorithms," *Comput. Methods Appl. Mech. Eng.*, vol. 186, no. 2–4, pp. 311–338, 2000.

[13] B. L. Miller y D. E. Goldberg, "Genetic algorithms, tournament selection, and the effects of noise," *Complex Systems*, vol. 9, no. 3, pp. 193–212, 1995.

[14] A. E. Eiben y J. E. Smith, *Introduction to Evolutionary Computing*, 2.ª ed. Berlín, Alemania: Springer, 2015.

[15] Q. Huangfu y J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.

[16] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[17] A. A. Hagberg, D. A. Schult y P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," en *Proc. 7th Python in Science Conf. (SciPy)*, Pasadena, CA, EE. UU., 2008, pp. 11–15.

[18] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[19] Streamlit Inc. (2026). *Streamlit documentation* [En línea]. Disponible: https://docs.streamlit.io

[20] M. Bayer. (2026). *SQLAlchemy 2.0 documentation* [En línea]. Disponible: https://docs.sqlalchemy.org

[21] The PostgreSQL Global Development Group. (2026). *PostgreSQL documentation* [En línea]. Disponible: https://www.postgresql.org/docs/

[22] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, EE. UU.: Addison-Wesley, 1994.

[23] R. S. Pressman, *Ingeniería del software: un enfoque práctico*, 7.ª ed. Ciudad de México, México: McGraw-Hill, 2010.

[24] I. Sommerville, *Ingeniería del software*, 7.ª ed. Madrid, España: Pearson Addison-Wesley, 2005.

[25] L. Bass, P. Clements y R. Kazman, *Software Architecture in Practice*, 3.ª ed. Boston, MA, EE. UU.: Addison-Wesley, 2012.

[26] R. N. Taylor, N. Medvidović y E. M. Dashofy, *Software Architecture: Foundations, Theory, and Practice*. Hoboken, NJ, EE. UU.: Wiley, 2010.

[27] *IEEE Recommended Practice for Architectural Description of Software-Intensive Systems*, IEEE Std 1471-2000, 2000.

[28] M. Cohn, *User Stories Applied: For Agile Software Development*. Boston, MA, EE. UU.: Addison-Wesley, 2004.

[29] Project Management Institute, *Guía de los fundamentos para la dirección de proyectos (Guía del PMBOK)*, 6.ª ed. Newtown Square, PA, EE. UU.: PMI, 2017.

[30] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[31] N. Provos y D. Mazières, "A future-adaptable password scheme," en *Proc. USENIX Annual Technical Conf.*, Monterey, CA, EE. UU., 1999, pp. 81–91.

[32] H. Krawczyk, M. Bellare y R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF, RFC 2104, 1997.

[33] Universidad de los Llanos, "Proyecto final de curso: Optimización de la red logística de recepción y procesamiento de residuos agropecuarios utilizando modelos de redes y algoritmos genéticos," Programa de Ingeniería de Sistemas, Optimización, Villavicencio, Colombia, 2026.

[34] M. A. Gutiérrez Cárdenas, "Introducción a los estilos arquitectónicos en desarrollo de software," Ingeniería de Software I, Universidad de los Llanos, Villavicencio, Colombia, notas de clase, 2022.